

AI Platform & Backend Infrastructure

*Course companion and
interview workbook*

Docker and systems · TypeScript · MCP
Kubernetes · Helm · OAuth · Kyma

THREE STAGES / TWO TRACKS / ONE EVOLVING SERVICE

84 questions with complete answer keys

6 integrated exercise milestones

3 stage interviews and a combined final interview

Continue the courses you already started. Use this workbook to explain the mechanisms, diagnose failures and demonstrate the same service through every layer.

Research edition · 22 September 2026

Editable reference text, source bibliography and full project solutions.

Mock data only. No employer configuration is assumed. Executed checks and untested integrations are identified explicitly.

Contents

0. Using the workbook.....3

1. Docker and the systems it runs on.....7

2. TypeScript and MCP foundations.....20

3. Stage 1 practice, checkpoint and mock interview.....32

4. Kubernetes: desired state, traffic and diagnosis.....36

5. Implement the MCP service and understand its auth boundary.....49

6. Stage 2 practice, checkpoint and mock interview.....61

7. Main track: Helm and the Kyma boundary.....66

8. Parallel track: MCP authorization and client integration.....80

9. Stage 3 practice, checkpoint and mock interview.....94

10. Final integrated assessment.....100

11. Targeted revision, commands and glossary.....106

12. Source bibliography and version notes.....111

13. Coverage audit and evidence limits.....122

Appendix A. Complete reference project.....126

Use the navigation pane for chapter and question headings. Within each teaching chapter, attempt the question block before moving to its nearby answer key.

Progress states: Not attempted / Needed hints / Answered independently / Demonstrated practically.

0. Using the workbook

0.1 A practical study loop

Continue the Docker course you already started. After a relevant course topic, read the matching teaching section, close it, attempt its questions, and only then open the answer key. Explain one answer aloud before looking at the model wording. The purpose is to build an explanation you understand, not to impersonate the text.

Keep the approximate 70% main-track and 30% parallel-track allocation from your roadmap. A large parallel-track chapter does not mean another full course each evening. For a 90-minute session, a sensible split is 60 minutes of the main course and its lab, and 30 minutes of one parallel topic. Stop at a subsection or a group of two questions. The optional exercises can occupy several sessions; they are milestones of the same service, not deadlines.

Mark every question or demonstration with one of four states: **Not attempted / Needed hints / Answered independently / Demonstrated practically**. Record a date and one piece of evidence, such as a command output, a diagram you drew, or a corrected manifest. “Watched the lesson” and “can diagnose the failure” are different accomplishments.

You already know programming, SQL and backend work. Python comparisons therefore explain unfamiliar language mechanisms without reteaching what a function is. Infrastructure explanations start from observable processes, files and requests. Optional stretch questions are not prerequisites for continuing.

0.2 Reading the evidence labels

A — Published interview question. The substance is paraphrased from a named public guide or report. A hiring guide proves publication, not that a company actually asked it. A candidate report remains anecdotal. Answers here are checked against primary technical sources rather than copied from the guide.

B — Documentation-based knowledge check. This is practice material built from official documentation. It is not presented as a question reported from an employer interview.

C — Author-designed practical scenario. The situation, observations and exercise are original teaching material. Sources support the technical mechanisms, not a claim that the incident actually happened.

Question priorities are **Core**, **Useful**, or **Stretch**. Each question also identifies the skill being tested. “Likely follow-up” means an author-designed continuation that would probe the same reasoning, not a prediction about a particular employer.

The bibliography uses stable source IDs. A citation such as [D07](#) points to the relevant source and its limitations. Source dates are not invented when a live page does not provide one. The supplied brief controls the syllabus; public research verifies the technical content.

0.3 Version boundary: read before copying code

The research date is **22 September 2026**. The latest MCP specification found was **2026-07-28**, with the TypeScript SDK v2 available. This workbook deliberately keeps its runnable reference design on

the maintained **SDK v1.30.0 / 2025-11-25 protocol family**, so that course-era examples form one internally coherent lab. This is a compatibility choice, not advice to downgrade or upgrade a work project. [M08](#) [M09](#) [M10](#)

The distinction is significant. The July 2026 protocol removes the old initialization/session model and introduces per-request capability/version information and server discovery changes. Consequently, the `initialize` → `initialized` → `tools/list` traces in Chapters 2 and 5 are explicitly **v1-family lab traces**, not universal descriptions of the latest MCP. Do not paste v2 package imports into this v1 project or copy the old handshake into a v2 implementation. Start any real code review by checking the dependency lockfile, client version and negotiated protocol. [M04](#) [M09](#)

The project uses Node **22.x**, npm **10.x**, TypeScript **5.8.3**, SDK **1.30.0**, Zod **3.25.76**, and jose **6.1.3** as its selected dependency baseline. The authoring environment actually had Node **22.16.0** and TypeScript **5.8.3**. Exact package selections are not a claim that these are the newest releases. The initial dependency resolution must be performed locally; no fictitious lockfile is supplied.

Helm command examples target **Helm 4.3.0**. Some course material and parts of the template guide use older CLI conventions. In particular, verify rollback-on-failure and waiting flags rather than assuming a Helm 3 command is unchanged. Kubernetes examples use stable v1 and apps/v1 APIs. Choose a supported local kind release and its documented node image, record the resulting Kubernetes version, and check the installed Helm/Kubernetes compatibility policy rather than combining arbitrary newest versions. [H06](#) [H08](#) [H10](#) [K15](#)

Kyma exposure discussion uses **APIRule v2**, but the installed module, gateway and mesh configuration must be inspected in an approved environment. The document does not assert what your employer has installed. The local identity-provider exercise selects **Keycloak 26.7.4** from the downloads page and uses development mode only on loopback. [Y01](#) [Y02](#) [O14](#)

0.4 What was and was not executed

The native HTTP starter was run against six request checks, followed by a clean SIGTERM shutdown check. The dependency-free TypeScript domain and its four automated tests were compiled and executed. Seven configuration-boundary checks also passed. JSON and ordinary YAML received structural checks, and the two auth shell files passed Bash syntax checks.

The authoring environment did not provide Docker, Kubernetes, Helm or a running identity provider, and package-registry access was unavailable. Therefore the MCP SDK integration, full dependency build, Docker images, cluster behavior, Helm rendering, Keycloak flow and Kyma configuration are **reviewed reference examples, not claimed successful deployments**. Every lab includes commands for you to verify them. Treat sample outputs as expected observations, never as logs from a test that was not performed.

This boundary matters: passing a pure function test cannot establish transport interoperability, correct identity-provider policy, or a safe rollout. The verification checklist deliberately progresses through these separate layers.

0.5 Course-to-workbook topic map

All mappings are **topic-level**, not assertions about the exact numbering in your purchased Udemy editions. Public syllabuses were reviewed; paid video contents were not watched or inferred.

Roadmap stage and track	Course/reference retained	Attempt after	Workbook
1 main	[1] KodeKloud Docker C01	Containers, images, run/build, ports, storage, networking	Chapter 1; Exercise E1
1 main	[2] DevOps prerequisites C02	Selected Linux, DNS, IP/ports, HTTP/TLS lessons	Sections 1.1 and 1.4; SYS questions
1 parallel	[3] Beginner's TypeScript C03	Types, objects, unions, optional values	Sections 2.1–2.3; Exercise E2
1 parallel	[4] HF MCP Unit 1 C04	Host/client/server and protocol primitives	Sections 2.4–2.6
2 main	[5] KodeKloud Kubernetes C06	Pods, YAML, controllers, Services and labs	Chapter 4; Exercise E4
2 main	[8] Kubernetes focused references K07 K08 K09 K10 K11 K12 K13 K14	Configuration, resources, probes, storage, access	Sections 4.3–4.6
2 parallel	[3] TypeScript plus relevant JavaScript T03 T04 T05 T06	Imports, promises, async errors, npm	Sections 5.1–5.2
2 parallel	[4] selected HF Unit 2 C05 , [6] SDK/server docs M01 M07	Server registration and compatible client	Sections 5.3–5.4; Exercise E3
2 parallel	[7] MCP authorization walkthrough M12	OAuth roles, discovery and token checks	Sections 5.5–5.6
3 main	[9] KodeKloud Helm C07 and Helm guide H01 H02 H03 H04 H05 H06 H07 H08 H09	Charts, values, templates, releases	Chapter 7; Exercise E5
3 main	Kyma documentation Y01 Y02 Y03	Only after the ordinary Service/Deployment is understood	Section 7.7 and KYMA questions
3 parallel	[7] MCP authorization and standards M05 O01 O02 O03 O04 O05 O06 O07 O08 O09 O10 O11 O12 O13	Metadata, registration, validation, permissions	Chapter 8; Exercise E6
3 parallel	Actual client/plugin documentation for your project	Packaging and configuration review	Section 8.8; INT questions

The prerequisite course is a gap-filler, not an extra certification. The HF Unit 2 technology choices are illustrative; adopting Gradio or Hugging Face hosting is not required. The official authorization

tutorial may use a different SDK revision from your project, so compare imports and flow rather than replacing work code to match a tutorial.

0.6 What current roles suggest—and what they do not

A small purposive sample of employer descriptions was checked, not a statistically representative labor-market survey. Canonical's EMEA-eligible distributed-systems testing role emphasizes Python/Go, Linux, networking, test design and diagnosis. Polarise's Germany platform role explicitly combines Docker/Kubernetes, API platforms, DNS/networking and OAuth/OIDC/JWT. Preisenergie's Munich junior role combines Python APIs, TypeScript and a Docker/Kubernetes environment, while also expecting frontend work outside this block. These sources support practicing transferable implementation and diagnostic skills; they do not establish interview-question frequency. [J01](#) [J02](#) [J03](#)

A senior AI-backend description at credium also ties model integration to backend reliability, observability and containerization. It is an adjacent direction, not proof that you already meet that senior role's requirements. Broader requirements in advertisements—Terraform, advanced pipelines or frontend frameworks—do not silently expand this first block. My prioritization judgment is to become able to explain and operate one small service end to end before adding more products. [J04](#)

The public interview reports reviewed include a Service-to-Pod question and a much deeper senior Kubernetes discussion. Their useful lesson is that role depth varies. They are not evidence that you must learn etcd quorum internals before completing beginner Kubernetes. [I08](#) [I09](#)

0.7 Starting diagnostic—attempt before Chapter 1

Write a two-sentence answer to each prompt without searching. This is a placement aid, not an additional exam.

DG-01. A server works on your laptop but fails after containerization. Name three boundaries that changed.

DG-02. Does a TypeScript interface validate JSON received over a network?

DG-03. What does an HTTP 401 tell you that a TCP connection timeout does not?

DG-04. What is the difference between “the process is alive” and “the service should receive traffic”?

DG-05. What is one fact you must establish before choosing an MCP SDK example?

Diagnostic answers. DG-01: process environment/filesystem, listening interface and network namespace, and published ports are valid choices; missing runtime dependencies is another. DG-02: no, runtime data needs explicit validation. DG-03: an HTTP component responded with an authentication challenge/error; a timeout may occur before any application response and does not identify its cause. DG-04: a running process can still be initializing or draining. DG-05: the actual SDK/protocol and client compatibility, not simply the latest tutorial date. [D06](#) [D07](#) [T01](#) [T11](#) [K09](#) [M09](#)

Award one point for each materially correct answer. Four or five means you can move briskly through explanations you can demonstrate. Two or three means use the worked examples. Zero or one means prioritize Chapter 1's boundaries and Chapter 2's validation distinction; it does not mean restart your Docker course.

STAGE 1 — FINISH DOCKER AND BUILD THE FOUNDATIONS

1. Docker and the systems it runs on

Stage / track: 1, main. **Course:** existing Docker course [C01](#), selected systems lessons [C02](#). **Attempt after:** images/containers, basic run commands, Dockerfiles, networking and storage. **Prerequisites:** read a small server, use a terminal, distinguish source code from a running program. **Outcomes:** construct an image, explain its execution and network boundaries, preserve intended data, and diagnose an unreachable or exiting container. Do not introduce Kubernetes into this chapter.

1.1 Begin with the process, not the logo

The problem containerization addresses is packaging an application and its userspace dependencies with a predictable execution environment. A container is not a miniature computer containing an independent kernel. For Linux containers, the host kernel runs isolated processes; namespaces change their view of resources, while resource controls limit or account for usage. Docker Desktop adds another boundary because the Linux environment is commonly inside a VM on a non-Linux host. That is why “localhost on Windows,” “localhost inside the Linux VM,” and “localhost inside the container” should not be casually equated. [D01](#)

An **image** is the packaged filesystem and execution configuration from which a container is created. A **container** adds a particular running or stopped instance and a writable layer. Stopping the instance is not the same as deleting it; recreating it from an image is not the same as restoring arbitrary files written into the old instance. A registry stores and distributes images. The application process, not the existence of an image, determines whether a request can be handled. [D02](#)

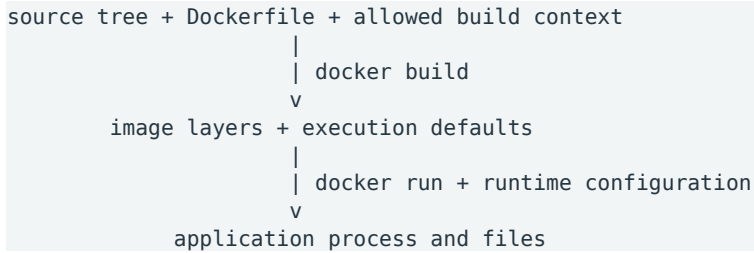
Before Docker, understand a few Linux observations. `pwd` answers where the shell currently is. `ls -l` shows file ownership and permission bits. `ps` shows processes, and `ss -ltn` shows listening TCP sockets where the tool is available. A command’s exit status is separate from its text output: `echo $?` reports the preceding command’s status in Bash. A pipeline connects one program’s stdout to another program’s stdin; stderr remains separate unless explicitly redirected. These are selected systems-course topics, not a requirement to memorize every flag. [C02 I03](#)

In this lab, `node server.mjs` must remain alive to accept requests. A successful build says nothing about whether that process can load its data file at startup. A shell command such as `PORT=3000 node server.mjs` sets the variable for that process invocation; it does not automatically reconfigure a previously running container. Quoting “`$PWD/data`” preserves a path containing spaces. On Windows, the lab commands assume Bash inside WSL2 rather than PowerShell syntax.

Workplace connection. When someone says “the service is down,” first turn that into an observable statement: no process, no listener, no route, an HTTP failure, or an unsuccessful business operation. These are different failures with different owners and evidence.

1.2 Build time is a different world from run time

A Docker build consumes a **build context**, executes build instructions and produces an image. The context is the set of files made available to the builder—not the entire host filesystem. `COPY` can only obtain suitable files from that context or an explicitly named build stage. `.dockerignore` removes files from the context before ordinary `COPY` instructions use them. This can prevent accidental inclusion of credentials and also accidentally omit a runtime asset. [D06](#)



The first arrow constructs an artifact. The second starts an instance. A database becoming reachable at runtime does not make it available during RUN; a variable supplied with `docker run -e` does not retroactively change a completed build.

Docker reuses eligible cached build steps. A useful arrangement copies dependency manifests before frequently changed application source, installs dependencies, then copies source. Editing a handler can then avoid repeating dependency installation. Copying the entire source tree before installation unnecessarily couples the dependency step's cache key to unrelated source changes. It is not magic: a changed manifest, base image, command or preceding layer can still invalidate reuse. [D04](#)

```

COPY package.json package-lock.json ./
RUN npm ci
COPY src ./src
RUN npm run build
  
```

This fragment becomes relevant to the TypeScript implementation in Stage 2. The Stage 1 HTTP starter uses only Node built-ins, so its first Dockerfile needs no dependency installation. Do not add npm simply to make the Docker exercise look more complicated.

RUN executes during construction. COPY adds files. CMD provides a default command or arguments for container startup. ENTRYPOINT defines the executable-oriented entry point and is often combined with a CMD supplying default arguments. JSON exec form avoids an unnecessary shell and gives clearer argument and signal behavior. `RUN node server.mjs` would try to run the server during the build, not start it each time a container starts. [D05](#)

1.3 Files, dependencies and reproducibility

A multi-stage build separates a compilation environment from the runtime artifact. The final image must still contain every file the running program requires. TypeScript compilation does not automatically copy arbitrary JSON, templates or helper scripts. In the reference service, `dist/server.js` reads `../data/status.json`; the Dockerfile therefore copies `data/` explicitly. A bundler can change which files are embedded, but “bundled” does not establish that every dynamic filesystem access is satisfied. [D11](#)

Use a small diagnostic chain for a missing file: identify the exact runtime path, establish the working directory, inspect whether the file entered the context, inspect the relevant COPY, and finally inspect the final stage. A file present in the build stage is not necessarily present in the runtime stage. Fix the smallest responsible boundary rather than copying the entire repository into the image.

An image tag is a name that can move. This includes a tag that looks like a version number. A digest identifies particular image content. Recording a digest helps establish which artifact was deployed; it does not freeze external databases, runtime environment variables or every upstream build input. The reference Dockerfiles use `node:22-bookworm-slim` for a convenient local baseline, not a claim of bit-for-bit reproducibility. For a reviewed deployment, resolve and record an approved base-image digest and establish an update process. [D03](#)

Basic image hygiene includes a deliberately scoped context, trusted base image, necessary runtime packages only, review of dependency changes, and a non-root application user where practical. Running as non-root limits some damage but does not grant the process access to a root-owned file. File ownership and directory traversal permissions still matter. A read-only root filesystem is useful only when the application has an appropriate plan for legitimate writable paths. [D03](#) [D05](#)

Secrets should not be copied into the image and deleted in a later layer. Build secrets belong in a build-time secret mechanism when needed; runtime secrets must be supplied through the environment's approved mechanism. ARG and ENV are not secure secret stores. An environment variable is a configuration transport, not proof that a value is confidential. [D12](#)

1.4 Follow the request across network boundaries

An IP address locates a network interface; a TCP port identifies an endpoint on that address; DNS resolves a name to address information. HTTP adds request methods, paths, headers and responses over a transport. HTTPS adds TLS protection and peer-certificate validation. A TLS certificate does not authorize an MCP tool, and DNS success does not prove that the application is listening. [C02](#) [T11](#) [T12](#)

```
host curl -> 127.0.0.1:8080 [published host port]
          -> container interface:3000
          -> Node listener on 0.0.0.0:3000
          -> GET /healthz
```

Here, `docker run -p 127.0.0.1:8080:3000 ...` makes the service reachable through host loopback only. Inside the container, a listener bound only to `127.0.0.1:3000` generally does not accept traffic addressed to the container interface through that published mapping. `0.0.0.0` means all IPv4 interfaces in that process's network namespace, not "the public internet everywhere." Binding broadly inside the container and publishing narrowly on the host are compatible choices. [D07](#)

`EXPOSE 3000` records an intended container port; it does not create the host mapping. Two containers on a user-defined bridge network can use Docker-provided name resolution for one another. Neither should use its own `localhost` to refer to its neighbor. The host's DNS settings, a container's DNS behavior and the upstream hostname are separate observations when a dependency lookup fails. [D08](#)

A **connection refusal** commonly means a reachable endpoint rejected the connection, often because no listener exists there. A **timeout** means the operation did not finish within its deadline; packet filtering, routing, congestion, DNS or an unresponsive peer may be involved. An **HTTP 500** means some HTTP component responded with an application/server-side error. These symptoms narrow the investigation but do not prove a unique root cause. Use `curl -v --max-time 3` to separate connection, TLS and HTTP evidence, without exposing authorization headers in shared logs. [T10](#) [T11](#)

1.5 Persistence and lifecycle

A bind mount exposes a chosen host path in the container. It is convenient for development, but couples the container to that path and its permissions. It can also obscure files already present at the mount point in the image. A Docker-managed volume decouples persistent data from the lifecycle of a particular container. Neither mechanism automatically supplies application-level backups, transaction consistency or safe multi-writer behavior. [D09](#) [D10](#)

Keep this status service stateless: the mock JSON is an immutable input packaged with the image. To learn mounting, replace that input using a read-only bind mount. Do not invent a database just to justify a volume. A writable volume is appropriate when the workload genuinely owns persistent state; an application log that the platform already collects from stdout/stderr is not automatically such a case.

Container restart policies react to process/container lifecycle conditions; they are not a complete health management system. A process can be running but unable to serve useful traffic. Conversely, repeatedly restarting an application with an invalid configuration only hides the original startup message in a stream of restarts. Read the exit state and recent logs before changing the policy. [D13](#)

Graceful shutdown is a bounded process: stop admitting work, complete or cancel in-flight operations, close resources, and exit before the platform's deadline. The reference server handles SIGTERM and SIGINT, marks itself unready, stops accepting new connections, and enforces a ten-second deadline. Docker's stop operation sends the configured stop signal and later forces termination if necessary. A shell wrapper or a process that ignores the signal can prevent the intended behavior. [D14](#) [T09](#) [T10](#)

Useful addition: evidence-based debugging. This belongs here because deployment work is often diagnosis rather than writing new Dockerfiles. Before a change, save the observed command, container ID/image, relevant configuration names, error and hypothesis. Do not paste secrets. After the change, rerun the same check and one adjacent regression check. That is a small, reproducible engineering record rather than a vague "Docker was broken."

1.6 Questions—attempt before the answer key

DKR-01 — Image, registry and running instance

Provenance: A · published guide, adapted from I01 question 6. **Sources:** [I01](#) [D01](#) [D02](#). **Priority:** Core. **Type:** comparison.

Distinguish an image from an image registry. Then explain where a running container fits.

Testing: Artifact, distribution and execution boundaries.

DKR-02 — States and restart behavior

Provenance: A · published guide, adapted from I01 question 5. **Sources:** [I01](#) [D13](#) [T09](#). **Priority:** Core. **Type:** explanation.

What can running, exited and restarting states tell you, and when would you use a restart policy?

Testing: Process lifecycle versus application health.

DKR-03 — Network driver and names

Provenance: A · published guide, adapted from I01 question 3. **Sources:** [I01](#) [D08](#). **Priority:** Core. **Type:** explanation.

What is the usual default Docker network driver, and how would two local containers find one another?

Testing: Container-network scope, DNS and localhost.

DKR-04 — A volume mount means what?

Provenance: A · published guide, adapted from I01 question 11. **Sources:** I01 D09 D10. **Priority:** Core. **Type:** configuration review.

Explain `--mount type=bind,src="$PWD/data",dst=/app/data,readonly`. When would a named volume be different?

Testing: Filesystem mapping, persistence and permissions.

DKR-05 — The latest-tag trap

Provenance: A · published guide, adapted from I01 question 9. **Sources:** I01 D03. **Priority:** Core. **Type:** design.

Why is deploying an image tagged latest risky? Is a version-looking tag enough?

Testing: Artifact identity and reproducibility.

DKR-06 — A suddenly slow build

Provenance: C · original troubleshooting scenario. **Sources:** D04 D06 T07. **Priority:** Core. **Type:** troubleshooting.

After moving `COPY . . above RUN npm ci`, every handler edit makes the build much slower. Diagnose without assuming the registry is slow.

Testing: Build-cache dependency reasoning.

DKR-07 — COPY, RUN, CMD and ENTRYPOINT

Provenance: B · documentation-based practice. **Sources:** D05. **Priority:** Core. **Type:** comparison.

Explain the four directives and predict the mistake in `RUN node server.mjs` as the only server-start instruction.

Testing: Construction versus execution semantics.

DKR-08 — The application has no configuration

Provenance: C · original configuration scenario. **Sources:** D05 T09. **Priority:** Core. **Type:** troubleshooting.

The host process sees `API_MODE=mock`, but the container reports it missing. Where would you investigate?

Testing: Environment inheritance and explicit configuration.

DKR-09 — Missing runtime support file

Provenance: C · original troubleshooting scenario. **Sources:** D06 D11. **Priority:** Core. **Type:** troubleshooting.

Compilation succeeds, but startup reports that `/app/data/status.json` does not exist. Give an evidence-led investigation.

Testing: Build context, final-stage contents and path resolution.

DKR-10 — Separate build and runtime dependencies

Provenance: B · documentation-based practice. **Sources:** D11 T07 T08. **Priority:** Useful. **Type:** design.

Why use a multi-stage TypeScript image, and what can go wrong when removing development dependencies?

Testing: Minimal runtime packaging without deleting required components.

DKR-11 — Non-root is not a secret store

Provenance: B · documentation-based practice. **Sources:** [D03](#) [D05](#) [D12](#). **Priority:** Core. **Type:** configuration review.

A Dockerfile copies a credential, deletes it later, and runs as a non-root user. Is that a satisfactory security design?

Testing: Layer persistence, least privilege and distinct controls.

DKR-12 — A bounded graceful shutdown

Provenance: B · documentation-based practice. **Sources:** [D14](#) [T09](#) [T10](#). **Priority:** Core. **Type:** design.

What should the service do when Docker sends SIGTERM? Why is an unlimited graceful wait unsafe?

Testing: Signal handling, draining and deadlines.

SYS-01 — Shell versus Bash

Provenance: A · published guide, adapted from I03 question 1. **Sources:** [I03](#) [C02](#). **Priority:** Core. **Type:** explanation.

What is a shell, and is Bash the only shell? Why does this matter for the lab commands?

Testing: Command interpretation and environment assumptions.

SYS-02 — A diagnostic pipeline

Provenance: A · published guide, adapted from I03 recursive-search question. **Sources:** [I03](#) [C02](#) [T09](#). **Priority:** Useful. **Type:** prediction.

How would you count matching error lines in a local log file without losing sight of exit status or accidentally sharing secrets?

Testing: Pipes, stdout/stderr and diagnostic restraint.

1.7 Answer key

DKR-01 — Model answer

Interview-ready answer. An image is a packaged filesystem plus execution configuration. A registry is a service that stores and distributes images. A container is a particular instance created from an image, with its own runtime state and writable layer. Pulling an image does not start its application, and stopping a container does not remove the registry copy. I would use an image identifier to establish what code was packaged, then inspect the container to establish what is actually running.

Deeper explanation. This distinction prevents investigating the wrong object. A correct registry artifact can still produce a failing process because runtime configuration or networking differs. Linux containers share a kernel; virtual machines normally provide a separate guest-kernel boundary. [I01](#) [D01](#) [D02](#)

Example. `docker image inspect status-lab:0.1.0` inspects the artifact; `docker inspect status-lab` inspects the named container.

Common weak answer. “A registry is a running image” confuses distribution with execution. “A container is a lightweight VM” omits the kernel boundary.

Follow-up with answer. Does stopping the container lose its writable files? Not merely by stopping it. Removing and replacing that container loses its unpersisted writable layer; a separately retained volume has its own lifecycle.

DKR-02 — Model answer

Interview-ready answer. Running means the container process is running, not that the application is healthy. Exited means the main process stopped; I would inspect its exit status and logs. Restarting means repeated lifecycle attempts may be hiding a consistent startup failure. A restart policy can restore a process after suitable failures, but it cannot fix a missing file or bad configuration. I would first understand why the process stopped, then choose a policy appropriate to the workload.

Deeper explanation. A one-shot job can legitimately exit zero; a long-lived server normally should not. Restart policy semantics depend on the selected policy and manual-stop behavior. Do not infer useful service availability from a green-looking process state. [I01](#) [D13](#) [T09](#)

Example. `docker ps -a`, `docker inspect status-lab`, and `docker logs --tail 80 status-lab` provide different pieces of evidence.

Common weak answer. “Running means no issue” is too strong. Restarting forever is not a diagnostic strategy.

Follow-up with answer. Would you restart on every HTTP 500? No. A failed request can be handled without process corruption. Investigate error rate and cause; restarting can interrupt unrelated successful requests.

DKR-03 — Model answer

Interview-ready answer. For ordinary standalone Linux Docker containers, bridge networking is the usual default. For a small multi-container application, I would create a user-defined bridge and

attach both containers. They can then use the configured container names through Docker's name resolution, while each container's localhost still refers to itself. Publishing a port is needed for the intended host access, not necessarily for communication between containers on that same network. I would avoid relying on a changing container IP.

Deeper explanation. A user-defined bridge provides isolation and name-resolution behavior that differs from the legacy default bridge. Host networking is a different choice and reduces this network-namespaces separation; its availability also varies across host environments. [I01 D08](#)

Example. `docker network create workbook-net` creates the named lab network; attach containers with `--network workbook-net`.

Common weak answer. "Use localhost:5432 from the application to reach any database container" names the wrong network namespace.

Follow-up with answer. Would successful DNS prove connectivity? No. It establishes name resolution. You still need the correct port, a listener, permitted routing and a successful application response.

DKR-04 — Model answer

Interview-ready answer. This exposes the host data directory at `/app/data` inside the container, read-only to that container. It is a bind mount, so the host path and its permissions matter. It also hides any image files at that mount point while mounted. A named volume is managed by Docker and can outlive a particular container without being tied to this explicit project path. Neither option automatically creates a backup or resolves concurrent application writes.

Deeper explanation. The direction matters: the source is the host-side path and the destination is the process-visible path. Read-only mounting protects against this mount being used to write, but does not repair incorrect file contents or grant missing read permissions. [I01 D09 D10](#)

Example. Mount the starter's alternative status JSON directory, then repeat `GET /status?service=api` to observe the changed mock input after restart.

Common weak answer. "Mounting merges the host directory with the image directory" is misleading; the mount obscures the underlying location.

Follow-up with answer. Why not persist every application directory? It weakens artifact predictability and can obscure dependencies. Persist deliberately owned state; package immutable application files with the image.

DKR-05 — Model answer

Interview-ready answer. The risk is that a tag can be moved to different content, so the same deployment declaration may later select a different artifact. A version-looking tag is better communication but is still mutable unless the registry's policy prevents changes. I would record the tested image digest for a controlled deployment and keep a deliberate update process. Pinning a digest identifies image content; it does not guarantee that runtime configuration, external services or the overall application behavior stayed unchanged.

Deeper explanation. There are two useful records: a human-oriented version and a content identifier. Dependency lockfiles and base-image provenance address different build inputs. Reproducibility is a chain of recorded inputs, not a special property of one word in a tag. [I01 D03](#)

Example. An image reference of the form `repository@sha256:...` selects content; use an actual resolved digest, never the ellipsis.

Common weak answer. “A semantic version tag is immutable” assumes a registry policy that may not exist.

Follow-up with answer. Does digest pinning eliminate security maintenance? No. It prevents silent changes but also requires explicitly rebuilding or selecting a patched image when an update is needed.

DKR-06 — Model answer

Interview-ready answer. I would compare the old and new Dockerfiles and inspect which steps actually lost cache hits. Moving the full source copy earlier makes unrelated source edits affect the dependency-install step. If that is the observed change, I would copy the dependency manifests first, run the locked installation, and copy source afterward. Then I would repeat a no-change build and a source-only edit to verify the intended reuse. A separate network problem remains possible, but the changed cache boundary is the first testable hypothesis.

Deeper explanation. Cache reuse depends on inputs and prior steps. Timing alone cannot prove the cause. A deliberately changed package-lock should still invalidate dependency installation, which is correct behavior rather than a regression. [D04 D06 T07](#)

Example. Expected experiment: a source-only edit reuses the install step; a lockfile edit causes it to execute again. Record observations rather than inventing a speed-up percentage.

Common weak answer. “Docker randomly forgot the cache” ignores the explicit dependency introduced by instruction order.

Follow-up with answer. Could a cache mount help too? Yes, it can reuse package-download data even when installation runs, but it does not replace correct instruction ordering or lockfile discipline.

DKR-07 — Model answer

Interview-ready answer. `COPY` adds files while constructing the image, and `RUN` executes a command during that construction. `CMD` defines a default startup command or arguments. `ENTRYPOINT` defines the executable-oriented entry point, often with `CMD` supplying default arguments. `RUN node server.mjs` starts the server during the build step; it does not arrange for the server to start whenever a container is created. For this simple service I would use the JSON-form `CMD` with `node` and the script path.

Deeper explanation. Shell-form startup can introduce a shell and change signal forwarding and argument expansion. Exec form treats each array element as an argument, which is clearer for a single long-lived Node process. [D05](#)

Example. `CMD ["node", "server.mjs"]` runs Node at container startup. It does not expand shell variables inside the strings.

Common weak answer. “`RUN` and `CMD` are equivalent ways to execute commands” omits when they execute and what the resulting image records.

Follow-up with answer. When is `ENTRYPOINT` useful? For an image that behaves like a specific executable while allowing argument overrides. Avoid elaborate wrappers unless they perform a necessary, reviewed startup task.

DKR-08 — Model answer

Interview-ready answer. I would establish how the host process obtained the variable, then inspect how the container was actually started. Docker does not automatically copy every environment variable from my shell into a new container. I would pass the intended non-secret value explicitly, validate it at application startup, and recreate the container with the corrected configuration. I would also check spelling and quoting. Rebuilding the image is unnecessary when the design intentionally reads that setting at runtime.

Deeper explanation. ARG supplies a build-time input; ENV records an image default, and runtime options can override that default. A .env file is not automatically consumed by every runtime or framework simply because it exists beside the source. [D05 T09](#)

Example. `docker run -e API_MODE=mock . . .` supplies a value. Avoid printing a complete environment when it may contain credentials.

Common weak answer. “Put all configuration in the Dockerfile” produces environment-specific artifacts and risks leaking secrets.

Follow-up with answer. How should invalid configuration fail? Early, with a useful message naming the invalid setting but not revealing its secret value; do not silently switch production authentication off.

DKR-09 — Model answer

Interview-ready answer. I would confirm the exact path from the error and inspect the final image, not only the source tree or build stage. Then I would check whether .dockerignore excluded the file, whether COPY included it, and whether the final multi-stage image retained it. I would also verify how the application resolves relative paths. The smallest fix might be an explicit data-directory copy or a corrected path, not shipping the entire repository. Finally I would start the image and exercise the endpoint that reads the data.

Deeper explanation. TypeScript normally transforms TypeScript files; it does not make an arbitrary filesystem dependency disappear. A dynamic read is a runtime requirement even when the main application bundle exists. [D06 D11](#)

Example. `docker run --rm --entrypoint node IMAGE -e "console.log(require('fs').existsSync('/app/data/status.json'))"` tests final-image presence without starting the server.

Common weak answer. “The build passed, so Docker must have the file” conflates successful compilation with complete runtime packaging.

Follow-up with answer. What prevents recurrence? An image-level smoke test that starts the final artifact and exercises a code path using the asset, plus an explicit packaging rule reviewed with the path change.

DKR-10 — Model answer

Interview-ready answer. A build stage needs tools such as the TypeScript compiler, while the runtime stage needs emitted JavaScript, production libraries and runtime assets. A multi-stage build lets me keep compilation tools out of the final image. However, I must classify dependencies correctly: the MCP SDK and validation library are runtime dependencies, not merely development

tools. I would test the final image after pruning, because a successful build can depend on packages that were later removed.

Deeper explanation. Smaller is not automatically safer or more compatible. Native dependencies may need runtime shared libraries, and changing the base distribution between stages can break their assumptions. The reference uses the same Node image family in both stages. [D11 T07 T08](#)

Example. The reference builds with `npm ci`, compiles, then prunes with `npm prune --omit=dev` before copying `node_modules` into the final stage.

Common weak answer. “Everything imported by TypeScript belongs in `devDependencies`” confuses source language with runtime necessity.

Follow-up with answer. Would disabling all lifecycle scripts be harmless? No. Some packages use installation scripts to prepare native or generated artifacts. Evaluate the package behavior and verify the resulting runtime rather than toggling the flag blindly.

DKR-11 — Model answer

Interview-ready answer. No. Non-root execution is a useful runtime restriction, but it does not erase a secret that entered image layers or build metadata. I would remove the credential from the build context and use an appropriate build-secret mechanism only when the build genuinely needs it. Runtime credentials should come from the approved runtime secret mechanism. I would also verify ownership and permissions, because choosing a non-root user does not automatically make required files readable or create safe writable directories.

Deeper explanation. Security controls address different threats. Least privilege constrains a compromised process; careful artifact construction prevents distributing secrets; patching reduces known vulnerabilities. One control cannot be used as evidence that all others are unnecessary. [D03 D05 D12](#)

Example. Use `COPY --chown=node:node` for deliberately readable application files, and keep `.env` files out of the image context.

Common weak answer. “It is safe because the secret is deleted in the last layer” ignores earlier layer content.

Follow-up with answer. What about a leaked real credential already pushed? Removing a future image is insufficient. Follow the incident process, revoke or rotate the credential, and assess where the artifact and secret were distributed.

DKR-12 — Model answer

Interview-ready answer. I would stop admitting new work, mark the service as draining, let appropriate in-flight operations finish, close resources, and exit before the platform’s termination deadline. The wait must be bounded because a stuck request could otherwise prevent deployment or shutdown indefinitely. I would run Node directly with `exec-form` startup and test signal handling rather than assume it works. For this read-only mock service, a short deadline is sufficient; a state-changing service would need more careful cancellation and consistency rules.

Deeper explanation. The application deadline should fit inside the container or orchestrator grace period. Force termination is a final bound, not the preferred completion path. Some clients may still retry after a disconnect, so operation semantics matter. [D14 T09 T10](#)

Example. `docker stop --timeout 15 status-lab` allows the reference application's ten-second shutdown bound to finish first.

Common weak answer. "Catch SIGTERM and do nothing" prevents the default exit without providing a completion strategy.

Follow-up with answer. Does graceful shutdown guarantee no failed requests? No. Routing propagation, long-lived connections and dependency failures can still affect requests. Test the actual path and report measured behavior, not an absolute guarantee.

SYS-01 — Model answer

Interview-ready answer. A shell interprets commands, performs operations such as variable expansion and redirection, and starts programs with an environment. Bash is one shell, not the only one. This matters because syntax that works in Bash may not work unchanged in PowerShell or another shell. The workbook assumes Bash in WSL2 on Windows, or a compatible terminal on Linux or macOS. I would distinguish a shell parsing failure from a failure in the program the shell was meant to launch.

Deeper explanation. Quoting is part of argument construction, not decoration. A path containing spaces must remain one argument. An executable script additionally depends on its interpreter line, permissions and line endings in the target environment. [I03 C02](#)

Example. `printf "%s\n" "$PWD"` prints the current directory as one value. Use `bash auth/run-keycloak.sh` rather than assuming the executable bit survived a ZIP transfer.

Common weak answer. "The terminal is Bash" confuses the terminal application with the command interpreter running inside it.

Follow-up with answer. Why can a script work on Windows but fail in a Linux image? Check the interpreter path, CRLF line endings, execute/read permissions and required commands before blaming the container runtime.

SYS-02 — Model answer

Interview-ready answer. I would use a narrowly scoped text search and count the matching lines, while remembering that a line count is not necessarily an incident count. In Bash a simple pipeline normally reports the last command's exit status, so an earlier failure can be hidden unless I inspect it or use suitable pipe-failure handling. I would search only the relevant local log and redact sensitive values before sharing output. The command should answer a specific hypothesis rather than dump the entire machine environment.

Deeper explanation. A log may contain several lines per event or repeated retries. A successful `wc` process does not prove `grep` read the intended file. Observation quality depends on both command semantics and the meaning of the data being counted. [I03 C02 T09](#)

Example. `grep -F "request_failed" app.log | wc -l` counts matching lines. In a Bash script, consider `set -o pipefail` and handle a legitimate no-match result explicitly.

Common weak answer. "The pipeline returned zero, so every command succeeded" is not generally valid with default pipeline status behavior.

Follow-up with answer. What is a better production error count? A well-defined structured event or metric with a known time window and dimensions; use logs to investigate causes, not to invent precise rates from arbitrary multiline text.

1.8 Observable checkpoint and stopping point

After this chapter, can you explain image, container and registry without treating them as synonyms; build the starter from a known context; demonstrate the difference between EXPOSE and publishing; fix a loopback-bound listener; replace a mock data file with a read-only mount; and show why SIGTERM produces a controlled exit?

For each dimension, record an artifact. **Explain:** draw the two build/run arrows. **Use:** save a successful localhost request. **Diagnose:** keep before-and-after observations for one deliberate failure. **Justify:** explain why this mock service does not need persistent writable storage. Missing one detail calls for focused practice, not restarting the whole course. Stop here before switching to the parallel track.

2. TypeScript and MCP foundations

Stage / track: 1, parallel. **Courses:** Beginner's TypeScript [C03](#), selected JavaScript references [T01](#) [T02](#) [T03](#), HF MCP Unit 1 [C04](#). **Attempt after:** object types, functions, unions and optional values; MCP roles and primitives. **Prerequisites:** ordinary programming and the process/network boundaries in Chapter 1. **Outcomes:** read a small handler, distinguish type checking from runtime validation, identify host/client/server responsibilities, and trace a version-labelled tool call. Async implementation belongs in Stage 2.

2.1 TypeScript is a checked description of JavaScript

TypeScript helps catch inconsistent assumptions before execution. The compiler can reject passing a number where a string is expected, but the emitted program is still JavaScript. A Python comparison is useful: a Python type annotation does not, by itself, validate arbitrary JSON at runtime either. TypeScript adds a rich compile-time analysis; it does not turn untrusted input into trusted data. [T01](#)

```
interface StatusRequest { service: string; }
const request: StatusRequest = { service: 'api' };
// const wrong: StatusRequest = { service: 17 }; // compiler error
```

The interface describes a shape. An object with compatible required members may be assignable even if it was not created from a named class. This structural approach differs from treating every accepted object as an instance of a nominal class hierarchy. It is particularly convenient for JSON-shaped request and response objects, but you must still establish where those objects came from.

Use `unknown` at an uncertain boundary and narrow it before use. `any` mostly asks the checker to stop protecting that value. A type assertion such as `payload as StatusRequest` changes the compiler's treatment; it does not parse the payload or prove a field is present. The workplace implication is direct: an HTTP body, model-produced JSON, environment variable or file can violate the declared type. Validate the boundary, then let the internal code work with a trustworthy type.

2.2 Unions encode possibilities; narrowing establishes which one

A union is useful when the domain genuinely has alternatives. It should make impossible combinations harder to construct, not merely replace every type with an enormous list.

```
type Lookup =
  | { ok: true; status: 'operational' | 'degraded' }
  | { ok: false; reason: 'unknown_service' };

function describe(result: Lookup): string {
  if (result.ok) return result.status;
  return result.reason;
}
```

The `ok` field is a discriminator. Checking it tells the compiler which fields are available. This makes error paths explicit instead of relying on a nullable object plus an unrelated error string that could contradict it. Narrowing can also follow `typeof`, a null check or another supported guard. [T02](#)

An optional field may be absent. Under strict null checking, a caller must account for that absence rather than assuming every property access is safe. `value ?? fallback` selects the fallback for null or undefined; `value || fallback` also substitutes for other falsy values such as zero or an empty string. For a numeric retry count, zero might be a meaningful “do not retry” setting. Choose based on domain semantics, not style preference.

`const` fixes a binding, not all state reachable from it. A `const` object can still have mutable properties. TypeScript readonly communicates and checks intended use at compile time; freezing an object at runtime is a different mechanism. Avoid claiming either provides a security boundary against untrusted code.

2.3 Read one handler before building a framework

A useful handler review has four questions: what enters, what is checked, what side effect occurs, and what leaves? Here is the dependency-free core of the later MCP tool:

```
type Status = 'operational' | 'degraded';
type Table = Readonly<Record<string, Status>>;

function lookup(table: Table, name: string) {
  const value = table[name];
  if (value === undefined) return undefined;
  return { service: name, status: value, source: 'mock' as const };
}
```

The implementation does not call an LLM, fetch production data or modify a service. It reads a value from an already validated table. The eventual MCP adapter is responsible for checking an incoming tool argument and translating this domain result into the protocol's response shape. Keeping those responsibilities separate makes the domain easy to test without a network server.

Modules express code dependencies. With the reference project's ESM configuration, imports and exports connect files, while Node executes the emitted `.js` modules. The `.js` suffix in a TypeScript source import can therefore be intentional under the selected NodeNext configuration. Do not switch package type, module compiler settings and import conventions independently and expect identical runtime behavior. [T03](#)

Useful addition: small automated tests. Testing belongs here because a clear boundary makes infrastructure debugging much easier. If `lookup` has a tested contract, you can distinguish a domain bug from a transport failure. The supplied domain tests cover valid mock data, unknown services, invalid shapes and exact scope-name matching. Scope enforcement itself is taught later; the pure string-membership test does not require OAuth knowledge now.

2.4 MCP connects an application to capabilities

The **host** is the application coordinating a user/model experience. A **client** is its protocol component that communicates with a server. A **server** exposes capabilities such as tools, resources and prompts. Do not confuse "server" with "a remote VM": a server process can run locally and communicate over standard input/output. Nor is the MCP server necessarily the LLM host; our status server contains no model. [C04](#)

```
user <-> host application and model
      |
      | host policy selects/permitted tool interaction
      v
MCP client component
      |
      | version-compatible protocol transport
      v
MCP server -> input validation -> handler -> mock table
      |
      +----- result back to host
```

The host's decision to propose or approve a tool is separate from the server's responsibility to validate inputs and enforce its own permissions. A malicious caller can bypass a user interface and invoke a known operation directly; server-side checks must therefore remain authoritative.

Tools represent callable operations. A resource exposes data identified in the protocol's resource model. A prompt supplies a reusable interaction template. These primitives influence how a host discovers and presents capabilities; they are not three interchangeable REST endpoints. Describing an operation as read-only is useful metadata, but an annotation is not a substitute for implementation review or authorization. [M01](#) [M03](#)

2.5 Transport and protocol are different layers

With **stdio**, a host typically launches a local server subprocess and communicates using its stdin/stdout. Plain application logging on stdout can corrupt that protocol stream. Send ordinary diagnostic logs to stderr, or use the protocol's supported logging facilities where appropriate. The host controls the subprocess environment and needs the local executable and any supporting files. [M02](#)

With the lab's **2025-11-25 Streamable HTTP** transport, the application runs an HTTP endpoint and a compatible client exchanges MCP messages over it. Legacy HTTP+SSE examples are a different transport convention, not an interchangeable spelling. Streamable HTTP can use JSON responses or streams depending on the supported interaction; an ordinary GET `/healthz` is neither discovery nor a tool invocation. [M02](#)

For the v1-family lab, the client handles initialization/capability negotiation and then tool discovery and calls. You should be able to explain this trace without implementing JSON-RPC yourself:

```
client -> initialize / initialized      [v1-family lifecycle]
client -> tools/list -> available tool schema
client -> tools/call {name, arguments}
server -> validate -> authorize -> handler
server -> tool result or appropriate error
```

Initialization establishes protocol-level expectations. Tool discovery establishes which operations and schemas are advertised. Calling the handler is an application operation. A connection can pass the first step and fail later. The current July 2026 protocol changes this lifecycle, so use the version comparison in Section 0.3 before applying the trace to another project. [M04](#) [M09](#)

2.6 Local and remote packaging is an architectural decision

In a stdio integration, the executable, runtime, scripts and data needed by that process must exist where the host launches it. In an HTTP integration, the remote server's application files belong in its image/deployment; the client needs endpoint and authentication configuration and any locally executed companion code. A plugin package might contain instructions, configuration and scripts, but there is no universal rule that every plugin contains the MCP server itself.

For your work, replace guesses with an inventory: identify the entry point, follow an import into one handler, identify each file the handler opens, and mark which machine executes it. A remote URL does not automatically ship missing helper files, and a local instruction document does not automatically run its referenced shell command. Section 8.8 returns to this inventory after authentication is understood.

Core boundary check. A TypeScript type tells a developer what code expects. A runtime schema checks incoming data. An MCP tool schema advertises the calling contract. Authorization decides

whether this caller may use that operation on this resource. These cooperate but do not replace one another. [T01](#) [M03](#) [M05](#)

2.7 Questions—attempt before the answer key

TS-01 — What does a type actually guarantee?

Provenance: B · documentation-based practice. **Sources:** [T01](#). **Priority:** Core. **Type:** explanation.

What does a TypeScript interface guarantee about an HTTP JSON body at runtime?

Testing: Static checking versus boundary validation.

TS-02 — Shape compatibility

Provenance: B · documentation-based practice. **Sources:** [T01](#). **Priority:** Core. **Type:** comparison.

How do interfaces, type aliases and structural compatibility help model a small status request?

Testing: Choosing a readable data contract without unnecessary classes.

TS-03 — A union with a discriminator

Provenance: B · documentation-based practice. **Sources:** [T02](#). **Priority:** Core. **Type:** prediction.

Given Lookup in Section 2.2, why can the success branch read status but the failure branch read reason?

Testing: Narrowing and representation of alternatives.

TS-04 — Optional is not the same as falsy

Provenance: B · documentation-based practice. **Sources:** [T01](#) [T02](#). **Priority:** Core. **Type:** prediction.

A retries option may be zero. Compare `retries || 3` and `retries ?? 3`.

Testing: Defaults, absence and domain meaning.

TS-05 — Type safety without type gymnastics

Provenance: A · published hiring guide, adapted from I04. **Sources:** [I04](#) [T01](#) [T02](#). **Priority:** Useful. **Type:** design.

How would you balance useful type safety with development speed in this small service?

Testing: Pragmatic boundaries and maintainable types.

TS-06 — Explain a project without inventing experience

Provenance: A · published hiring guide, adapted from I04 project question. **Sources:** [I04](#) [T03](#). **Priority:** Core. **Type:** explanation.

Describe how you would organize a small TypeScript service from scratch, or accurately describe this learning project once completed.

Testing: Architecture explanation and honest ownership.

TS-07 — Read a handler boundary by boundary

Provenance: C · original code-reading exercise. **Sources:** [T01](#) [T02](#) [M03](#). **Priority:** Core. **Type:** configuration review.

Review lookup in Section 2.3. Which inputs are already trusted, and what must the future MCP adapter add?

Testing: Separating domain logic from validation and transport.

MCP-01 — Host, client and server

Provenance: B · documentation-based practice. **Sources:** [C04](#) [M01](#). **Priority:** Core. **Type:** explanation.

Identify the host, client and server when a desktop assistant calls the mock `read_status` tool.

Testing: MCP responsibilities without assuming a deployment topology.

MCP-02 — Tools, resources and prompts

Provenance: B · documentation-based practice. **Sources:** [C04](#) [M01](#) [M03](#). **Priority:** Core. **Type:** comparison.

When would the same status domain be exposed as a tool, a resource, or a prompt?

Testing: Protocol primitives and interaction intent.

MCP-03 — Discovering is not invoking

Provenance: B · documentation-based practice. **Sources:** [M03](#) [M04](#). **Priority:** Core. **Type:** explanation.

What does `tools/list` establish, and what additional work happens for `tools/call`?

Testing: Schemas, discovery and execution.

MCP-04 — stdio versus remote HTTP

Provenance: B · documentation-based practice. **Sources:** [M02](#) [C04](#). **Priority:** Core. **Type:** comparison.

Compare deployment and failure boundaries for stdio and the lab's Streamable HTTP transport.

Testing: Transport selection and local/remote responsibilities.

MCP-05 — The harmless-looking log line

Provenance: C · original protocol failure scenario. **Sources:** [M02](#) [T09](#). **Priority:** Core. **Type:** troubleshooting.

A stdio server stops working after adding `console.log("starting server")`. What would you investigate?

Testing: Protocol streams versus application diagnostics.

MCP-06 — Version first, trace second

Provenance: B · documentation-based practice. **Sources:** [M04](#) [M08](#) [M09](#) [M10](#). **Priority:** Core. **Type:** explanation.

Explain the lab's connection-to-tool-call trace, then identify why it cannot be called the universal latest MCP flow.

Testing: Version-aware protocol explanation.

MCP-07 — What actually ships?

Provenance: C · original integration inventory. **Sources:** [C04](#) [M02](#). **Priority:** Useful. **Type:** design.

A plugin contains instructions and names a helper script, while its MCP endpoint is remote. Where must each file exist?

Testing: Execution-location inventory rather than assumed packaging rules.

2.8 Answer key

TS-01 — Model answer

Interview-ready answer. The interface tells the compiler what shape the code expects; it does not validate the bytes received over HTTP. I would treat the parsed body as unknown, validate it at the boundary, and only then pass the resulting typed value to internal code. A type assertion is not validation. This is similar to the distinction between Python type hints and runtime parsing: both can describe intent, but untrusted external data still needs an explicit check.

Deeper explanation. Types are erased from the emitted JavaScript. A safe boundary either returns a valid domain value or a controlled failure; it should not leave partially trusted values flowing into handlers. [T01](#)

Example. `const x = JSON.parse(text) as StatusRequest` can still produce an object with a numeric service field. The assertion adds no runtime test.

Common weak answer. “TypeScript prevents invalid API requests” confuses developer checks with a server’s input boundary.

Follow-up with answer. Why prefer unknown over any? Unknown requires narrowing before use. Any can let an unsafe assumption propagate without a compiler complaint; use it only for a justified, contained escape hatch.

TS-02 — Model answer

Interview-ready answer. For a simple object contract, either an interface or an object type alias can describe the required fields. Type aliases also conveniently express unions such as operational or degraded. TypeScript generally checks structural compatibility, so an object need not be constructed from a particular class just to satisfy a shape. I would choose the form that makes the domain clear and follow the project’s conventions, rather than inventing a deep class hierarchy for a JSON request.

Deeper explanation. The important design question is what values and combinations are valid. Interfaces have declaration-merging capabilities, while aliases can directly name unions and other type expressions; those differences matter only when the project uses them. [T01](#)

Example. `type Status = "operational" | "degraded"` rejects another string at checked call sites, while runtime validation still handles external strings.

Common weak answer. “Interfaces exist at runtime and reject extra JSON fields” assigns them behavior they do not have.

Follow-up with answer. Would you use a class here? Only if the object needs meaningful construction or behavior. A plain validated data object is sufficient for the mock status request.

TS-03 — Model answer

Interview-ready answer. The `ok` field has a different literal value in each union member. Checking it narrows the value to the matching member, so the compiler knows which other fields exist. This expresses a successful status lookup and an unknown-service result as alternatives rather than one object with several unrelated optional fields. I would use that structure to make callers handle the

failure path explicitly and avoid contradictory states such as success being true while only an error message is present.

Deeper explanation. The narrowing is based on evidence in control flow. An unchecked assertion that chooses one branch would discard this protection. Exhaustive handling becomes especially useful when more domain outcomes are added. [T02](#)

Example. `if (result.ok) return result.status; return result.reason;` follows the union rather than guessing a field exists.

Common weak answer. “A union means every member’s properties always exist” misunderstands alternatives as an intersection.

Follow-up with answer. What changes when another error case is added? Review all consumers and tests. A well-designed discriminated union can make omitted cases visible to the compiler instead of leaving silent fallback behavior.

TS-04 — Model answer

Interview-ready answer. The logical-or expression substitutes three for any falsy value, including zero. The nullish expression substitutes only for null or undefined. If zero intentionally means no retries, I would use a nullish default after validating the option’s type and range. The correct operator depends on what absence means in the domain. I would not use a defaulting operator to hide a malformed string or a negative retry count.

Deeper explanation. Optional values require two decisions: whether a value was provided and whether a provided value is valid. These are different checks. Environment variables are strings, so numeric conversion and validation must happen before numeric default semantics are trusted. [T01](#) [T02](#)

Example. `Number(process.env.PORT ?? "3000")` still needs an integer/range check; a non-numeric supplied value becomes NaN rather than a sensible default.

Common weak answer. “Both operators provide the same fallback” loses legitimate zero, false or empty-string values.

Follow-up with answer. Does `const` make the configuration object immutable? No. It prevents rebinding that variable; runtime object mutation and compile-time readonly declarations are separate matters.

TS-05 — Model answer

Interview-ready answer. I would use strict checking, clear request and result types, and runtime validation at external boundaries. I would keep the domain functions small and let inference handle obvious local values rather than annotate everything. I would avoid elaborate generic machinery that makes a three-field request harder to understand. Where a library boundary is awkward, I would isolate and test the adapter instead of spreading any through the application. The goal is fewer inconsistent assumptions, not the most sophisticated type expression.

Deeper explanation. A good type model mirrors real alternatives and invariants. More annotations do not necessarily create better runtime behavior. Validation, tests and readable error handling remain necessary even in a fully typed codebase. [I04](#) [T01](#) [T02](#)

Example. The reference domain has a two-value Status union and a read-only table; the SDK adapter handles protocol-specific details separately.

Common weak answer. “Use any until the project is done” postpones the very feedback that prevents incompatible assumptions from spreading.

Follow-up with answer. When is a cast acceptable? At a narrowly justified boundary whose invariant has been checked or cannot be expressed conveniently, with a comment and test—not as a way to silence an unexplained error.

TS-06 — Model answer

Interview-ready answer. I would separate startup/configuration, the pure status domain, the protocol adapter and the client. That lets me test the domain without starting a network server and replace the transport without rewriting the business rule. I would state the module and build convention explicitly and keep runtime assets visible in the packaging rules. When describing this workbook project, I would distinguish the supplied starter and reference code from the changes, tests and debugging I personally performed.

Deeper explanation. The question is not permission to invent a production story. Evidence such as a commit, a failing test you added or a deployment observation is more credible than an unsupported claim about scale. [I04](#) [T03](#)

Example. `domain.ts`, `mcp.ts`, `server.ts` and `client.ts` have separate responsibilities; `data/status.json` is a runtime asset, not a compiler output.

Common weak answer. “I built a production platform” is inaccurate when the work was a local mock lab with supplied scaffolding.

Follow-up with answer. What would you say about AI assistance? Name where it helped draft or explain code, then explain what you reviewed, changed and verified. Do not claim to understand a security check you cannot justify.

TS-07 — Model answer

Interview-ready answer. Lookup assumes it receives a validated table and a string name. It reads the table and returns a mock status or undefined; it does not validate raw JSON, identify a user, enforce permissions or format an MCP result. The future adapter must validate incoming arguments, obtain an authorized caller context, and translate the result into the protocol contract. I would keep those checks visible rather than assume the TypeScript signature proves every caller has followed the rules.

Deeper explanation. A pure function is easy to reason about precisely because its assumptions are narrow. Tests should cover valid data and unknown names, while separate adapter tests cover invalid input and permission failures. [T01](#) [T02](#) [M03](#)

Example. An unknown service should produce an explicit unknown-service result; it should not silently return operational or access an employer endpoint.

Common weak answer. “The function is read-only, so it needs no authorization” ignores that reading data can still require permission.

Follow-up with answer. Why use a validated table with an appropriate key representation? To avoid accepting arbitrary malformed data and to make unknown keys predictable, including keys resembling inherited object properties.

MCP-01 — Model answer

Interview-ready answer. The desktop assistant is the host coordinating the user and model interaction. Its MCP client component speaks the protocol to our server. The server advertises the tool and executes the validated, authorized handler against mock data. The server need not run an LLM and need not be remote: with stdio it can be a local subprocess. I would label the actual processes and transport in a diagram instead of using the word client for both the human user and every software component.

Deeper explanation. One host may manage several client connections and servers. Policy or confirmation in the host is useful, but it does not remove the server's responsibility to enforce its own input and access rules. [C04](#) [M01](#)

Example. The terminal reference client is a minimal protocol client; it is not itself a conversational LLM host.

Common weak answer. "The MCP server is the language model" confuses capability execution with model orchestration.

Follow-up with answer. Can a server be containerized while remaining local? Yes. Location, packaging and transport are separate choices; a local container can serve a local HTTP endpoint or participate in a suitably configured stdio integration.

MCP-02 — Model answer

Interview-ready answer. A tool is useful for a parameterized operation such as reading the status of a named service. A resource can expose a data item through the protocol's resource model for the host to obtain or present. A prompt is a reusable interaction template, for example instructions for discussing an incident, rather than the status data itself. I would choose the primitive that fits how the host should discover and use the capability. These labels do not themselves enforce authorization.

Deeper explanation. The same domain can support more than one primitive, but duplication should have a purpose. A prompt mentioning a command does not guarantee execution of that command, and a resource is not simply every arbitrary HTTP endpoint. [C04](#) [M01](#) [M03](#)

Example. This workbook implements one read-only tool and does not add resources or prompts solely to increase the feature count.

Common weak answer. "Prompts are server-side tool code" confuses instructions supplied to a host with executable implementation.

Follow-up with answer. Does readOnlyHint guarantee safety? No. It is descriptive metadata. The implementation, underlying data sensitivity and server-side policy determine the actual risk.

MCP-03 — Model answer

Interview-ready answer. In this version-labelled lab, tools/list lets the client learn the advertised tool names, descriptions and schemas. It does not execute the status lookup. A tools/call request names an operation and supplies arguments; the server then checks the request, validates the

input, applies authorization and runs the handler. A successfully listed tool can still reject a particular call. I would test discovery and execution separately because a correct catalog response does not prove the handler or its dependencies work.

Deeper explanation. Discovery assists interoperability and user experience. It is not a permission boundary by itself: a caller may already know a tool name and call it directly. The invocation must enforce the operation's policy. [M03](#) [M04](#)

Example. The supplied client calls `listTools` and then `callTool`, making the two checks visible in its output.

Common weak answer. "The user saw the tool, so they are authorized to run it" treats presentation as enforcement.

Follow-up with answer. Should discovery be filtered by permission? It can improve least-exposure behavior, but per-call checks remain necessary. This lab keeps the mock catalog visible to connected callers and demonstrates denial at invocation.

MCP-04 — Model answer

Interview-ready answer. With `stdio`, the host usually launches a local process and exchanges protocol messages through its standard streams. Failures include a missing executable, wrong working directory or polluted `stdout`. With Streamable HTTP, the client reaches an HTTP endpoint, so DNS, listener, TLS, routing and HTTP authentication become additional boundaries. The remote server's code and assets live with that service. I would first verify the protocol and SDK versions because legacy HTTP+SSE and the current protocol are not interchangeable with every course example.

Deeper explanation. The transport carries the protocol; it does not define the domain operation. Moving to HTTP introduces deployment and security concerns without automatically changing what `read_status` means. [M02](#) [C04](#)

Example. A localhost health request tests an HTTP endpoint; launching a `stdio` server requires the client to control `stdin/stdout` rather than use `curl`.

Common weak answer. "Change the URL and `stdio` becomes HTTP" ignores different process ownership and transport setup.

Follow-up with answer. Which should the work project use? Match the intended deployment and supported clients. Do not change an already supported transport merely to match a tutorial's age or preferred framework.

MCP-05 — Model answer

Interview-ready answer. I would check whether the new log goes to the same `stdout` stream that carries MCP protocol messages. If it does, an otherwise harmless text line may make the client fail to parse the stream. I would move ordinary diagnostics to `stderr`, keep `stdout` for valid protocol output, and retry initialization and a tool call. I would still inspect the actual parse error before declaring the log the root cause, since the change could also have altered startup behavior.

Deeper explanation. A transport boundary assigns meaning to bytes. The same log line is less likely to corrupt an HTTP response when a framework separates logs from response bodies, but sensitive data must still not be logged. [M02](#) [T09](#)

Example. Use `console.error` for a local startup diagnostic. Do not print bearer tokens or full request headers on either stream.

Common weak answer. “All logging is safe because it is not part of the handler return value” misses the shared standard-output channel.

Follow-up with answer. What about protocol logging notifications? Use the protocol-supported mechanism and capabilities where appropriate; it is different from injecting arbitrary plain text into the transport.

MCP-06 — Model answer

Interview-ready answer. For the SDK v1 and 2025-11-25 family used here, the client performs initialization and capability negotiation, then discovers tools and invokes `read_status` with arguments. The server validates, authorizes and returns a result or an appropriate error. I would explicitly state that baseline. The July 2026 protocol changes initialization and session-related behavior, so I would consult its versioned specification and matching SDK rather than describe this older trace as the current universal protocol.

Deeper explanation. Version labels are part of correctness, not a footnote. A working old client/server pair can be more appropriate for an existing project than a partially migrated combination of new and old components. [M04](#) [M08](#) [M09](#) [M10](#)

Example. Record the SDK package and lockfile version beside a captured request trace; do not infer the version from the use of HTTP alone.

Common weak answer. “MCP always begins with this exact handshake” ignores a material protocol change verified during research.

Follow-up with answer. Would you automatically upgrade the employer server? No. Assess client compatibility, migration changes, tests and team policy first. A tutorial is not authorization to alter a work project’s dependency strategy.

MCP-07 — Model answer

Interview-ready answer. I would identify which process is supposed to execute the helper. If the local host runs it, the script and its runtime dependencies must be available locally through the supported package format. If the remote MCP handler runs it, they belong in the remote service artifact. Instructions that merely name the script do not ship or execute it automatically. I would also separate endpoint configuration from credentials and verify the actual client/plugin contract rather than assume one universal packaging format.

Deeper explanation. The inventory should trace entry points, imports, runtime file reads and configuration ownership. A remote server can still depend on local client components, but each dependency needs an explicit distribution mechanism. [C04](#) [M02](#)

Example. Create columns for file, executing process, installation location and configuration source; mark unknown employer-specific details as questions to resolve from the actual project.

Common weak answer. “The plugin contains everything” is not an architectural explanation unless its actual contents and execution model were inspected.

Follow-up with answer. What is the first check for a missing helper after bundling? Determine whether it is imported into the bundle or accessed as a filesystem path at runtime, then inspect the artifact on the machine that executes it.

2.9 Observable checkpoint and stopping point

Explain the supplied handler without calling every layer “the API.” Demonstrate a compile-time type error and a runtime invalid-input rejection as separate experiments. Draw which process owns stdout in stdio. Name the actual protocol version before drawing its connection flow. Identify the local and remote files for one hypothetical integration without assuming your employer’s packaging format.

Evidence should include one short typed function, its invalid-input test, and a labelled request diagram. You do not need to build the OAuth implementation yet. The useful stopping point is being able to read and explain the pieces that Stage 2 will connect.

3. Stage 1 practice, checkpoint and mock interview

Stage / track: 1, both tracks. **Course points:** Docker run/build/network/storage topics, TypeScript object/union topics, and HF MCP Unit 1. **Prerequisites:** Chapters 1–2. **Outcome:** demonstrate one local service and explain where the future MCP adapter belongs. The full file contents are in Appendix A and the accompanying status-lab project; open the reference solution only after making your own attempt.

3.1 Default local setup and safety boundary

Use **Bash**, **Node 22.x**, **TypeScript 5.8.3** and **Docker with Linux containers**. On Windows, use WSL2 with the appropriate Docker integration and keep the project in the Linux filesystem where practical. On macOS/Linux, use your usual compatible terminal. Check `node --version`, `tsc --version` and `docker version`. These are tool assumptions, not claims about what is already installed on your computer.

Commands start at the status-lab repository root unless a `cd` is shown. Quote paths. Use a disposable project copy for break/fix work. Published ports bind to **127.0.0.1**; do not expose this unauthenticated mock service publicly. No employer credentials, private code, paid cloud account or production permissions are required.

3.2 Exercise E1 — Containerize and deliberately break the HTTP service

Objective. Carry the same mock status domain through native execution, image construction, runtime configuration, network diagnosis and a read-only input mount. **Prerequisites.** Chapter 1 and the course's Dockerfile, ports and storage lessons. **Files.** `milestones/01-http/server.mjs`, `data/status.json`, `Dockerfile`, `.dockerignore`.

Independent task. Read the starter, run it without Docker, then write your own Dockerfile before comparing the supplied one. It must copy the data, use a non-root user, start Node at runtime and listen on the correct container interface. Predict whether `EXPOSE` alone makes a host request succeed.

```
cd milestones/01-http
node server.mjs
# In another terminal:
curl --fail http://127.0.0.1:3000/healthz
curl --fail 'http://127.0.0.1:3000/status?service=api'
```

Stop the native process with `Ctrl+C` before publishing the same port. Build and run your image:

```
docker build -t workbook-http:0.1.0 .
docker run --rm --name workbook-http \
  -p 127.0.0.1:3000:3000 workbook-http:0.1.0
```

Success criteria. The process is non-root; `/healthz` returns HTTP 200; `api` returns mock operational status; an unknown service returns 404; and stopping the container produces the shutdown event. These are expected observations. The native starter checks were executed during authoring, but this Docker sequence was not.

Break it. Stop and recreate the container with `-e HOST=127.0.0.1`. Keep the host port mapping unchanged. Predict the outcome before requesting `/healthz`.

Diagnostic procedure. First ask where the request originates, what exact command created the container, and whether the error is refusal, timeout or an HTTP response. Observe `docker ps`,

`docker port workbook-http` and the startup log before changing anything. Test the container-local endpoint with a Node fetch executed inside the container. If that works while the host request fails, compare the listener interface with the destination reached through the published mapping. A wrong host port, crashed process and loopback-only listener are distinct hypotheses; the symptom alone does not choose among them.

```
docker logs --tail 40 workbook-http
docker exec workbook-http node -e \
  'fetch("http://127.0.0.1:3000/healthz").then(r=>console.log(r.status))'
```

Graduated hints. Hint 1: the two uses of 127.0.0.1 do not name the same network namespace. Hint 2: compare the startup log's host value with the Dockerfile default. Hint 3: remove the override or use `HOST=0.0.0.0` inside the container, while retaining host-loopback-only publishing.

Reference solution. The supplied Dockerfile copies both code and data, selects `USER node`, sets `HOST=0.0.0.0`, and uses `CMD ["node", "server.mjs"]`. Recreate the container without the bad override. Verify the same host request and the unknown-service case. Prevention is a final-image smoke test that reaches the service through the intended published boundary, not only an in-container test.

Mount variation. Create a separate directory containing a valid `status.json`, change `api` to `degraded`, and mount that directory read-only at `/app/data`. Restart to load the file. The result should reflect the replacement input. If startup fails, inspect file name, JSON syntax and read permissions before relaxing permissions broadly. The application loads this file at startup; live editing does not automatically refresh its in-memory table.

Stretch. Delete the data-copy instruction in a disposable Dockerfile, diagnose startup failure, and show that rebuilding the final image with an explicit copy repairs it. Do not add Kubernetes here.

Cleanup. `docker stop workbook-http` removes the running `--rm` container. Remove only the named lab image or your temporary data directory when no longer needed. Avoid blanket prune commands. **Interview evidence:** DKR-07, DKR-08, DKR-09, DKR-12 and the host/container network explanation.

3.3 Exercise E2 — A typed domain and a paper MCP trace

Objective. Separate compile-time feedback from runtime checks and identify the adapter boundary. **Tools.** TypeScript 5.8.3 and Node 22.x; no SDK or Kubernetes required. **Files.** Use `service/src/domain.ts` as the solution; first create a scratch `lookup.ts` with the types and function in Section 2.3.

Independent task. Add a compile-time-invalid call using a number instead of a service name. Then remove that invalid call and implement a function accepting unknown input that rejects a malformed table. Write two successful examples and two invalid examples. Finally draw a v1-family MCP trace showing where this function would be called.

From the repository root, the reference domain can be compiled separately:

```
tsc service/src/domain.ts --strict --target ES2022 \
  --module commonjs --outDir artifacts/ts1
node <<'JS'
const d = require("./artifacts/ts1/domain.js");
console.log(d.readStatus(d.parseTable({api:"operational"}),"api"));
JS
```

Success criteria. The deliberately ill-typed source fails the compiler. A malformed runtime table fails `parseTable` even after compilation. An unknown service produces no invented status. Your diagram puts external validation before the domain function and does not claim the function performs authentication.

Hints. First distinguish “which variable exists” from “what shape an unknown value has.” Then use object/null/array checks and iterate actual entries. Finally compare with `parseTable` in the solution, which validates names and the two allowed status values before returning a frozen, null-prototype table.

Reference solution. `domain.ts` supplies `Status`, `StatusTable`, `parseTable` and `readStatus`. `domain.test.ts` supplies the later automated tests. The paper trace is client discovery/call → adapter input validation → authorization boundary, introduced in Stage 2 → domain lookup → MCP result conversion. The handler has no authority to access a real SAP service merely because a model asks it to.

Stretch and cleanup. Add a third domain status and identify all validation/type/test changes needed. Delete only the generated artifacts/`ts1` directory. **Interview evidence:** TS-01, TS-03, TS-07, MCP-01 and MCP-03.

3.4 Closed-book and troubleshooting checkpoint

S1-CB1. Explain the build/run distinction and name one runtime input not fixed by an image digest.

Answer: building constructs the artifact; running starts an instance with configuration and dependencies such as environment values or external services. See 1.2–1.3 and DKR-05.

S1-CB2. Explain why a type assertion cannot secure a tool input. **Answer:** it changes static interpretation, not runtime bytes; validate the input and independently authorize the operation. See 2.1 and TS-01.

S1-CB3. Distinguish a health request, tool discovery and tool invocation. **Answer:** they test HTTP reachability/health, protocol catalog availability and actual operation execution respectively. See 2.5 and MCP-03.

S1-CB4. Explain what is local in stdio versus HTTP. **Answer:** stdio normally places the launched server and required files with the host; HTTP places server execution and assets at the remote service, with endpoint/auth/client code locally as required. See MCP-04 and MCP-07.

S1-T — Trouble ticket. A container is running, the host request times out, and its logs show a successful startup. Do not declare the cause. Request the launch command and exact address; inspect port publishing and listener; compare inside/outside requests; verify DNS only if using a hostname; separate TLS and HTTP errors. Apply one evidence-supported fix and repeat the original request. A running process and startup message are insufficient to prove route or readiness.

S1-D — Design discussion. Package a read-only local status service for another developer. A strong answer chooses a small image with mock data, explicit runtime configuration, loopback-only host exposure, no embedded secrets, a clear version record and a bounded shutdown. A writable database and public internet endpoint add no value to this requirement. Explain how the later MCP adapter would reuse the domain without implementing it today.

Scoring. Score five dimensions from 0–4: conceptual distinctions, evidence-led diagnosis, working or accurately walked-through commands, design justification, and honest communication. Four means independent and correct with a useful boundary; three means mostly correct with a minor

omission; two needs a hint; one confuses a major layer; zero cannot yet explain it. A score of 14–20 is ready to continue, 9–13 calls for targeted revision, and below 9 calls for repeating the relevant small exercise. Optional stretch mastery is separate. This is a learning aid, not a validated employability measure. No perfect score is required.

3.5 Stage 1 mock interview — 25 minutes

Opening, 4 minutes. “Explain the service and the difference between its image and process.” Use DKR-01. A strong answer separates artifact, process and mock domain, without claiming production experience. **Model:** “This is a local learning service with an immutable mock-data file. Docker packages its runtime; a particular container runs Node with explicit configuration. I verified the layers I actually exercised and would describe the remaining deployment checks separately.”

Diagnosis, 8 minutes. “It runs but I cannot reach it from the host.” Use the E1 evidence sequence. **Branch:** inside-container fetch succeeds. **Model:** “That narrows the application/listener hypothesis, but I still need the exact listener interface and mapping. A loopback-only listener may explain this difference; I will compare it with the published target and retest after one change.” **Alternative branch:** inside fetch also fails. **Model:** “I would inspect process state, actual port and startup errors before modifying publishing.”

Code reasoning, 6 minutes. “Can this interface replace request validation?” Use TS-01. **Branch:** “The JSON came from our own model.” **Model:** “That does not make it valid. The boundary still accepts external generated data, so I validate its shape and permitted values and handle failure explicitly.”

MCP explanation, 5 minutes. “Where does a tool call execute?” Use MCP-01 and MCP-04. **Branch:** “The server is remote but the plugin references a local script.” **Model:** “I would identify which process runs that script. Remote hosting does not distribute a local dependency automatically; the package and execution inventory must agree.”

Reflection, 2 minutes. “What have you not verified?” **Model:** “I would name the tests I ran, their versions and boundaries. I would not call a Dockerfile successful merely because native Node tests passed.” Evaluate the same five dimensions above. Revisit 1.4 for network confusion, 1.2–1.3 for packaging confusion, 2.1 for validation confusion and 2.4–2.6 for protocol or placement confusion.

STAGE 2 — KUBERNETES AND AN IMPLEMENTED MCP SERVICE

4. Kubernetes: desired state, traffic and diagnosis

Stage / track: 2, main. **Courses:** Kubernetes for Absolute Beginners [C06](#), focused official configuration/access/probe references [K07](#) [K08](#) [K09](#) [K10](#) [K11](#) [K12](#) [K13](#) [K14](#). **Attempt after:** Pods, YAML, ReplicaSets, Deployments and Services; then study the focused topics alongside their sections. **Prerequisites:** Chapter 1's process, image, port and configuration model. **Outcomes:** explain the object relationships, deploy ordinary YAML, distinguish scheduling from startup and readiness, route traffic to intended Pods, and diagnose a failed rollout. Helm is not required here.

4.1 Why an orchestrator exists—and when not to add one

Docker can start a process on a machine. Kubernetes adds a declarative control system for workloads across a cluster: you submit a desired state, and controllers act to move observed state toward it. A scheduler chooses a suitable node for an unscheduled Pod; the node's components arrange for its containers to run. This is an ongoing reconciliation process, not a single successful remote shell command. [K01](#) [K02](#)

For a tiny personal service, a managed application platform or a carefully operated single host may be sufficient. Kubernetes brings its own configuration, security, upgrade and troubleshooting work. Its value is clearer when a team needs consistent deployment and recovery behavior across many workloads or an existing approved cluster. "Our organization uses it" can be a practical reason to learn it without claiming it is automatically the cheapest choice for every application.

The basic ownership chain is:

```
Deployment: desired application rollout and replica count
|
+-- ReplicaSet: maintain replicas of one Pod template
    |
    +-- Pod: scheduled execution unit
        +-- application container
```

A Pod is not a permanently named VM. It can be replaced, get a new IP, and lose Pod-local state. A Deployment manages changes to a replicated application through ReplicaSets. Normally edit the Deployment's template, not an individual managed Pod or ReplicaSet. Otherwise the controller may replace your manual change while pursuing its declared state. [K02](#) [K03](#)

Workplace connection. A container image is only one input. A team also needs desired replicas, configuration, readiness criteria, resource expectations and a network entry point. This chapter makes those choices explicit before Helm starts generating them.

4.2 YAML names objects; labels connect them

A typical manifest uses `apiVersion`, `kind`, `metadata` and a kind-specific spec. The API version identifies the schema to interpret; the kind identifies the object type. A resource's name is not automatically a selector for another resource. Labels are key-value metadata, while selectors express which labelled objects a controller or Service should target. [K04](#)

```
# Deployment Pod template labels
metadata:
  labels:
    app: status-lab
# Service selection
```

```
spec:
  selector:
    app: status-lab
```

These snippets illustrate the matching values, not two complete deployable objects. In the complete manifest, the labels belong under the Deployment's `spec.template.metadata`, and the selector belongs under the Service's `spec`. Putting a correct value in the wrong YAML location can be as wrong as choosing the wrong value.

A namespace scopes many resource names and administrative operations. It is useful for the disposable workbook environment, but does not automatically isolate all network traffic or supply every security policy. Explicitly use `-n workbook` in commands so a similarly named workload in another namespace does not mislead your investigation. Permissions may allow creating a Deployment inside an existing namespace without allowing namespace creation. If namespace creation is forbidden, use a namespace explicitly allocated for your development work rather than requesting cluster-admin access.

YAML validity is not the same as Kubernetes validity. A parser may accept a document whose field name or type the API rejects. A valid object can still describe a non-working system, such as a Service selector that matches no intended Pods. Validation is therefore layered: parse, validate against the API, then observe behavior.

4.3 Services provide an address, not a running server

A Service offers a stable way to reach a changing set of backends. In the ordinary selector-based case, matching Pods are represented through endpoint information, and readiness influences which backends should receive ordinary traffic. The Service itself does not run your Node handler or make an unready application healthy. [K05](#)

```
in-cluster client
-> DNS name: status-lab.workbook.svc.cluster.local
-> Service port 80
-> selected ready Pod IP, target port named http
-> container listener on port 3000
-> application path /mcp or /healthz
```

The arrows cross distinct boundaries. The Service's port: 80 is the port a client addresses. `targetPort: http` resolves to the appropriately named Pod container port, which is 3000 in the lab. `containerPort` is a declaration used by these configurations; it does not cause Node to listen. Node must still bind to the configured interface and port. A Service can exist with no correct backends.

ClusterIP is appropriate for this local/private lab. NodePort exposes a port on nodes under its networking rules, and LoadBalancer asks for supported external load-balancer integration. None of these choices alone supplies a complete public HTTPS and application-authorization solution. Do not switch Service type to public exposure merely because port-forwarding is unfamiliar. [K05](#)

`kubectl port-forward service/status-lab 3000:80` is convenient for a local demonstration. It selects and forwards to a backend through the control connection; it does not prove the normal in-cluster DNS/Service data path works. To test that path, use an allowed in-cluster client and call the Service name. Interpret the two tests as complementary, not interchangeable. [K06](#)

4.4 Configuration, identity and state

A ConfigMap separates non-secret runtime configuration from the image. A Secret is intended for sensitive values, but base64 representation is encoding, not encryption. Protection depends on

cluster storage/encryption configuration, RBAC, workload access and operational practices. Do not print entire Secret objects into a shared debugging channel. [K07](#) [K08](#)

When environment variables are populated from a ConfigMap or Secret, an existing process does not automatically receive later value changes. Recreate or roll out the workload when that is the intended update mechanism. Mounted configuration can update under documented conditions, but the application must reread it, and subPath mounting has different update behavior. The lab reads its mock file at startup, so editing a mounted file is not evidence that the in-memory table changed. [K07](#)

A service account gives a workload an identity for Kubernetes API interactions. Kubernetes RBAC grants API permissions such as reading a ConfigMap in a namespace. It is separate from the end user's OAuth permissions for `read_status`. This service does not call the Kubernetes API, so the reference disables automatic mounting of a service-account token and supplies no broad RoleBinding. Disabling that mount does not disable OAuth or stop incoming HTTP traffic. [K11](#) [K12](#)

Storage needs an explicit lifecycle. Container writable state is not a durable data design. An `emptyDir` is associated with a Pod: it can survive an individual container restart within that Pod, but not the Pod being removed and replaced. A persistent volume claim requests storage that can have a lifecycle separate from a specific Pod, subject to the storage class, access mode and reclaim policy. None of this makes arbitrary application files safe for multiple concurrent writers. [K13](#)

The mock service reads an immutable file and keeps no user state. Therefore replicas can independently serve the same data without a writable volume. That is a deliberate stateless design, not a claim that every MCP server is stateless. Protocol sessions, caches or application transactions may require a different design.

4.5 Health signals, resources and termination

A **startup probe** gives a slow-starting container a chance to initialize before the other probes take over. A **readiness probe** asks whether the application should receive ordinary traffic. A **liveness probe** detects a condition for which restarting the container is expected to help. These are different policies, even if a tiny application happens to use similar checks initially. [K09](#)

A readiness failure does not itself restart the container. A failing liveness or startup probe can. A liveness check that fails whenever a downstream service is temporarily unavailable can create a restart storm without repairing the dependency. Prefer a check reflecting the local process's ability to make progress. The reference mock uses `/healthz` for basic liveness and `/readyz` for readiness/draining, without querying a real downstream service.

Requests inform scheduling and resource allocation decisions. Limits constrain usage under the relevant resource's behavior: CPU limits can lead to throttling, while memory pressure against a memory limit can lead to termination. CPU `100m` means one tenth of a CPU unit. A memory request is not a guarantee that the application will never need more. A Node process's memory footprint also includes more than its JavaScript heap. [K10](#)

The lab values—100m CPU/64Mi memory requested, 500m/256Mi limited—are starting exercise values, not benchmark results or production sizing advice. Measure actual behavior before choosing a real allocation. An unschedulable request, an out-of-memory termination and an application exception require different evidence and different fixes.

On shutdown, the Pod's termination grace period must accommodate the application's drain behavior and any platform hooks. The reference uses a thirty-second Pod grace period and a ten-second application bound. Traffic and endpoint changes are not instantaneous proof that every existing connection drained cleanly. Two replicas, readiness and a rolling strategy are prerequisites for improving availability, not a mathematical zero-downtime guarantee. [K14](#) [K02](#)

4.6 A repeatable diagnostic order

Use this sequence before experimenting with random fixes: **desired object → observed Pod state → application process → selected backend → request path → caller permissions**. At each step ask what evidence would distinguish competing explanations.

```
kubectrl -n workbook get deployment,replicaset,pod,service
kubectrl -n workbook describe deployment status-lab
kubectrl -n workbook get pods -l app=status-lab -o wide
kubectrl -n workbook get events --sort-by=.metadata.creationTimestamp
kubectrl -n workbook get endpointslice \
  -l kubernetes.io/service-name=status-lab
```

Then choose a specific Pod and examine describe, current logs and, for a restarted container, logs --previous. exec is useful only when the container is running, you are permitted to use it, and the image contains an appropriate executable. A minimal runtime image may have no shell; the reference can use Node for a local fetch. Lack of a shell is not evidence of an application fault. [K06](#) [K14](#)

A Pending Pod may lack a feasible node or an attachable volume; inspect events rather than assuming capacity. ImagePullBackOff describes a repeated image-pull problem, not a distinct Pod phase; check the exact image, registry reachability and pull permissions. CrashLoopBackOff describes repeated container failures; inspect prior termination and logs. OOMKilled is evidence relevant to memory termination, not proof that increasing a limit is the only correct response.

For a Service failure, compare selector values to **Pod template labels**, inspect endpoint readiness, confirm Service and target ports, and test progressively closer to the application. For a rollout failure, check whether replacements become ready and whether resource headroom permits the surge. Preserve old healthy capacity while identifying the cause. In an ordinary Deployment, an undo changes the Pod template revision; it does not restore every external ConfigMap value or undo database side effects. [K02](#) [K06](#)

Useful addition: structured logs. A bounded event name, request identifier and non-sensitive outcome let you connect an observed error to a handler without leaking credentials. This belongs beside troubleshooting, not as a separate monitoring-platform course. A request ID helps correlate evidence; it does not prove identity or authorize a request.

4.7 Questions—attempt before the answer key

K8S-01 — Why orchestration?

Provenance: A · published guide, adapted from I01 orchestration question. **Sources:** [I01](#) [K01](#) [K02](#). **Priority:** Core. **Type:** design.

What problem does Kubernetes add to container execution, and when could it be unnecessary?

Testing: Desired-state operation and proportional architecture.

K8S-02 — Pod, ReplicaSet and Deployment

Provenance: B · documentation-based practice. **Sources:** [K01](#) [K02](#) [K03](#). **Priority:** Core. **Type:** explanation.

Explain the ownership chain and why editing a managed Pod is usually the wrong way to release a new version.

Testing: Reconciliation and stable desired-state ownership.

K8S-03 — How does a Service find its Pods?

Provenance: A · candidate interview report, adapted from I08; anecdotal. **Sources:** [I08](#) [K04](#) [K05](#). **Priority:** Core. **Type:** configuration review.

Explain how the reference Service selects Pods and sends traffic to the application port.

Testing: Selectors, readiness and port mapping.

K8S-04 — Namespaces and YAML correctness

Provenance: B · documentation-based practice. **Sources:** [K04](#) [K01](#) [K02](#). **Priority:** Core. **Type:** configuration review.

A valid YAML file is accepted by a text parser but the application is absent from your kubectl view. What would you check?

Testing: API schema, namespace and context.

K8S-05 — Container communication in Kubernetes

Provenance: A · published guide, adapted from I05 communication question. **Sources:** [I05](#) [K01](#) [K05](#) [K06](#). **Priority:** Core. **Type:** explanation.

How would one application reach this service from inside the cluster? What is different within one Pod?

Testing: Pod networking, service discovery and stable addresses.

K8S-06 — Changing a ConfigMap

Provenance: B · documentation-based practice. **Sources:** [K07](#) [T09](#). **Priority:** Core. **Type:** prediction.

You change PORT in a ConfigMap used through envFrom. Does the running Node process immediately change its listener?

Testing: Configuration delivery versus application reload.

K8S-07 — Protecting sensitive configuration

Provenance: A · published hiring guide, adapted from I06 security/access question. **Sources:** [I06](#) [K08](#) [K12](#). **Priority:** Core. **Type:** design.

How would you keep application secrets separate from ordinary configuration and restrict access?

Testing: Secret limitations and least privilege.

K8S-08 — Three probes, three questions

Provenance: B · documentation-based practice. **Sources:** [K09](#) [K14](#). **Priority:** Core. **Type:** comparison.

Distinguish startup, readiness and liveness probes. Diagnose restarts introduced by a new dependency-checking probe.

Testing: Health semantics and failure amplification.

K8S-09 — Requests, limits and observed failure

Provenance: B · documentation-based practice. **Sources:** [K10](#) [K14](#). **Priority:** Core. **Type:** comparison.

Explain the reference resource values and distinguish Pending, CPU throttling and an OOM termination.

Testing: Scheduling versus runtime resource behavior.

K8S-10 — Storage follows a lifecycle

Provenance: B · documentation-based practice. **Sources:** [K13](#) [K01](#). **Priority:** Useful. **Type:** design.

Compare a container writable layer, emptyDir and a persistent volume claim for the mock service.

Testing: State ownership and replacement behavior.

K8S-11 — Two different identities

Provenance: A · published hiring guide, adapted from I06 access-control question. **Sources:** [I06](#) [K11](#) [K12](#) [M05](#).

Priority: Core. **Type:** comparison.

Distinguish the Pod's Kubernetes service account from an OAuth user calling the MCP server.

Testing: API-plane access versus application permissions.

K8S-12 — A failing Pod investigation

Provenance: A · published hiring guide, adapted from I06 failing-Pod question. **Sources:** [I06](#) [K14](#) [K09](#) [K10](#). **Priority:** Core. **Type:** troubleshooting.

Describe a repeatable investigation for a Pod that does not become healthy.

Testing: Evidence selection and hypothesis separation.

K8S-13 — Deploying a new version safely

Provenance: A · published hiring guide, adapted from I06 deployment question. **Sources:** [I06](#) [K02](#) [K09](#) [K14](#). **Priority:** Core. **Type:** design.

What must be true before a rolling update can preserve useful service availability?

Testing: Readiness, capacity, compatibility and bounded shutdown.

K8S-14 — A Service with no useful backend

Provenance: C · original troubleshooting scenario. **Sources:** [K04](#) [K05](#) [K06](#). **Priority:** Core. **Type:** troubleshooting.

The Service exists, port-forward works to a Pod, but the in-cluster Service request fails. What do you inspect next?

Testing: Distinguishing backend health from Service routing.

4.8 Answer key

K8S-01 — Model answer

Interview-ready answer. Kubernetes manages declared workload state across a cluster: placement, replica maintenance, service addressing and application updates are coordinated through its control system. A container runtime alone does not provide that whole operating model. I would use Kubernetes when those capabilities or an existing team platform justify the operational cost. For one small personal service, a simpler hosting model may be sufficient. Learning Kubernetes for the approved work environment does not require claiming it is optimal for every application.

Deeper explanation. Reconciliation is continuous. Submitting YAML does not mean the application instantly reaches that desired state. Scheduling, pulling an image and passing readiness checks are separate observed steps. [I01](#) [K01](#) [K02](#)

Example. A Deployment with replicas set to two expresses an intended count; it does not ask the application to fork itself or guarantee sufficient cluster capacity.

Common weak answer. “Kubernetes makes any service scalable automatically” omits capacity, application state and explicit scaling configuration.

Follow-up with answer. Does it replace application reliability work? No. It can restart or replace containers, but cannot repair incorrect business logic, unsafe retries or incompatible database migrations.

K8S-02 — Model answer

Interview-ready answer. A Pod is the scheduled unit containing one or more closely associated containers. A ReplicaSet maintains a set of Pods from a template. A Deployment manages replica sets and application rollout changes. I would update the Deployment’s Pod template so the controller’s desired state matches the intended release. Editing or replacing an individual managed Pod is temporary at best because the controller can recreate it from the unchanged template. I would inspect ownership when an apparently fixed change keeps disappearing.

Deeper explanation. Multiple containers in one Pod share important execution context such as networking, but that does not mean unrelated services should all be combined into one Pod. The unit should match scheduling and lifecycle needs. [K01](#) [K02](#) [K03](#)

Example. `kubectl -n workbook get deployment,replicaset,pod` shows the different object types; their names and counts are not interchangeable.

Common weak answer. “A Deployment is the running container” skips the control objects and makes rollout behavior hard to explain.

Follow-up with answer. Does deleting a managed Pod reduce the desired replica count? No. The controller normally creates a replacement. Change the owner’s replicas setting to alter desired capacity intentionally.

K8S-03 — Model answer

Interview-ready answer. The Service selector matches labels on Pods, in this case `app equals status-lab`. The endpoint information then identifies suitable backends, with readiness affecting

ordinary traffic eligibility. Clients address Service port 80, while targetPort named http maps to the Pod's named port 3000. Node must actually listen on that port. The Service name does not automatically select a Deployment with the same name. I would inspect labels, endpoints and ports separately if requests reached no application.

Deeper explanation. The candidate report establishes only that a Service/Pod manifest question was reported. Its proposed explanation is not treated as technical authority; the selector and traffic mechanism here come from Kubernetes documentation. [I08](#) [K04](#) [K05](#)

Example. `kubectl -n workbook get endpointslice -l kubernetes.io/service-name=status-lab` helps establish whether intended backends were selected.

Common weak answer. "The matching port number connects the Service to the Deployment" omits Pod selection and the distinction between Service and target ports.

Follow-up with answer. Can the Service port differ from the container port? Yes. Port 80 can map to targetPort 3000 or its named equivalent. The application listener still has to agree with the target.

K8S-04 — Model answer

Interview-ready answer. I would check the active cluster context and namespace, then whether the API accepted the intended resource kinds and names. Valid YAML syntax does not prove a valid Kubernetes object or a working deployment. I would inspect the apply response and list the resources in the namespace specified by the manifest. If permissions prevent access, I would use the approved namespace and request only the necessary permission rather than assume the cluster is empty or ask for administrator access.

Deeper explanation. Labels, metadata and spec fields live at kind-specific paths. A label on the Deployment object is not automatically a label on its Pods; the template must carry the labels the Service selects. [K04](#) [K01](#) [K02](#)

Example. `kubectl config current-context` and `kubectl -n workbook get deploy,pod` establish scope before deeper diagnosis.

Common weak answer. "The YAML parsed, so the deployment exists" collapses syntax, API acceptance and observed state into one test.

Follow-up with answer. Does a namespace create a network security boundary by itself? No. Namespacing helps scope resources and policy, but traffic restrictions require the relevant network/security mechanisms and configuration.

K8S-05 — Model answer

Interview-ready answer. A separate application Pod would normally use the Service's DNS name and Service port instead of a particular backend Pod IP. That keeps the caller independent of Pod replacement. Containers in the same Pod share its network namespace, so localhost can be appropriate between those containers; it is not how a separate Pod is reached. I would verify the caller's namespace, the DNS name and the selected target port, then inspect any applicable traffic restrictions if the basic service configuration is correct.

Deeper explanation. DNS and reachability are different checks. A Service name resolving successfully can still point to a Service with no ready endpoint. A port-forward test is useful but does not exercise the same path as a normal in-cluster request. [I05](#) [K01](#) [K05](#) [K06](#)

Example. `http://status-lab.workbook.svc.cluster.local/healthz` uses Service port 80 in the ordinary local-cluster configuration.

Common weak answer. “Every container uses localhost to reach other containers” confuses the within-Pod exception with cross-Pod communication.

Follow-up with answer. Would you hard-code Pod IPs for performance? Not for this service. Their lifecycle instability creates unnecessary coupling; first measure any real bottleneck before bypassing standard discovery.

K8S-06 — Model answer

Interview-ready answer. No. Environment values are supplied when the container process is started; changing the ConfigMap does not rewrite the environment of that existing process. I would deliberately roll out the workload after checking that the Service target and container port settings remain consistent. Mounted configuration has different update behavior, but the application still needs to reread and apply it. I would not infer successful reconfiguration from the ConfigMap's new contents alone.

Deeper explanation. Even a process that rereads a file needs a safe reconfiguration design. For this lab, startup-only configuration is simpler and explicit. The Helm chapter adds a Pod-template checksum to make a chart-managed ConfigMap change trigger a rollout. [K07 T09](#)

Example. Compare the live ConfigMap with the startup log of a newly created Pod; do not print sensitive environment values from a real service.

Common weak answer. “Kubernetes automatically reloads all application configuration” overstates both delivery and application behavior.

Follow-up with answer. Can a subPath-mounted file be assumed to update like a normal ConfigMap volume? No. Check the documented mounting semantics; do not base a reload design on an unsupported assumption.

K8S-07 — Model answer

Interview-ready answer. I would keep non-sensitive settings in ordinary configuration and obtain credentials through the environment's approved secret mechanism. A Kubernetes Secret is not secure merely because its data is base64 encoded; storage encryption, RBAC and workload access still matter. I would grant only the required identities access, avoid embedding values in images or source control, and ensure diagnostics do not print them. The mock status service needs no client secret, so adding one would not make the lab more secure.

Deeper explanation. Reading a Secret through the API and executing inside a Pod that consumes it are both sensitive capabilities. A clean architecture considers indirect access as well as the obvious get-secret permission. [I06 K08 K12](#)

Example. The chart accepts an existing Secret reference rather than placing a real credential in `values.yaml`. The default reference is empty.

Common weak answer. “Secrets are encrypted because they are unreadable base64” confuses encoding with encryption and access control.

Follow-up with answer. What should happen after credential rotation? Ensure the application obtains the new value according to its reload/restart design, verify the new credential, and revoke the old one through the provider's supported process.

K8S-08 — Model answer

Interview-ready answer. Startup allows initialization to finish before the other checks take over. Readiness controls whether the Pod should receive normal traffic. Liveness identifies a condition where restarting the container should help. If adding a dependency check causes restarts, I would verify which probe uses it and inspect events and timing. A temporary downstream outage may justify changing traffic readiness, but restarting every dependent process can amplify the incident without fixing the dependency. I would design the check around the intended recovery action.

Deeper explanation. The initial delay, period, threshold and timeout must reflect application behavior. A startup probe can prevent a legitimately slow startup from being killed by an overly eager liveness check. [K09](#) [K14](#)

Example. Change only readiness to a nonexistent path and observe endpoint eligibility; change liveness in a separate disposable experiment and observe restart count.

Common weak answer. "Readiness restarts unhealthy Pods" confuses traffic selection with process recovery.

Follow-up with answer. Should readiness always check every dependency? No. Include only dependencies necessary for the traffic being served and consider partial operation, cascading failures and the cost of the check.

K8S-09 — Model answer

Interview-ready answer. The requests describe the resources Kubernetes uses when placing and accounting for the workload; the limits constrain usage according to the resource's behavior. A 100m CPU request is one tenth of a CPU unit. A Pod may remain Pending if its scheduling requirements cannot be met. CPU limits can throttle execution, while memory-limit pressure can terminate a container. I would inspect events, termination reason and measurements before increasing values, because these symptoms are not interchangeable and the lab numbers are not production sizing results.

Deeper explanation. A Node process uses memory outside the JavaScript heap as well. Setting a heap size equal to the entire container memory limit leaves no allowance for native allocations and other overhead. [K10](#) [K14](#)

Example. The manifest requests 64Mi and limits 256Mi for the exercise. Record actual usage and workload behavior before selecting different values.

Common weak answer. "A request is a hard maximum" reverses the roles. "OOM means Kubernetes randomly killed a healthy app" ignores a relevant resource boundary.

Follow-up with answer. Would adding CPU fix a blocked event loop? It may not. A single JavaScript callback doing CPU work still blocks its event loop; consider the algorithm, worker isolation or replicas after identifying the bottleneck.

K8S-10 — Model answer

Interview-ready answer. A container's writable layer is tied to that container's lifecycle. An `emptyDir` belongs to the Pod and can survive an individual container restart, but not replacement of the Pod. A persistent volume claim requests storage whose lifecycle can be independent of that Pod, under the storage system's policies. Our mock service reads immutable input, so none of these needs to hold application-owned writable state. I would add persistence only for a real requirement and define backup and concurrent-access behavior separately.

Deeper explanation. Storage access modes describe supported mounting behavior, not database transaction safety. In particular, `ReadWriteOnce` is not a universal statement that exactly one Pod can ever access the data; its node/mount semantics depend on the storage behavior. [K13 K01](#)

Example. Deleting one stateless lab Pod should not change the mock data returned by its replacement because the input is in the image.

Common weak answer. "A volume makes an application highly available" omits attachment, data consistency, failures and recovery.

Follow-up with answer. What if a future tool records progress? Choose where that state lives and how replicas access it before scaling. Do not quietly store required shared state in one Pod's local filesystem.

K8S-11 — Model answer

Interview-ready answer. The service account identifies the workload when it interacts with the Kubernetes API, and RBAC controls those API actions. The OAuth caller's token identifies and authorizes a request to the application. Granting a Pod permission to read a `ConfigMap` does not authorize an end user to invoke a tool, and a user's tool scope does not grant the Pod cluster-admin rights. Since this mock application does not call the Kubernetes API, I disable unnecessary service-account token mounting and keep its application permission checks separate.

Deeper explanation. There can be other identities at the gateway or mesh layer too, but do not collapse them into one token. Draw which verifier trusts which issuer for which resource. [I06 K11 K12 M05](#)

Example. `automountServiceAccountToken: false` reduces unnecessary credential exposure in this Pod; it does not prevent normal Service traffic.

Common weak answer. "Use the Kubernetes token as the MCP user token" ignores audience, trust and permission-model differences.

Follow-up with answer. How would you check your own API permission? Use a targeted `kubectl auth can-i` query for the intended verb/resource/namespace; do not attempt privilege escalation or broad permission changes as a diagnostic shortcut.

K8S-12 — Model answer

Interview-ready answer. I would first confirm the namespace and desired Deployment, then inspect the Pod's conditions, container state and events. If it is unscheduled, I examine placement requirements and capacity. If image pulling fails, I verify the image and registry access. If it starts and exits, I read current and previous logs and the termination reason. If it runs but is unready, I inspect the readiness check and application behavior. I would preserve evidence before restarting and apply the smallest fix supported by it.

Deeper explanation. Labels such as `CrashLoopBackOff` and `ImagePullBackOff` describe waiting/retry circumstances, not a complete root-cause diagnosis. A single Pod phase can hide several important container conditions. [I06 K14 K09 K10](#)

Example. `kubectl -n workbook logs POD --previous` can expose the startup error from the last terminated container; it is not useful when no previous instance exists.

Common weak answer. “Delete the Pod until it works” may discard useful evidence while recreating the same defect.

Follow-up with answer. What if `exec` is unavailable? Use events, logs and approved diagnostic mechanisms. A minimal image or insufficient permission is not a reason to add a privileged shell to production without review.

K8S-13 — Model answer

Interview-ready answer. I would need a tested image, meaningful readiness, enough capacity for the configured surge, and application behavior compatible with old and new replicas overlapping. I would also verify graceful shutdown and client behavior during connection changes. With `maxUnavailable` zero, old healthy Pods should be retained while replacements become ready, but that is not an absolute zero-downtime guarantee. I would watch rollout status and real requests, and stop or reverse the rollout if the replacement never becomes healthy.

Deeper explanation. A deployment controller cannot guarantee that the database schema, external configuration or protocol client remains compatible. These are release prerequisites outside its Pod-template management. [I06 K02 K09 K14](#)

Example. `kubectl -n workbook rollout status deployment/status-lab --timeout=120s` provides rollout evidence, followed by an actual SDK tool-call check.

Common weak answer. “Two replicas means zero downtime” ignores readiness quality, shared failures, available capacity and in-flight connections.

Follow-up with answer. What does rollout undo not undo? External database changes, provider configuration, secret revocation and arbitrary side effects. It restores a prior Pod-template revision, not the entire world.

K8S-14 — Model answer

Interview-ready answer. The Pod-level success suggests a functioning listener for that path, but it does not prove Service selection or the in-cluster network path. I would compare Service selectors with Pod labels, inspect `EndpointSlices` and readiness, and verify Service port and `targetPort`. Then I would test the Service DNS name from the relevant namespace and consider network policy or environment-specific routing. I would avoid rewriting the application until evidence points there, and I would retest through the same Service path after the fix.

Deeper explanation. A mismatch may select zero Pods or the wrong Pods; a correct selector with an incorrect target port is another independent failure. A direct Pod test bypasses both possibilities. [K04 K05 K06](#)

Example. If the Service selects `app: status-api` while the Pod template labels `app: status-lab`, correct the intended selector and verify the endpoint set.

Common weak answer. “Port-forward worked, so cluster networking must be fine” overgeneralizes a different request path.

Follow-up with answer. How would this differ in Kyma? Inspect its gateway/mesh policies as an additional boundary. Do not assume an APIRule-protected workload has the same internal-access behavior as this ordinary local cluster.

4.9 Observable checkpoint

Explain the Deployment → ReplicaSet → Pod chain; create and inspect the lab namespace resources; show exactly why the Service selects its backends; demonstrate that readiness failure and liveness failure have different effects; distinguish a request from a limit; and explain why this Pod has no mounted Kubernetes API token.

For diagnosis, preserve evidence for one scheduling/startup failure and one traffic failure. For justification, explain why the lab is stateless, private and not automatically “zero downtime.” These are application-operator skills, not a requirement to administer the cluster control plane.

5. Implement the MCP service and understand its auth boundary

Stage / track: 2, parallel. **Courses:** continued TypeScript/JavaScript [C03](#) [T03](#) [T04](#) [T05](#) [T06](#) [T07](#) [T08](#), selected HF Unit 2 [C05](#), official version-matched SDK examples [M01](#) [M07](#), authorization overview [M12](#). **Attempt after:** modules, promises, async/await and npm basics; then server/client examples. **Prerequisites:** Chapter 2's types, runtime validation and MCP roles. **Outcomes:** build a small read-only tool, use a compatible client, distinguish HTTP/protocol/domain errors, and explain token validation without yet designing a complete identity system.

5.1 Async means waiting without monopolizing the event loop

Node can start an I/O operation and resume JavaScript when a result is available. A promise represents an eventual fulfillment or rejection. `await` suspends the surrounding async function until the promise settles; it does not make a CPU-heavy loop non-blocking. Other callbacks need the event loop to become available. This is why a supposedly asynchronous handler can still delay every health request if it performs expensive synchronous work. [T04](#) [T05](#) [T06](#)

```
async function fetchStatus(url: string) {
  const response = await fetch(url, {
    signal: AbortSignal.timeout(3000),
    redirect: 'error',
  });
  if (!response.ok) throw new Error(`upstream_status_${response.status}`);
  return response.json(); // still needs runtime validation at the boundary
}
```

This is a focused illustration, not a new required upstream service. It shows three distinct responsibilities: a bounded wait, an HTTP-status check and later validation of the body. A fulfilled fetch promise can still represent HTTP 500; a rejected promise is not the only failure channel. Python's `async def` comparison helps with suspension, but do not infer identical runtime scheduling or library behavior from similar syntax.

Place `await` inside the `try` that is meant to handle a rejection. Starting an async operation and ignoring its returned promise can lose error handling and produce a response before the work finishes. Conversely, awaiting two independent operations sequentially may unnecessarily serialize them. `Promise.all` can express concurrency, but unbounded fan-out can overwhelm a dependency. Keep this lab's handler small and deterministic before exploring parallelism.

Useful addition: timeouts and bounded retries. These belong beside async behavior because a stuck external call consumes time and capacity even when it does not block the event loop. A read-only lookup may be retryable for selected transient failures, but malformed input or insufficient permission is not fixed by repeated attempts. Use a total deadline, bounded attempts and backoff where justified; do not attach an infinite retry loop to every catch block.

5.2 npm is part of the build contract

`package.json` describes direct dependencies and scripts. The lockfile records a resolved dependency tree. `npm install` is used when establishing or deliberately updating that resolution; `npm ci` consumes an existing matching lockfile and fails rather than silently reconciling a mismatch. It removes an existing `node_modules` tree before the clean installation. The exact installation flags affecting dependency layout must also be consistent. [T07](#)

The reference archive intentionally contains no invented lockfile because registry access was unavailable during authoring. Your first bootstrap is therefore:

```
cd service
npm install
npm test
npm ci
```

Inspect the generated lockfile, confirm tests, and commit it for your learning project. After that, Docker's COPY package.json package-lock.json ./ and RUN npm ci use the actual lock. If you skip initial resolution, the Docker build should fail rather than pretend a lockfile exists.

Lifecycle scripts may execute package-provided code during installation. Disabling them can reduce one execution path, but can also omit necessary native builds or generated assets. --ignore-scripts is not a universal safe replacement for dependency review. Explicit npm run build still runs the requested script, while associated pre/post hooks are affected by the option's documented semantics. Diagnose a resulting missing module or binary by inspecting the package's requirements rather than assuming the flag is harmless. T08

The TypeScript compiler and Node type declarations are development dependencies. The MCP SDK, Zod runtime schema and jose verifier are runtime dependencies. The final image contains emitted code, runtime libraries, package metadata needed for ESM behavior, and the mock data file. A compiler success alone does not test that final set.

5.3 Register one tool and keep the domain separate

The reference implementation creates an MCP server, registers read_status, and attaches a Streamable HTTP transport. makeMcp receives a validated status table and a request-specific caller context. The schema constrains the service name; the handler checks status: read, performs the domain lookup, and returns a text content item containing the mock result. M01 M03 M07

```
server.registerTool('read_status', {
  description: 'Read mock service status',
  inputSchema: { service: z.string().regex(/^[a-z][a-z0-9_-]{0,31}$/) },
  annotations: { readOnlyHint: true },
}, async ({ service }) => {
  // Reference solution also checks the caller's permission here.
  const result = readStatus(table, service);
  return result
    ? { content: [{ type: 'text', text: JSON.stringify(result) }] }
    : { isError: true, content: [{ type: 'text', text: 'unknown_service' }] };
});
```

This teaching extract omits the permission branch only to isolate registration; **the complete reference implementation includes it**. A read-only annotation does not enforce the operation or prove that the data is non-sensitive. A model-generated service name is still untrusted input.

The HTTP layer handles /healthz, /readyz, the protected-resource metadata path and /mcp separately. The lab's server has no session store and constructs request-scoped protocol objects with the v1 SDK's stateless transport setting. That makes the mock's replication model simpler. It does not mean every v1 server is stateless, nor that the old protocol initialization sequence disappears merely because this transport setting is used. M02 M04 M07

5.4 Error behavior should identify the failed layer

A transport cannot report a tool result before it has a usable protocol interaction. Missing credentials can therefore be an HTTP authentication failure. A malformed protocol request or

unavailable method is a protocol-level failure. An accepted tool call whose service name is valid but unknown in the domain can be a tool execution error. The SDK's exact validation/error representation must be checked for its version rather than replaced with a hand-written approximation. [M03](#)

The complete reference uses these observable categories: unknown HTTP path → 404; unsupported `/mcp` method in this JSON-only stateless lab → 405; missing or invalid token → 401; missing connection-level scope → 403; authenticated call lacking the tool scope → an MCP error result; unknown mock service → an MCP error result. Therefore HTTP 200 can still carry `isError: true` for a tool operation. A monitor or test must inspect the relevant layer, not only the HTTP status.

The terminal client connects with the matching v1 SDK, lists tools, then invokes `read_status`. `curl /healthz` is a useful first test, but never a substitute for that interaction. The local teaching client is deliberately loopback-only, rejects redirects, and is not a complete remotely deployable OAuth client. It demonstrates the integration while keeping production credential-handling responsibilities out of a beginner's custom implementation.

5.5 OAuth roles: who issues, who presents, who verifies?

OAuth separates the application requesting access, the authority granting it, and the resource enforcing it. Here the MCP client obtains and presents an access token. The authorization server issues it according to its policies. The MCP server is the protected resource server that validates the token and authorizes the requested operation. An identity provider may authenticate the human involved, often as part of the same product, but its identity role is not interchangeable with the resource server's permission check. [M05](#) [O01](#)

Authentication establishes an identity or credential validity in the relevant context. Authorization determines which operation that identity may perform. A valid token can still lack permission for `read_status`. A client-credentials token can represent a workload rather than a human; do not assume every valid token implies a person logged in.

```
client ---- discovery / authorization ----> authorization server
client <----- access token ----- authorization server
client ---- bearer token + MCP request --> resource server
resource server ---- validate token, then operation policy ----> handler
```

The arrows do not imply that the resource server sends the user's password to the identity provider. They show separate protocol interactions and trust decisions. OIDC adds identity-oriented concepts such as ID tokens; an ID token intended for a client is not automatically the access token an API should accept. [O11](#)

5.6 Discovery and token validation at introductory depth

Protected-resource metadata tells a client which authorization server(s) are associated with the resource and can advertise supported scopes. Authorization-server metadata tells a client where the issuer's endpoints and capabilities are. These are different documents serving different roles. A resource server may point to metadata through its authentication challenge; the client must follow its version's discovery rules and verify the resulting issuer and resource relationship. [O02](#) [O03](#) [M05](#)

A token that looks like a JWT can be decoded without proving authenticity. Validation requires an appropriate trusted verification method, expected issuer and audience, time constraints and the provider's token profile. An opaque token cannot be validated by guessing at dot-separated fields;

the resource may use a supported introspection mechanism or another provider-approved method. OAuth does not require every access token to be a JWT. [O06](#) [O07](#)

The reference defaults to JWT authentication and fails startup when required configuration is absent. An explicit local-lab opt-in enables unauthenticated mock operation for the private Docker/Kubernetes exercises. That shortcut is never an acceptable public-deployment default. Full metadata, registration, PKCE, key rotation, scope policy and negative tests belong in Chapter 8, not in a homemade authentication system here.

5.7 Questions—attempt before the answer key

TS-08 — Node threads and a blocked health check

Provenance: A · published guide, adapted from I02 threading question. **Sources:** [I02](#) [T06](#). **Priority:** Core. **Type:** explanation.

Is Node simply single-threaded? Why can a CPU-heavy handler delay a health endpoint?

Testing: JavaScript execution versus runtime concurrency.

TS-09 — Async errors need a real boundary

Provenance: A · published guide, adapted from I02 error-handling question; promise extension is practice. **Sources:** [I02](#) [T04](#) [T05](#). **Priority:** Core. **Type:** explanation.

Compare an error-first callback with a rejected promise, and explain how you would contain an async handler failure.

Testing: Error propagation and meaningful responses.

TS-10 — A reproducible install is not a magic flag

Provenance: B · documentation-based practice. **Sources:** [T07](#) [T08](#). **Priority:** Core. **Type:** comparison.

Compare npm install and npm ci, and explain the possible effects of --ignore-scripts.

Testing: Lockfile consistency and installation-time code.

MCP-08 — Registering a tool is not writing the protocol

Provenance: B · documentation-based practice. **Sources:** [M01](#) [M03](#) [M07](#). **Priority:** Core. **Type:** explanation.

What responsibilities remain yours when the SDK registers read_status?

Testing: SDK versus application contracts.

MCP-09 — Valid shape, unknown service

Provenance: C · original error-classification scenario. **Sources:** [M03](#) [T01](#). **Priority:** Core. **Type:** comparison.

Compare an invalid service argument, a valid but unknown service, and a server exception.

Testing: Validation, domain errors and defects.

MCP-10 — Health is not protocol integration

Provenance: C · original test-review scenario. **Sources:** [M02](#) [M03](#) [M04](#). **Priority:** Core. **Type:** configuration review.

A deployment check only curls /healthz and declares the MCP integration complete. What is missing?

Testing: Layered verification.

MCP-11 — Replicas and statelessness

Provenance: B · documentation-based practice. **Sources:** [M02](#) [M04](#) [M07](#) [K02](#). **Priority:** Useful. **Type:** design.

Why can this lab run behind two replicas more simply than a server with process-local session state?

Testing: Application state versus transport sessions.

MCP-12 — Client cannot connect or discover

Provenance: C · original troubleshooting scenario. **Sources:** [M02](#) [M04](#) [M05](#) [M09](#). **Priority:** Core. **Type:** troubleshooting.

The HTTP endpoint responds, but the MCP client cannot complete connection or discovery. Give the next evidence-driven steps.

Testing: Transport, version, path and auth separation.

MCP-13 — Timeouts and safe retries

Provenance: C · original small design scenario. **Sources:** [T04](#) [T06](#) [T10](#). **Priority:** Useful. **Type:** design.

A future read-only status dependency sometimes times out. How would you add retries without creating an outage amplifier?

Testing: Bounded latency and operation semantics.

AUTH-01 — OAuth roles in this project

Provenance: B · documentation-based practice. **Sources:** [M05](#) [O01](#) [O11](#). **Priority:** Core. **Type:** explanation.

Name the client, authorization server, resource server and identity-provider role.

Testing: Who requests, issues and enforces access.

AUTH-02 — Authenticated but forbidden

Provenance: B · documentation-based practice. **Sources:** [O04](#) [M03](#) [M05](#). **Priority:** Core. **Type:** comparison.

How can a valid token still be insufficient for a tool call?

Testing: Credential validity versus operation permission.

AUTH-03 — Two metadata documents

Provenance: B · documentation-based practice. **Sources:** [O02](#) [O03](#) [M05](#). **Priority:** Core. **Type:** comparison.

Distinguish protected-resource metadata from authorization-server metadata.

Testing: Discovery without confusing issuer and audience.

AUTH-04 — Decoding versus validating

Provenance: B · documentation-based practice. **Sources:** [O07](#) [O13](#) [M05](#). **Priority:** Core. **Type:** explanation.

A token decoder shows the expected username and an unexpired exp. Why can the server still reject it?

Testing: Cryptographic and semantic validation.

AUTH-05 — OAuth does not imply JWT

Provenance: B · documentation-based practice. **Sources:** [O06](#) [O07](#) [O11](#). **Priority:** Core. **Type:** comparison.

How would validation differ for an opaque access token?

Testing: Token-format independence and provider contracts.

5.8 Answer key

TS-08 — Model answer

Interview-ready answer. JavaScript callbacks normally execute on an event loop, while the runtime and operating system can perform other work concurrently, and worker threads are available when explicitly used. Calling Node single-threaded without qualification is therefore misleading. A long synchronous JavaScript computation blocks that event loop, delaying unrelated callbacks such as health requests. Making the function async does not move its CPU work elsewhere. I would identify the actual blocking operation before considering algorithm changes, workers or separate replicas.

Deeper explanation. The published guide contains older threading commentary; the current answer uses Node's official execution model. I/O concurrency, worker-pool activity and parallel JavaScript execution are related but distinct concepts. [I02](#) [T06](#)

Example. An async handler containing a large synchronous loop still prevents that event loop from servicing another request until the loop yields or finishes.

Common weak answer. "Async uses another thread for the whole function" assigns semantics that async/await does not provide.

Follow-up with answer. Would Promise.all parallelize two CPU-heavy synchronous functions? Not by itself. Evaluating those functions still runs their synchronous work on the calling thread; promises coordinate completion, not automatic CPU isolation.

TS-09 — Model answer

Interview-ready answer. An error-first callback convention passes an error as the first argument and a result separately. With promises, failure is represented by rejection. In an async function, I would await the promise inside the try/catch that should handle the failure, then translate a known error into the appropriate response and log a safe diagnostic category. I would not start an async operation, ignore its promise and assume an outer synchronous catch will handle a later rejection.

Deeper explanation. Error translation should preserve the distinction between invalid input, expected domain failure and an internal defect. Swallowing every exception into an empty successful result hides defects and misleads the caller. [I02](#) [T04](#) [T05](#)

Example. `try { const value = await operation(); } catch (error) { /* classify safely */ }` handles the awaited rejection; an HTTP response still needs its own status/body checks.

Common weak answer. "Catch everything and return success so the server stays up" sacrifices correctness and observability.

Follow-up with answer. Should you expose the stack trace to the MCP user? Normally no. Return a stable non-sensitive error and correlate a request ID with protected diagnostics; stack traces can expose paths or implementation details.

TS-10 — Model answer

Interview-ready answer. I use npm install to establish or deliberately update dependency resolution, then review the lockfile. npm ci requires a matching existing lockfile, removes the installed tree and performs a clean installation without updating the manifests. Disabling lifecycle scripts can avoid executing some package-provided setup code, but may also omit required native

builds or generated files. I would inspect the dependency behavior and test the final runtime rather than treat the option as universally harmless or as a replacement for dependency review.

Deeper explanation. The reference cannot start with `npm ci` until you generate its real lockfile locally. Direct dependency pins alone do not record every transitive version. Flags affecting tree layout must be consistent when reproducing the installation. [T07](#) [T08](#)

Example. Bootstrap with `npm install`, run tests, commit `package-lock.json`, and then demonstrate `npm ci`. Explicit `npm run build` still runs its named script under the documented `ignore-scripts` behavior.

Common weak answer. “`npm ci` creates a better lockfile” is wrong; it consumes the existing matching one.

Follow-up with answer. Why copy manifests before source in Docker? So a source-only change need not invalidate dependency installation. A manifest or lock change should invalidate it because the dependency inputs changed.

MCP-08 — Model answer

Interview-ready answer. The SDK handles protocol mechanics and the transport integration for its supported version, but I still define the operation’s schema, implementation, error behavior and authorization. I would keep the status lookup separate from the MCP adapter so the domain can be tested independently. Registering a tool does not prove its description is accurate or its inputs are safe. I would test discovery, valid calls, malformed arguments, unknown service names and permission denial through a compatible client.

Deeper explanation. The selected SDK API and imports are v1-specific. Using a maintained library avoids hand-writing JSON-RPC, but it does not excuse skipping version checks or application-level tests. [M01](#) [M03](#) [M07](#)

Example. `makeMcp` receives a validated table and a request-specific Principal, then registers one read-only mock tool.

Common weak answer. “The SDK makes the tool secure” attributes application policy and implementation review to a transport library.

Follow-up with answer. Would you add resources and prompts too? Only for a concrete interaction requirement. One well-tested tool is enough to learn this deployment stack without inflating application scope.

MCP-09 — Model answer

Interview-ready answer. An invalid argument should be rejected by the runtime schema before the lookup. A valid name that is absent from our mock table is an expected domain outcome and should produce a clear unknown-service tool error. An unexpected exception is a different category: I would log a safe diagnostic and return a controlled failure without inventing data. The exact protocol representation of schema errors follows the selected SDK version; I would test it rather than assume every failure is HTTP 500.

Deeper explanation. The domain should never return operational merely to avoid an error. That would make the tool appear reliable while supplying false status information. A stable error taxonomy makes client behavior and tests more meaningful. [M03](#) [T01](#)

Example. A number instead of a service string tests schema validation; missing-service tests the allowed syntax but unknown domain; a missing data file tests startup packaging.

Common weak answer. “All errors are equivalent” prevents choosing safe retries and identifying which component needs repair.

Follow-up with answer. Does HTTP 200 prove the tool succeeded? No. A valid MCP response may contain `isError` true. Inspect both the transport/protocol response and the operation result.

MCP-10 — Model answer

Interview-ready answer. That check proves only that the health path responded through the tested route. It does not establish a compatible MCP lifecycle, tool discovery, correct schemas, successful handler execution or authorization. I would retain it as a useful first smoke test and add a matching SDK client that connects, lists tools and calls `read_status`. I would also test a negative input and, once auth is enabled, missing and insufficient-permission credentials. Each test should state the boundary it actually covers.

Deeper explanation. A health endpoint is intentionally simple and may bypass application authentication so a platform can probe it. That makes it even less appropriate as evidence that protected tool access works. [M02](#) [M03](#) [M04](#)

Example. Run `npm run client` after the health request; its `listTools` and `callTool` outputs distinguish catalog access from operation execution.

Common weak answer. “The server returned 200, so MCP works” collapses different paths and protocols into one status code.

Follow-up with answer. What if the client gets 405 on `GET /mcp`? For this POST-based JSON-only stateless lab, GET is not the health endpoint. Verify client transport/version before changing the server to accept an unrelated request shape.

MCP-11 — Model answer

Interview-ready answer. Each request uses the same immutable mock data and does not depend on earlier requests being handled by the same process. The selected v1 transport also avoids a process-local session store for this lab. That makes independent replicas straightforward. A server with in-memory sessions or mutable workflow state would need an explicit strategy for routing, shared state or a different supported protocol design. I would not assume that increasing replicas preserves correctness until I know which state is local and which interaction relies on it.

Deeper explanation. Stateless HTTP handling in the v1 SDK is not identical to the later protocol’s removal of the old session/lifecycle model. Keep transport settings and protocol-version claims distinct. [M02](#) [M04](#) [M07](#) [K02](#)

Example. The reference creates a server/transport for the request and closes it afterward rather than sharing a global current-user object between requests.

Common weak answer. “MCP is always stateless” ignores version and application behavior.

Follow-up with answer. What if a cache is added? Determine whether it is merely an optimization or required for correctness. Independent caches can be acceptable; indispensable per-process workflow state changes the replica design.

MCP-12 — Model answer

Interview-ready answer. I would record the client and server SDK/protocol versions, exact endpoint path, HTTP status and safe error message. Then I would confirm the selected transport rather than mixing legacy SSE, v1 Streamable HTTP and current-v2 examples. If the response is an authentication challenge, I would inspect metadata discovery rather than changing tool registration. If lifecycle succeeds but tools/list fails, I would inspect capabilities and the server's registration/error logs. HTTP reachability narrows the search but does not establish protocol compatibility.

Deeper explanation. A reverse proxy can route /healthz correctly while rewriting or blocking /mcp. Capture which component responded and whether the expected content type and path reached the application. [M02](#) [M04](#) [M05](#) [M09](#)

Example. Compare a direct local SDK connection with the routed connection using the same compatible client and mock call.

Common weak answer. "Reinstall the SDK until it works" changes several variables and may introduce a protocol migration unintentionally.

Follow-up with answer. Would you disable authentication to diagnose? Only in the explicit isolated local mock mode, never by exposing a real unprotected endpoint. Prefer inspecting the challenge and known test credentials first.

MCP-13 — Model answer

Interview-ready answer. I would define a total request deadline, a shorter timeout for each attempt and a small retry budget only for selected transient failures. I would use backoff and avoid synchronizing many clients into immediate retries. Invalid arguments and permission denial should not be retried as transient network failures. Although this read-only lookup is naturally safer to repeat, I would still measure dependency load and latency. A future state-changing tool would need an explicit idempotency or duplicate-handling design before automatic retries.

Deeper explanation. Async waiting can still consume sockets, memory and pending work. A timeout on receiving an HTTP request is not necessarily a timeout on every downstream operation or the complete handler. [T04](#) [T06](#) [T10](#)

Example. The teaching fetch fragment uses AbortSignal.timeout; the current mock tool has no real upstream and therefore needs no retry loop.

Common weak answer. "Retry until success" gives no bound on latency or resource consumption.

Follow-up with answer. What should the caller see after the budget is exhausted? A controlled unavailable/timeout result with a safe correlation identifier, not fabricated status data or a raw credential-bearing upstream response.

AUTH-01 — Model answer

Interview-ready answer. The MCP client obtains and presents an access token. The authorization server grants and issues that token according to its policy. Our MCP service is the resource server that validates it and decides whether the operation is permitted. An identity provider can authenticate the human involved, often within the same product, but that does not replace the resource server's authorization decision. I would draw these logical roles even when two of them are hosted by one system.

Deeper explanation. OAuth is about delegated access and related authorization flows. OIDC supplies an identity layer with additional token semantics. Product names should not obscure which role is responsible for a particular failure. [M05](#) [O01](#) [O11](#)

Example. In the local Stage 3 lab, Keycloak issues the token and the Node MCP service verifies it; the user password is entered at the local provider, not sent to the tool handler.

Common weak answer. “The MCP server logs the user in by decoding a token” conflates several steps and omits verification.

Follow-up with answer. Is an authorization server always a separate deployment? No. The roles are conceptually distinct even when colocated. The actual provider and trust configuration determine the implementation.

AUTH-02 — Model answer

Interview-ready answer. A token can be authentic, issued for this resource and within its validity window while granting only a limited set of permissions. I would first validate it, then evaluate the policy for the requested tool and resource. In the lab, `mcp:connect` permits the protocol connection, while `status:read` permits the status operation. A caller with only the first scope can still be denied the tool. I would not grant access merely because login succeeded or the token had a recognizable format.

Deeper explanation. Scopes are one input to policy, not a complete universal permissions model. Real applications may also need tenant, object ownership or application-permission checks defined by their provider and business rules. [O04](#) [M03](#) [M05](#)

Example. The reference tool returns `permission_denied` as an MCP error result when the connected principal lacks `status:read`.

Common weak answer. “Authenticated users can call every tool” discards least privilege.

Follow-up with answer. Must denial always be HTTP 403? No. A transport-level scope failure may be 403; a tool-specific denial after a valid protocol interaction can be represented as a tool error. Test the selected layer.

AUTH-03 — Model answer

Interview-ready answer. Protected-resource metadata describes the API resource and which authorization servers it uses. Authorization-server metadata describes the issuer and endpoints such as authorization and token endpoints, plus supported capabilities. The client starts from the resource and follows the appropriate discovery rules to find a trusted authorization flow. These documents do not themselves grant access. I would verify that the discovered issuer and requested resource match the intended deployment before obtaining or sending a token.

Deeper explanation. A resource authentication challenge can point at its metadata URL. Path-bearing issuers require the version’s documented discovery construction rather than blindly appending one suffix in every case. [O02](#) [O03](#) [M05](#)

Example. The lab resource publishes `/.well-known/oauth-protected-resource`, while Keycloak exposes its own realm discovery document.

Common weak answer. “Both `.well-known` URLs are the login endpoint” mistakes metadata for an interactive authorization endpoint.

Follow-up with answer. What if metadata points to an unexpected issuer? Stop and investigate trust/configuration rather than sending credentials or tokens there. Discovery is input that must be validated, not unconditional permission to follow any URL.

AUTH-04 — Model answer

Interview-ready answer. Decoding only makes the encoded fields readable. The server must verify the signature with trusted key material and check the expected issuer, intended audience, time constraints and token profile. A readable username and future expiration can be forged in an unsigned or wrongly signed token, or belong to a token intended for a different API. I would use a maintained validation library and provider documentation, then apply permissions only to the successfully validated claims.

Deeper explanation. A key identifier helps select a key but is not itself a trust anchor. The server must obtain keys from a trusted configured issuer relationship, not an arbitrary URL supplied by the token. [O07](#) [O13](#) [M05](#)

Example. The reference uses `jose.jwtVerify` with configured issuer, resource audience, RS256 policy and required time/subject claims.

Common weak answer. “Base64 decoding is JWT verification” omits the essential authenticity and recipient checks.

Follow-up with answer. Could a legitimate signature still fail? Yes. Wrong audience, wrong issuer, expiration or an inappropriate token type can make a correctly signed token unsuitable for this resource.

AUTH-05 — Model answer

Interview-ready answer. An opaque token is not something the resource should interpret by guessing its internal structure. The provider may supply token introspection or another approved validation mechanism that reports whether it is active and what access it represents. A JWT can support local cryptographic verification under a suitable trust and claim-validation policy. OAuth does not force one format. I would first identify the provider’s access-token contract and not substitute an ID token just because it is easier to decode.

Deeper explanation. Introspection adds a runtime dependency and raises caching/revocation tradeoffs; local JWT verification has its own key-distribution and revocation considerations. Neither is universally superior for every provider. [O06](#) [O07](#) [O11](#)

Example. A failed attempt to split an opaque token on dots is a parser assumption, not evidence that the credential is invalid.

Common weak answer. “Every OAuth token has three base64 sections” confuses a common format with the authorization framework.

Follow-up with answer. Can the ID token be sent to the MCP API instead? Not by default. Its audience and purpose are generally the client’s identity interaction, not authorization to this resource server.

5.9 Observable checkpoint

Build the project from a real lockfile, explain the role of each dependency, and describe what would break if installation scripts were disabled. Demonstrate discovery and a successful tool call as

separate checks. Show an unknown-service error without crashing the process. Explain client, authorization server and resource server without asserting that OAuth always means JWT. Identify the explicitly disabled local-auth setting and show why a missing production auth configuration must not silently select it.

6. Stage 2 practice, checkpoint and mock interview

Stage / track: 2, both tracks. **Course points:** ordinary Kubernetes deployments and Services, focused probes/resources lessons, TypeScript async/build topics, and the selected MCP implementation examples. **Prerequisites:** Chapters 4–5 and the Stage 1 service. **Outcome:** connect an SDK client to a read-only tool, then carry that exact application through a private local Kubernetes deployment. Authentication is traced here and implemented in the Stage 3 local identity-provider exercise.

6.1 Exercise E3 — Add the MCP adapter without enlarging the product

Objective. Turn the existing status lookup into one validated, read-only MCP tool. **Tools and versions.** Node 22.x, npm 10.x and the exact package selections in `service/package.json`; the SDK is v1.30.0, not the differently packaged v2. **Files.** `service/src/domain.ts`, `mcp.ts`, `server.ts`, `client.ts`, `configuration/auth` modules, `package.json`, `tsconfig.json`, the data file and Dockerfile. Appendix A contains their full reference contents.

Independent task. Before opening `mcp.ts`, write the input contract, three error cases and the boundary between domain logic and transport. Implement your own adapter against the selected SDK documentation. Keep the mock table as data; do not add a database, model provider or employer API. Decide which failures should stop startup and which belong to an individual call. [M01](#) [M03](#) [M07](#)

Bootstrap the dependency resolution once, inspect what changed, and retain the resulting real lockfile:

```
cd service
npm install
npm test
npm ci
npm run build
AUTH_MODE=disabled ALLOW_UNAUTHENTICATED_LOCAL_LAB=true \
  HOST=127.0.0.1 npm start
# Another terminal, from service/:
npm run client
SERVICE_NAME=missing-service npm run client
```

`npm install` is the intentional first resolution, not a reproducibility claim. The supplied archive has no fabricated lockfile. After reviewing and committing the generated lockfile, use `npm ci` for repeat installations. Native setup may fail if a dependency or platform requires an unavailable installation step; diagnose that rather than adding `--ignore-scripts` blindly. [T07](#) [T08](#)

Expected observations, not executed integration results. Tests should pass, the client should list `read_status`, `api` should return operational, and a syntactically valid but unknown name should produce an explicit tool error rather than invented status. A health request should succeed independently of the MCP call. The client uses the SDK to perform the v1-family protocol interaction; a plain curl against `/healthz` does not replace it.

Independent negative tests. Make one client call with an argument of the wrong type, one with an unknown valid name, and one with a missing tool name. Record the observed transport status and protocol result separately. Do not assume all three become HTTP 500. Restore the reference client after each change. The schema and SDK determine invalid-argument/protocol behavior; the domain adapter explicitly determines the unknown-service result. [M03](#)

Hints. First test `readStatus` without networking. Next check that your tool name and schema match the client request. Finally compare `makeMcp` and the stateless transport construction with the version-matched example. The server creates a request-scoped principal; no shared global variable may represent “the current user.”

Reference solution. `parseTable` validates startup data. `makeMcp` registers the Zod-validated tool and closes over that table plus the request principal. The handler checks `status:read`, returns a meaningful tool error when it is absent, looks up the requested service and converts the result to MCP content. The HTTP server handles transport, a minimal health/readiness interface and a separate authentication boundary. `client.ts` connects, lists tools, invokes the operation and closes. These are distinct responsibilities, not one large handler.

Now build the final-stage application image, after stopping the native listener on port 3000:

```
docker build -t status-lab:0.1.0 .
docker run --rm --name status-lab \
  -p 127.0.0.1:3000:3000 \
  -e AUTH_MODE=disabled \
  -e ALLOW_UNAUTHENTICATED_LOCAL_LAB=true status-lab:0.1.0
# Another terminal, service/:
npm run client
```

Break it. In a disposable Dockerfile remove the runtime-stage copy of `data/`. Build a different tag, start it and compare native success with final-image failure. Ask what changed between execution environments; inspect exit state and the startup file error before blaming MCP. The smallest fix is to restore the required asset in the final stage, not copy the entire workstation or weaken validation. Verify the final image with the same client. Prevent recurrence with a final-artifact smoke test that exercises actual file loading.

Useful extension — bounded I/O. It belongs here because an async handler can otherwise wait indefinitely on a future dependency. Add an `AbortSignal.timeout` to a disposable mock fetch, test delayed responses, and return a clear timeout error. Do not retry permission errors.

`server.requestTimeout` governs receiving a request, not every unit of application work; a handler needs its own bounded operations. [T10](#)

Cleanup. Stop the named `--rm` container and native process. Retain the reviewed lockfile; remove only the broken-image tag and scratch files. **Interview evidence:** TS-08-10, MCP-08-13, DKR-08 and AUTH-01-05. The full dependency/MCP/Docker sequence must be verified locally; it was not executed in the authoring environment.

6.2 Exercise E4 — Ordinary Kubernetes YAML first

Objective. Deploy the same image behind a Service, prove reconciliation and routing, and diagnose a probe failure. **Tools.** Docker, a supported kind release, its documented node image and a compatible `kubectl`. Record `kind version`, `kubectl version --client` and the server version. This is a disposable local cluster, not an employer cluster. **Files.** `k8s/status-lab.yaml`; no Helm is required. [K15](#)

Independent task. Draft a Deployment with two replicas and a ClusterIP Service before inspecting the solution. Include matching labels/selectors, the actual container listener port, startup/readiness/liveness probes, modest explicit resource values and no automatic service-account token mounting. Explain each resource setting as a lab starting point, not as measured production sizing.

From the repository root:

```
kind create cluster --name workbook
kind load docker-image status-lab:0.1.0 --name workbook
kubectl config current-context
kubectl apply -f k8s/status-lab.yaml
kubectl -n workbook rollout status deployment/status-lab --timeout=120s
kubectl -n workbook get pods,svc,endpointslices
kubectl -n workbook port-forward service/status-lab 3000:80
```

Check that the context is the intended local cluster before applying. If a managed sandbox does not allow namespace creation, use its approved namespace and adjust the manifests explicitly; do not request cluster-admin merely to follow an exercise. The supplied namespace creation is for this disposable kind lab.

In another terminal, run the Stage 2 client from `service/`. **Expected observations:** two Ready Pods, a Service whose backend endpoints match the intended Pods, successful health and tool calls through the forwarded connection. Port-forward is a useful application-path check, but it is not proof that the full in-cluster Service data path works. [K05](#) [K06](#)

For a separate in-cluster check, execute a request from a Pod using the already present Node runtime:

```
kubectl -n workbook exec deployment/status-lab -- node -e \
  'fetch("http://status-lab/healthz").then(r=>console.log(r.status))'
kubectl -n workbook describe service status-lab
kubectl -n workbook get endpointslices \
  -l kubernetes.io/service-name=status-lab -o wide
```

A health request proves less than an MCP operation. The source Pod, DNS name, Service route and application response still provide useful separate evidence. A production-like independent diagnostic Pod is optional and must use an approved image and permissions, not an arbitrary download into the running application.

Reconciliation demonstration. Delete one named Pod only in this local lab and observe the Deployment replace it. Compare the old and new Pod names/IPs with the stable Service name. Do not delete the Deployment when the question is about replacing one managed Pod. [K02](#) [K03](#)

Break it — liveness failure after a probe edit. Change the liveness path in a disposable manifest to `/does-not-exist`, apply it and watch the restart count and events. Ask which probe changed, whether the application exits on its own, and whether startup completed. Compare current and previous logs, then inspect the probe definition and events. A 404 from the configured path is evidence of a wrong health contract, not proof of a memory leak.

```
kubectl -n workbook get pods -w
kubectl -n workbook describe pod POD_NAME
kubectl -n workbook logs POD_NAME --previous
kubectl -n workbook get events --sort-by=.metadata.creationTimestamp
```

Hints. Begin with the container's last termination state and event reason. Then compare the configured path with the starter's routes. Finally restore `/healthz`, apply, wait for the rollout and verify that restart counts stop increasing over several probe intervals. Do not "fix" a wrong path by setting an enormous failure threshold. [K09](#) [K14](#)

Reference solution. The provided multi-document YAML creates a namespace, non-privileged service account, ConfigMap, Deployment and Service. Named port `http` points to container port 3000; Service port 80 targets that name. Startup protects warm-up, readiness controls normal Service eligibility, and liveness tests the deliberately minimal process endpoint. The application runs

non-root with a read-only root filesystem and does not need to call the Kubernetes API. Its mock file is packaged with the image, so this milestone has no persistence requirement.

Second diagnostic variation. Change only the Service selector to a label that no Pod has. Expected result: no matching backends, not necessarily a dead application. Compare labels, selector and EndpointSlices; restore the selector and verify the original request. This isolates routing from process health. [K04](#) [K06](#)

Stretch. Make a rollout reference an unavailable image and explain why the old healthy replicas may remain with `maxUnavailable: 0`, while the new ReplicaSet cannot progress. Inspect events and image identity; choose a known good tag/digest instead of deleting every Pod. Availability also depends on capacity for the surge and real readiness. [K02](#)

Cleanup. Stop port-forward. `kind delete cluster --name workbook` destroys this entire disposable lab cluster. Never run it against an unrelated cluster name. If retaining the cluster, delete only the named lab resources; note that `kubectl delete -f k8s/status-lab.yaml` also deletes the workbook namespace and everything in it. **Interview evidence:** K8S-01-14, MCP-10 and MCP-12.

6.3 Stage 2 checkpoint and targeted scoring

S2-CB1. Trace Deployment → ReplicaSet → Pod and Service → selected ready endpoints. **Answer:** the controller chain maintains desired replicas; the Service selects matching Pods and supplies a stable access abstraction. A Service does not create the application Pods. See 4.1–4.3 and K8S-01-04.

S2-CB2. Explain startup, readiness and liveness without saying “all restart the Pod.” **Answer:** startup gates other probes while startup succeeds, readiness governs normal traffic eligibility, and liveness failure can restart the container. Configuration and thresholds matter; neither a Ready condition nor a liveness success proves the whole business workflow. See 4.5 and K8S-08.

S2-CB3. Explain why `async` does not fix CPU blocking and why a lockfile matters. **Answer:** promises organize completion; synchronous JavaScript still occupies the event loop. A lockfile records the dependency resolution used by clean installation, but external inputs still affect complete reproducibility. See 5.1–5.2 and TS-08-10.

S2-CB4. Explain authentication, token validation and per-tool authorization separately. **Answer:** the caller presents credentials, the resource server establishes whether they are acceptable for this issuer/resource/time and token format, and application policy determines which action that principal may perform. A decoded string is not proof of any of these. See 5.5–5.6 and AUTH-01-05.

S2-T. A Service exists, but the client cannot call the tool. **Complete approach:** obtain exact URL/error and source location; check rollout and readiness; inspect Service selectors/ports and endpoints; test process health; test the protocol with a compatible client; then inspect authentication or tool errors if an HTTP/MCP response is actually reaching the application. Do not tune OAuth while no packets can reach the listener, and do not change selectors after a `clear insufficient_scope` response.

S2-D. Design a two-replica read-only service without local session dependence. **Model:** place immutable mock data in the image, inject non-secret deployment configuration, use a Deployment and stable Service, make readiness reflect admission to useful work, validate each request and attach identity to that request. Explain that load balancing does not replicate in-memory session state. The selected stateless JSON-response transport avoids needing a session store for this exercise; that is a deliberate scope choice, not a universal MCP architecture.

Use the Stage 1 five-dimension 0–4 rubric. Ready to continue: 14–20 with no unresolved confusion between network reachability and authorization. Revise probes/resources in 4.5 if restarts and unready states are conflated; Services in 4.3 if selectors/target ports are unclear; async/npm in 5.1–5.2 if builds or execution are unclear; MCP/auth boundaries in 5.3–5.6 if an HTTP 200 is treated as proof of a successful authorized tool call. Stretch mastery is optional.

6.4 Stage 2 mock interview — 35 minutes

Architecture, 6 minutes. “Explain the objects in your deployment.” **Model:** “The Deployment requests two replicas and manages their ReplicaSets; each Pod runs the same image. A Service selects their labels and routes to the named application port. Configuration is supplied separately, and the service account has no API token because this app does not need Kubernetes API access.”

Follow-up: “What survives a replacement?” **Answer:** the image/configuration specification and Service abstraction, not the old process memory or Pod identity.

Troubleshooting, 10 minutes. “The new version has no healthy replacement Pods.” **Model:** inspect rollout, new ReplicaSet, Pod scheduling/pull/start/probe events and available capacity in that order, preserving old healthy instances while investigating. **Branch: Pending.** Check scheduling constraints and requests. **Branch: ImagePullBackOff.** Check exact image, loading/registry access and pull credentials. **Branch: Running but not Ready.** Check readiness path, startup and dependencies. Do not claim all three are the same failure.

Implementation, 7 minutes. “Explain one tool call and its error boundaries.” **Model:** the selected SDK processes the protocol request, the adapter validates arguments and checks permission, the pure lookup accesses mock data, and the result distinguishes domain failure from transport failure. **Follow-up:** “Can you prove it using curl health alone?” **Answer:** no; use the compatible SDK client and negative cases.

Security, 7 minutes. “A token has a sensible payload. Why might it be rejected?” **Model:** its signature, algorithm, issuer, audience, time claims or required policy may be unacceptable; token format could also differ from the expected JWT profile. Decode only for inspection, validate through a trusted mechanism, and do not paste bearer tokens into third-party websites. **Follow-up:** “Does Kubernetes RBAC decide the tool permission?” **Answer:** not automatically; Kubernetes API permissions and application tool policy protect different resources.

Reflection, 5 minutes. “What would you investigate next?” **Model:** identify one unverified boundary, name the evidence and safe environment needed, and explain a minimal test. Evaluate reasoning, command choice, version awareness and honesty—not whether every command is recalled from memory. No claim of having operated employer production systems is required.

STAGE 3 — HELM, DEPLOYMENT INTEGRATION AND AUTHORIZATION

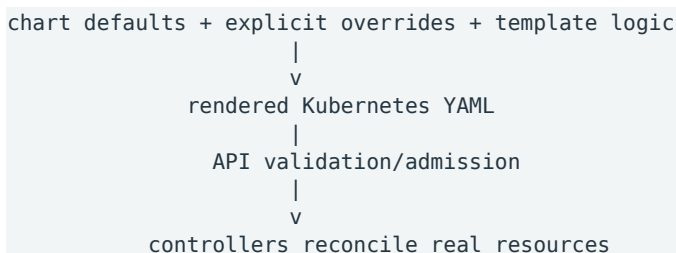
7. Main track: Helm and the Kyma boundary

Stage / track: 3, main. **Course / reference:** roadmap [9], KodeKloud Helm for Beginners and the official Chart Template Guide; roadmap [8] for Kubernetes behavior; official Kyma documentation for exposure. **Attempt after:** chart structure, values, templates and release-lifecycle topics, following ordinary Kubernetes YAML. **Prerequisites:** explain a Deployment, Service, selectors, readiness and runtime configuration. **Outcomes:** render and review a chart, operate a local release, diagnose ineffective values, explain rollback limits, and trace a version-appropriate Kyma exposure path without inventing your employer's settings. Mapping is topic-level, not a claim about paid lecture numbering. C07 H01 H09

7.1 The problem Helm solves—and the problem it does not

The Stage 2 YAML works, but copying it into separate development and sandbox directories creates a maintenance problem. A port fix or security-context change can be applied in one copy and forgotten in another. Helm lets you package a parameterized set of Kubernetes resources, render it with a chosen configuration and track a named installation. The package is a **chart**; a particular installed instance is a **release**. A repository distributes packages, but local chart development does not require publishing one. H09

The useful mental model is a small compiler plus a release tool, not a second Kubernetes control plane:



The first arrow substitutes values and evaluates template logic. The next boundary is the cluster API and its policies. The last is the controllers and runtime already learned in Stage 2. A valid render does not prove that an image exists, a Pod will schedule, a certificate is trusted or an authorized tool call will succeed. Kubernetes remains responsible for reconciliation. That is why you learned ordinary YAML first. H05 K02

In the practice service, templates do not change domain behavior. They consistently produce the Deployment, Service, ConfigMap and service account. One local release can use one replica for a constrained laptop; another approved environment can use two. This is a configuration choice, not automatic proof of high availability. A chart that exposes every field as a value is not necessarily easier to maintain: offer the settings operators actually need and keep structural invariants clear.

7.2 Chart structure, naming and the rendered contract

Chart.yaml identifies the chart and its package version. appVersion describes the application version but is not an automatic instruction to change the image tag unless the templates deliberately use it. values.yaml supplies defaults. templates/ contains resource templates; _helpers.tpl holds reusable

named fragments. A `values.schema.json` can reject invalid value types or missing inputs before an operator reaches cluster-side validation. [H09](#)

```
chart/status-lab/
  Chart.yaml           package identity
  values.yaml          reviewed defaults
  values.schema.json   values contract
  templates/_helpers.tpl names and selector labels
  templates/deployment.yaml desired application process
  templates/service.yaml stable in-cluster access
  templates/configmap.yaml non-secret application config
  templates/serviceaccount.yaml
```

The reference uses release `workbook` and chart `status-lab`, so the full resource name is `workbook-status-lab`. That naming choice also affects Service DNS and the application's allowed-host configuration. Changing a release name is not simply changing a decorative label. Read the rendered names and verify that external references agree.

Use a stable subset of labels for selectors and a broader set for descriptive metadata. A chart version or mutable application version can be useful metadata, but making it part of an immutable Deployment selector creates an upgrade problem. Helpers reduce accidental disagreement among Deployment selectors, Pod labels and Service selectors. They are reusable text templates, not independent Kubernetes resources. [H04](#) [K04](#)

7.3 Values precedence, Go-template context and YAML types

Values have an order of precedence. Chart defaults can be overridden by supplied values files, and later specified files or command-line values can override earlier choices. During an upgrade, flags such as `--reuse-values` affect whether previously supplied values are carried forward. For predictable learning and review, prefer a small explicit environment file plus a documented command rather than a remembered collection of shell overrides. Inspect the actual release values when diagnosing an existing deployment. [H01](#) [H06](#)

This fragment illustrates three separate contracts:

```
# In a template, not plain Kubernetes YAML:
ports:
  - name: http
    containerPort: {{ .Values.containerPort }}
env:
  - name: PORT
    value: {{ .Values.containerPort | quote }}
```

The container port is numeric because the Kubernetes field is numeric. The environment value is a string because environment variables carry strings. Quoting everything indiscriminately is no better than quoting nothing. The application then parses `PORT` and validates its range. That final application check does not become unnecessary because Helm had a schema.

`if` includes a block conditionally; `range` iterates values; `with` changes the active context. Inside a changed context, `.` may no longer refer to the original top-level object. Capture or use the root context deliberately when reaching release-level information. `include` returns helper output so it can be passed through functions such as `nindent`; indentation must still place the resulting lines under the correct YAML key. Whitespace trimming can accidentally join two YAML tokens if used carelessly. [H02](#) [H03](#) [H04](#)

A practical review asks: What is the value's exact key path? Was the intended override file supplied? Did a later file replace it? Does the template reference it? Did the result have the intended YAML type and nesting? Only after the render is correct should you ask whether Kubernetes accepted it

and whether the application consumed it. A values key can exist in a file and still do nothing if no template uses it.

7.4 Render, inspect, install, observe

Use different checks for different boundaries:

```
helm version
helm lint chart/status-lab
helm template workbook chart/status-lab -n workbook > rendered.yaml
# Review rendered.yaml before changing a cluster.
helm upgrade --install workbook chart/status-lab \
  -n workbook --create-namespace --wait=watcher --timeout=2m
helm get values workbook -n workbook --all
helm get manifest workbook -n workbook
kubectl -n workbook rollout status deployment/workbook-status-lab
```

These commands target Helm 4.3.0. Lint catches chart problems; template shows output; API validation checks the cluster's accepted shape and policy; rollout observation checks controller progress; an MCP client checks an application interaction. None subsumes all the others. Helm dry-run output may include sensitive rendered resources, so do not publish diagnostic output without reviewing it. [H05](#) [H06](#)

The chart hashes the rendered ConfigMap into a Pod-template annotation. When relevant configuration changes, the Pod template changes and Kubernetes performs a rollout. Without such a mechanism, a ConfigMap-backed environment variable does not update inside an already-running process. This checksum is a chart convention, not magic Kubernetes behavior. It does not automatically detect an independently changed external Secret. Define the separate Secret rotation/restart procedure with the environment owner. [K07](#) [K08](#)

Useful addition — code-review evidence. This belongs beside templating because it prevents deployment intent from being hidden in substitution logic. Review the source diff and the rendered diff. Ask a reviewer to locate the changed port, image or auth setting in the render and to state the expected effect. A passing formatter is not proof that the right field changed.

7.5 Release upgrades and the limits of rollback

A release history provides previous revisions and their recorded configuration. After an upgrade failure, inspect `helm status`, `helm history`, the rendered/installed manifests and Kubernetes events before deciding whether to fix forward or roll back. For a known-bad image in this read-only lab, returning to a known-good revision is a reasonable small corrective action. For a stateful application that changed an external database, reverting the Deployment may not restore compatibility. [H06](#) [H07](#)

```
helm history workbook -n workbook
helm rollback workbook GOOD_REVISION -n workbook \
  --wait=watcher --timeout=2m
```

`GOOD_REVISION` is a value you obtain from history, not a literal command argument to leave unchanged. A rollback creates another release revision based on earlier configuration; it does not reverse every action performed by an application, restore deleted external data, un-send messages or guarantee restoration of separately managed secrets. Helm is not a transaction manager for all dependencies.

Helm 4 exposes `--rollback-on-failure` for upgrade behavior; do not mechanically copy older `--atomic` examples without checking your CLI. Automatic rollback still depends on a meaningful failure

signal, a viable previous configuration and the resources needed to recover. A chart installation marked successful is not equivalent to an end-to-end service-level guarantee. [H06 H07](#)

Claims of “zero downtime” require conditions: old instances remain available, replacements can schedule and become genuinely ready, traffic stops reaching terminating instances in time, in-flight work can finish within the shutdown budget, clients handle bounded failures, and versions remain compatible during overlap. In this lab, `maxUnavailable: 0` and a surge of one support a rolling update, but cannot create extra cluster capacity or correct a misleading readiness endpoint. Say “designed to reduce interruption under these assumptions,” then test those assumptions. [K02 K09 K14](#)

7.6 Configuration, secrets and environment differences

Keep non-secret settings such as listener port, public resource identifier and issuer URL in reviewed values/ConfigMaps. A URL is not secret merely because it relates to authentication; a client secret, bearer token or private key is. The sample service verifies public-key-signed tokens and does not need an OAuth client secret just to act as a resource server. Do not invent a secret requirement. [O07 O13](#)

The chart supports an existing Secret reference for future environment-approved use, rather than embedding real credentials in `values.yaml`. Secret handling still needs access controls, rotation and care with logs or release storage. Base64 text in a rendered Secret is not encryption. Avoid putting secrets in command-line `--set` arguments because shell history and logs can capture them. [K08 H06](#)

The default chart deliberately enables the private, unauthenticated mock lab and creates no public exposure. `chart/values-auth.example.yaml` is a review-only authenticated-environment example using reserved `.invalid` domains. It is not a working remote identity-provider configuration. Moving to an approved environment requires actual issuer/audience/JWKS values, trusted HTTPS, appropriate host/origin policy and a client registration compatible with that environment. Never expose the default lab chart publicly and plan to “add auth later.”

7.7 Applying the knowledge to Kyma without guessing its installation

Generic Kubernetes tells you about Pods, Services, labels and readiness. Kyma’s API Gateway module adds an **APIRule custom resource** and associated gateway/mesh behavior. A CRD defines this extra API type; the module’s controller interprets its desired state. The current reference uses `gateway.kyma-project.io/v2`. Inspect the installed API and module version in an approved cluster before adopting a sample. A denied CRD-read permission is a reason to ask the platform owner for the version—not to demand administrator access. [Y01 Y02](#)

```
client --DNS/TLS--> approved HTTPS gateway
                        |
                APIRule path/method/security rules
                        |
                Service port 80 -> targetPort http
                        |
                Pod port 3000
                        |
                token/policy checks -> MCP handler
```

These arrows are logical routing/security boundaries; a mesh implementation may route to endpoints without a literal packet hop through a Service IP. The gateway port is not the Service port, and the Service port is not necessarily the Pod listener port. The APIRule example targets **Service port 80**, not container port 3000. [K05 Y01](#)

The current APIRule documentation and migration notes require attention to sidecar injection, rule ordering, JWT/extAuth/noAuth choices and in-cluster restrictions. These are not generic properties of every Kubernetes Service. A healthy Service in local kind does not establish that the Kyma gateway, certificate, allowed path, token audience, internal traffic policy or CORS configuration is correct. [Y01](#) [Y02](#) [Y03](#)

The review-only example separates unauthenticated public **metadata** from the protected /mcp operation. It does not publish health probes. Before deployment, verify the actual gateway reference, approved DNS/certificate, mesh enrollment, token preservation or trusted identity propagation, application resource URL, and whether 401 challenges and discovery routes survive the proxy. An application that independently verifies a bearer token must receive that token through an explicitly reviewed boundary. Do not assume a gateway preserves every header or that a gateway's JWT acceptance authorizes every tool.

For a browser-based client, add the exact reviewed cross-origin policy; an Origin allowlist alone is not a complete CORS implementation. The provided native CLI client avoids that additional integration requirement. For streaming or another MCP protocol version, review methods and timeout behavior rather than reusing the lab's POST-only JSON-response exposure unchanged. This is a version-aware application exercise, not a universal Kyma production manifest.

7.8 Question block — attempt before reading the answers

HELM-01 — What Helm adds to ordinary YAML

Provenance: A · published guide, paraphrased from I07 question on Helm versus kubectl apply. **Sources:** [I07](#) [H09](#) [K02](#). **Priority:** Core. **Type:** comparison.

Why use Helm after you already have working Kubernetes manifests?

Testing: Packaging, parameterization and operational boundaries.

HELM-02 — Chart, release and repository

Provenance: A · published guide, paraphrased from I07 terminology question. **Sources:** [I07](#) [H09](#). **Priority:** Core. **Type:** explanation.

Distinguish a chart, a release and a chart repository.

Testing: Package versus installation versus distribution.

HELM-03 — Chart structure and two versions

Provenance: A · published guide, adapted from I07 chart structure and version questions. **Sources:** [I07](#) [H09](#) [H04](#). **Priority:** Core. **Type:** explanation.

Explain Chart.yaml, values.yaml, templates and helpers. Why distinguish chart version from appVersion?

Testing: The package contract and version semantics.

HELM-04 — Override precedence without guesswork

Provenance: A · published guide, adapted from I07 values question. **Sources:** [I07](#) [H01](#) [H06](#). **Priority:** Core. **Type:** prediction.

Two values files set replicaCount differently. How would you predict and verify the result?

Testing: Input precedence and reproducible commands.

HELM-05 — A value exists but changes nothing

Provenance: C · original ineffective-values scenario. **Sources:** [H01](#) [H05](#). **Priority:** Core. **Type:** troubleshooting.

You change `service.targetPort` in a values file, but the rendered Service remains unchanged. Diagnose before touching the cluster.

Testing: The difference between a declared input and a consumed template value.

HELM-06 — Quoting, indentation and context

Provenance: B · documentation-based practice. **Sources:** [H02](#) [H03](#) [H04](#). **Priority:** Core. **Type:** configuration review.

Why quote an environment value but not necessarily `containerPort`? What can go wrong inside range or with?

Testing: YAML field types and template execution context.

HELM-07 — What each deployment check proves

Provenance: B · documentation-based practice. **Sources:** [H05](#) [H06](#) [K02](#). **Priority:** Core. **Type:** comparison.

Compare helm lint, helm template, a successful upgrade and an end-to-end client call.

Testing: Layered verification rather than one green status.

HELM-08 — A rollback is not time travel

Provenance: B · documentation-based practice. **Sources:** [H07](#) [H06](#). **Priority:** Core. **Type:** explanation.

What does a Helm rollback restore, and what does it not reverse?

Testing: Release history versus external side effects.

HELM-09 — Config changed, process did not

Provenance: C · original configuration-rollout scenario. **Sources:** [K07](#) [H02](#) [K02](#). **Priority:** Core. **Type:** troubleshooting.

A ConfigMap changes but the running process still uses the old environment. Explain and repair safely.

Testing: Deployment-template changes versus live process environment.

HELM-10 — No healthy replacement during an upgrade

Provenance: C · original rollout design scenario. **Sources:** [K02](#) [K09](#) [K14](#) [H06](#). **Priority:** Core. **Type:** design.

Your chart requests two replicas, `maxUnavailable 0` and `maxSurge 1`. Can you promise zero downtime?

Testing: Availability assumptions and evidence.

KYMA-01 — Three ports, one request

Provenance: B · documentation-based practice. **Sources:** [Y01](#) [K05](#) [T12](#). **Priority:** Core. **Type:** explanation.

Trace an HTTPS request through an APIRule to a Service on port 80 and a Pod on port 3000.

Testing: Generic Kubernetes versus exposure configuration.

KYMA-02 — Version and permission discovery

Provenance: B · documentation-based practice. **Sources:** [Y01](#) [Y02](#) [Y03](#). **Priority:** Useful. **Type:** configuration review.

What would you confirm before adapting a Kyma APIRule example to an approved work environment?

Testing: Version awareness without employer assumptions.

KYMA-03 — Health works, authorization discovery fails

Provenance: C · original proxy-integration scenario. **Sources:** [Y01](#) [M05](#) [O02](#) [O03](#). **Priority:** Core. **Type:** troubleshooting.

The application is healthy through port-forward, but a remote client cannot discover authorization metadata. How do you investigate?

Testing: Public routing, metadata and authentication challenge boundaries.

KYMA-04 — Gateway acceptance versus tool permission

Provenance: C · original trust-boundary design scenario. **Sources:** [Y01](#) [M06](#) [O07](#). **Priority:** Core. **Type:** design.

A gateway accepts a JWT. Can the application trust any X-User header or allow every tool?

Testing: Identity propagation and defense against boundary confusion.

7.9 Answer key

HELM-01 — Model answer

Interview-ready answer. I would use Helm when the same related resources need consistent configuration across installations and a managed release history. The chart renders Kubernetes manifests from defaults and overrides, then Helm manages a named release. Kubernetes still validates and reconciles those resources. For one tiny fixed deployment, plain YAML can be enough; adding templates introduces another layer to review. I would choose Helm when that layer removes real duplication or operational friction, not because Kubernetes requires it.

Deeper explanation. A template abstraction is valuable only when its inputs and output are understandable. Keeping the rendered YAML visible preserves the debugging skills learned before Helm. A successful chart render cannot prove scheduling, image availability or application correctness. [I07 H09 K02](#)

Example. The status-lab chart parameterizes image, replica count and configuration while preserving the Service-to-Pod port relationship.

Common weak answer. “Helm runs the containers” assigns the runtime/controller responsibility to the packaging tool.

Follow-up with answer. Would you template every line? No. Expose meaningful environment choices and keep stable structural rules explicit. Too many independent switches can create unsupported combinations.

HELM-02 — Model answer

Interview-ready answer. A chart is a package describing related Kubernetes resources and their configurable templates. A release is a named installation of a chart into a cluster context and namespace, with its own values and history. A repository is a distribution location for charts; it is not the running application. I can install the same chart more than once using different release names, provided resource naming and external dependencies support it. Local development can install a chart directly from a directory.

Deeper explanation. Naming and scope still matter. Two installations do not automatically get separate external databases, DNS names or credentials. Those dependencies must be provisioned and referenced deliberately. [I07 H09](#)

Example. workbook is the lab release; chart/status-lab is the local chart directory; no repository publication is required.

Common weak answer. “A chart is one running Pod” confuses a resource template package with a runtime instance.

Follow-up with answer. Does uninstalling a release prove all associated data is deleted? No. Retained resources, persistent storage policies and external systems require separate inspection and cleanup decisions.

HELM-03 — Model answer

Interview-ready answer. Chart.yaml describes the chart package, including its version. values.yaml contains default inputs, and templates render the Kubernetes resources. Helpers hold reusable

named fragments such as labels and names. `appVersion` describes the application version, while the chart version tracks packaging changes; they can change independently. I would also inspect how the template chooses the actual image, because `appVersion` does not automatically set it unless the chart authors wired it that way.

Deeper explanation. A probe or resource-setting fix can justify a chart release without a new application binary. Conversely, a new image can be selected through values while the template structure remains the same, subject to the team's release policy. [I07](#) [H09](#) [H04](#)

Example. Changing `image.tag` from 0.1.0 to a verified build is effective only because the Deployment template reads `.Values.image.tag`.

Common weak answer. "Bumping `appVersion` always upgrades the container" overlooks the template's actual image expression.

Follow-up with answer. Why use a values schema? To reject unsupported types or missing required inputs earlier. It does not replace Kubernetes admission or application startup validation.

HELM-04 — Model answer

Interview-ready answer. I would first identify the complete command and the order of values files. Later supplied overrides take precedence for conflicting values, and command-line settings can override files. During an upgrade, `reuse/reset` flags also affect the starting values. I would render with exactly those inputs, inspect the resulting Deployment replicas and compare the stored release values when debugging an existing installation. I would not infer the result from the contents of one file in isolation.

Deeper explanation. A rendered result is stronger evidence than a recollection of which file was edited. Lists, nested maps and null removal also deserve explicit review rather than assuming every configuration structure merges intuitively. [I07](#) [H01](#) [H06](#)

Example. Render with `-f first.yaml -f second.yaml` and inspect `spec.replicas`; then reverse the order in a scratch render and explain the difference.

Common weak answer. "The file in Git always wins" ignores command-line overrides and prior release-value behavior.

Follow-up with answer. Should secrets be supplied with `--set`? Avoid real secrets in command lines and logs. Use the approved secret-management path and keep rendered output access-controlled.

HELM-05 — Model answer

Interview-ready answer. I would ask for the exact render or upgrade command, chart path and values file, then reproduce a local render. I would inspect whether the template actually reads `service.targetPort` or uses a named constant or a different key. Next I would check override order and value type. If the render is unchanged, Kubernetes is not the first place to debug. I would make the smallest correction to the key, command or template, compare the rendered diff and only then apply it.

Deeper explanation. This scenario deliberately does not assert a root cause. A stale chart path, unpassed file, later override and unused key can all create the same apparent symptom. Evidence should distinguish them. [H01](#) [H05](#)

Example. The reference Service targets named port http; its port number is governed by containerPort in the Pod template, not by an unused service.targetPort value.

Common weak answer. “Restart the Pods until Helm picks up the file” cannot repair a template that never consumes that value.

Follow-up with answer. What if the render changes but the live Service does not? Inspect the target context/namespace/release, upgrade outcome, resource ownership and admission errors before modifying application code.

HELM-06 — Model answer

Interview-ready answer. The output must match the Kubernetes field’s type: containerPort is numeric, while an environment variable value is a string. Template expressions should preserve that distinction. Inside range or with, the dot context changes, so a reference that worked at the top level may point somewhere else. I would inspect the rendered YAML, use the root context deliberately when needed and apply indentation functions to helper output rather than assume visually aligned template source means valid output.

Deeper explanation. There are two languages involved: template evaluation and YAML parsing. A valid expression can still generate invalid YAML, and valid YAML can still have the wrong API schema. [H02](#) [H03](#) [H04](#)

Example. value: {{ .Values.containerPort | quote }} is appropriate for PORT; containerPort: {{ .Values.containerPort }} preserves its integer field.

Common weak answer. “Quote everything” can turn required numeric or boolean fields into strings.

Follow-up with answer. Does helm lint prove all branches work? Not unless those branches are exercised by supplied values. Render representative supported configurations, especially optional sections.

HELM-07 — Model answer

Interview-ready answer. Lint checks chart conventions and detectable problems. Template lets me inspect generated manifests for chosen inputs. An upgrade submits changes and, with the selected waiting behavior, observes particular resource readiness conditions. An end-to-end client call tests a real application interaction across its chosen path. I use all of them because each has a narrower guarantee. A Ready Deployment can still have an incorrect public URL, an incompatible MCP client or a missing permission.

Deeper explanation. Even a successful client call is a sample, not proof that every input, replica, failure mode or permission boundary works. Negative tests complement the happy path. [H05](#) [H06](#) [K02](#)

Example. After a Helm upgrade, run the SDK client for a valid call and a denied-scope case in the authenticated setup, not only curl /healthz.

Common weak answer. “helm template succeeded, so production is safe” stops before API policy, scheduling and application execution.

Follow-up with answer. Can dry-run leak secrets? Yes, rendered manifests may include sensitive values. Use the CLI’s relevant hiding options and review access to all logs/artifacts.

HELM-08 — Model answer

Interview-ready answer. A rollback uses a selected prior release revision to update the managed configuration, producing a new revision. It is useful when a known-good application configuration can still run against current dependencies. It does not automatically reverse database migrations, restore deleted external data, undo sent messages or recover separately managed secrets. I would inspect history, choose a known-good revision, verify compatibility and observe the rollout and client behavior afterward.

Deeper explanation. Fix-forward can be safer when the external state is no longer compatible with the old binary. The decision depends on actual side effects and recovery design, not only the existence of a rollback command. [H07](#) [H06](#)

Example. For this immutable mock-data service, returning to a known-good image/configuration is comparatively simple; the exercise has no external write to undo.

Common weak answer. “Rollback restores the entire system exactly” invents transactional guarantees across unrelated components.

Follow-up with answer. Does automatic rollback eliminate incident analysis? No. You still need to know whether recovery completed, whether users saw failures and what prevented the new version from becoming healthy.

HELM-09 — Model answer

Interview-ready answer. Environment variables are supplied when the container starts; updating a ConfigMap does not rewrite the environment of an already-running process. I would confirm the changed ConfigMap and the current Pod creation time, then use the approved rollout mechanism. The reference chart places a configuration checksum in the Pod template, so its change triggers replacement Pods. I would verify the new effective setting without dumping secrets. If there is no automatic mechanism, an explicit controlled rollout is needed.

Deeper explanation. A mounted file can have different update behavior, but the application still needs to reread it. Environment injection, projected files and application-level configuration refresh are different mechanisms. [K07](#) [H02](#) [K02](#)

Example. Compare the Deployment checksum annotation and Pod creation time before and after a reviewed config change.

Common weak answer. “Kubernetes refreshes every variable instantly” confuses an API object update with mutation of a process environment.

Follow-up with answer. Will the checksum detect an independently rotated existing Secret? Not in this chart. Define a separate approved trigger/reload procedure and verify it; do not assume the ConfigMap checksum covers all dependencies.

HELM-10 — Model answer

Interview-ready answer. Those settings support maintaining old availability while a replacement starts, but they are not a guarantee. The cluster needs capacity for the surge, the new image must start, readiness must reflect useful work, and shutdown must drain requests within the termination budget. Shared dependencies and incompatible changes can still cause failures. I would describe the intended behavior, test a rollout under representative traffic and report observed failures rather than promise zero downtime from YAML alone.

Deeper explanation. A stalled rollout can be preferable to deleting healthy old replicas, but it still needs diagnosis and a recovery decision. An over-optimistic readiness endpoint can make controller progress look healthy while users fail. [K02](#) [K09](#) [K14](#) [H06](#)

Example. An unavailable image with spare capacity yields a pull failure, while excessive requests may leave the replacement Pending; inspect the actual state instead of treating both as downtime tuning.

Common weak answer. “Two replicas means no downtime” ignores routing, simultaneous failures and common dependencies.

Follow-up with answer. What happens with one local replica? A surge can still overlap old and new instances if capacity allows, but the single-replica steady state has no redundancy for an independent instance failure.

KYMA-01 — Model answer

Interview-ready answer. The client resolves the approved public host and establishes TLS with the configured gateway. The APIRule matches the path, method and access strategy, then targets the named Kubernetes Service at its Service port, 80 in this example. The Service’s targetPort resolves to the Pod listener, 3000 through the name http. The application then handles the MCP interaction and its own policy. These are logical boundaries; I would inspect the actual mesh implementation rather than insist that every packet traverses a Service IP.

Deeper explanation. The public URL is also security configuration when it identifies the OAuth resource. A private Pod address should not accidentally become the advertised public audience. [Y01](#) [K05](#) [T12](#)

Example. The review example’s spec.service.port is 80, matching the Service, while containerPort remains 3000.

Common weak answer. “Set every port to 443 because HTTPS is used” confuses TLS termination with internal listener configuration.

Follow-up with answer. Does a successful port-forward prove the APIRule works? No. It bypasses the public DNS/TLS/gateway path and cannot validate that exposure policy.

KYMA-02 — Model answer

Interview-ready answer. I would confirm the installed API Gateway module and APIRule API version, the permitted namespace, the approved gateway and host, mesh enrollment requirements, and the intended authentication strategy. Then I would compare the example with the installed schema and platform policy. I would not assume the newest documentation matches the cluster or migrate an existing API merely to match a tutorial. If my permissions do not expose a required fact, I would ask the platform owner for that specific information.

Deeper explanation. A custom resource is interpreted by its controller and installed schema. Generic Kubernetes familiarity helps, but does not define Kyma-specific sidecar or internal-traffic policy behavior. [Y01](#) [Y02](#) [Y03](#)

Example. Use kubectl api-resources and the permitted schema/status inspection commands; treat a forbidden response as an access boundary, not evidence the feature is absent.

Common weak answer. “I need cluster-admin to deploy anything” replaces a narrow information need with excessive privilege.

Follow-up with answer. Should the local kind lab install all Kyma modules? Not for this block. First demonstrate generic deployment behavior, then review or test the managed exposure path in an explicitly approved environment.

KYMA-03 — Model answer

Interview-ready answer. I would capture the client’s exact URL, status and redirect behavior without recording credentials, then test public DNS/TLS and the metadata path separately. I would inspect whether the gateway exposes the documented metadata route and preserves the 401 challenge needed by that client. Next I would validate the metadata’s resource identifier and authorization-server issuer against the configured public endpoints. A healthy Pod through port-forward does not prove these public routes or identifiers are correct.

Deeper explanation. A gateway can reject a request before the application can return its useful challenge. A redirect, wrong host or path rewrite can also break discovery while the underlying service stays healthy. [Y01](#) [M05](#) [O02](#) [O03](#)

Example. Compare the externally returned WWW-Authenticate resource_metadata URL with the configured HTTPS public metadata endpoint.

Common weak answer. “Disable authentication until discovery works” may expose the operation while leaving the actual routing error unresolved.

Follow-up with answer. Can metadata be public? The non-secret discovery document normally must be reachable as required by the chosen flow; exposing that document is not the same as allowing unauthenticated tool invocation.

KYMA-04 — Model answer

Interview-ready answer. No. I need an explicit trust contract for how identity reaches the application and how direct access is controlled. If the application verifies bearer tokens independently, the reviewed gateway path must preserve the token and the app must validate it for its own resource. If identity is propagated through headers, those headers must be established only by a trusted component and protected from spoofing. Even valid identity does not grant every tool; application policy still checks the requested action.

Deeper explanation. A deployment can have two valid designs, but an accidental mixture is unsafe: for example, expecting the gateway to verify identity while allowing callers to supply the same trusted header through another route. [Y01](#) [M06](#) [O07](#)

Example. The workbook app independently validates JWTs and then checks status:read at invocation. The Kyma sample is review-only until token propagation and network restrictions are confirmed.

Common weak answer. “The network is internal, so headers are trustworthy” omits who can reach the service and manufacture those headers.

Follow-up with answer. Must validation always happen twice? Not universally. The architecture determines the trusted enforcement boundary, but that boundary must be explicit, tested and consistent with bypass paths and least privilege.

7.10 Observable readiness checkpoint

After this chapter, demonstrate four things. **Explain:** walk from values through rendered YAML to controller behavior, naming each boundary. **Use:** render two configurations and identify the actual changed fields. **Diagnose:** find why an override has no effect, then separately investigate an accepted but unhealthy rollout. **Justify:** explain when rollback is safe for this mock service and why it might not reverse a database migration elsewhere.

For Kyma, draw the route without confusing gateway, Service and Pod ports; identify what must be confirmed by the platform owner; and show that authentication discovery and per-tool authorization are separate checks. You are ready for the integrated exercise when you can do this with documentation open. Perfect template syntax from memory is not the goal.

8. Parallel track: MCP authorization and client integration

Stage / track: 3, parallel. **Course / reference:** roadmap [7], official MCP authorization walkthrough and the version-specific authorization specification; [6], the matching SDK; OAuth/OIDC standards and Keycloak documentation for the local lab. **Attempt after:** the Stage 2 roles/token overview and the Stage 3 authorization-flow study. **Prerequisites:** trace an HTTP request and an MCP invocation; distinguish a compile-time type from validation; distinguish a Pod's service account from an application caller. **Outcomes:** explain discovery and registration, trace a code/PKCE flow, review token verification and tool policy, diagnose denied requests and inventory local versus remote artifacts. [M05](#) [M11](#) [M12](#)

8.1 Start with the protected operation and the trust boundaries

The security requirement is not “add JWT.” It is: only an acceptable caller may invoke an allowed operation on the intended service, using credentials intended for that service, without leaking them across other boundaries. A token format is one mechanism within that requirement.

Our example has two deliberately different permissions: `mcp:connect` admits a principal to the MCP endpoint, while `status:read` permits the read-only tool. This split is a teaching policy, not a mandated MCP scope vocabulary. Your employer may use other scopes, user groups, application permissions or an external policy system. Determine the actual claim contract rather than renaming it to fit this exercise. [M05](#) [O04](#)



The authorization server issues tokens according to its policy. The resource server validates an access token through an appropriate mechanism. The application decides whether the requested operation is allowed. The human, client software and resource server are not interchangeable identities. A model's tool selection expresses a requested action; it is not a grant of authority. Tool annotations and descriptions similarly do not override a server-side permission check. [M03](#) [M06](#)

OAuth concerns delegated access. OIDC adds an authentication layer and ID-token semantics. An ID token intended for the client is not automatically an API access token for the resource server. JWT is a representation; an OAuth access token may instead be opaque, with provider-supported introspection or another validation mechanism. The lab selects JWT verification because its chosen provider can issue that profile, not because OAuth requires JWT. [O06](#) [O07](#) [O11](#) [O12](#)

8.2 Two metadata documents answer different questions

Protected-resource metadata describes the service the client is trying to use, including its resource identifier and relevant authorization servers. **Authorization-server metadata** describes the issuer and its protocol endpoints/capabilities, such as authorization and token endpoints and

supported methods. The former does not replace the latter. Both are configuration inputs crossing a trust boundary, not arbitrary URLs that should be fetched with unrestricted privileges. [O02 O03](#)

For the selected v1-family flow, a client can learn the protected-resource metadata location through the WWW-Authenticate challenge or the specified well-known discovery process, then discover the advertised authorization server. It must retain the association among resource, issuer and client registration. The resource identifier should be the canonical intended resource, not whatever an untrusted incoming Host header happens to contain. [M05](#)

A conceptual example:

```
GET or attempted protected interaction at approved resource
-> 401 with resource_metadata location
-> fetch resource metadata; check resource/issuer relationship
-> fetch authorization-server metadata; verify issuer
-> use compatible registered client and supported flow
-> request token for intended resource
-> repeat protected MCP interaction with access token
```

The actual MCP interaction and supported HTTP methods are transport/version-dependent. The reference server supplies a root metadata endpoint and advertises it in its challenge. The native teaching OAuth client uses a **preconfigured trusted issuer**, not a complete generic MCP discovery implementation. Exercise E6 tests the metadata separately and explicitly identifies that limitation.

When diagnosing discovery, record status codes, safe response headers and endpoint identifiers. Avoid recording bearer values or authorization codes. A response from the wrong issuer is not made trustworthy by being syntactically valid JSON. Metadata-based outbound requests need protections against server-side request forgery and unsafe redirects; the local lab permits only its explicit loopback environment. [M06 O03](#)

8.3 Registration must match both ends

A **public client**, such as a distributed desktop/native application, cannot reliably keep a bundled client secret confidential. A **confidential client** operates in an environment where it can protect its client credentials according to the deployment's security model. Public does not mean unauthenticated users are always allowed; it describes the client's ability to protect a credential. [O01 O09](#)

Pre-registration gives an operator an explicit client ID, approved redirects and permitted flows. It works well for a controlled local exercise or an organization with managed clients, but requires an onboarding process. Dynamic Client Registration provides a protocol for registering clients where supported; its existence does not require every provider to permit arbitrary registration. Client ID Metadata Documents are another version-dependent mechanism. The July 2026 MCP changelog deprecates DCR in favor of metadata documents while retaining compatibility paths. Do not retrofit that change into a v1 work deployment without a compatibility and policy decision. [O10 M05 M09 M11](#)

The correct question is: "Which registration mechanisms does this actual client implement, and which does this actual authorization server allow?" Follow it with the redirect requirements, issuer binding and public/confidential client type. A tutorial that launches a permissive development identity provider does not establish the answers for SAP or another employer.

The lab pre-registers workbook-native, a public native client with an exact loopback callback, authorization-code flow enabled, direct password grants disabled and PKCE S256 required. It uses

no client secret. A machine-to-machine application may instead use client credentials or another supported workload flow, but the allowed mechanism and resulting application permissions must be confirmed with the provider and MCP/client integration. Do not turn a human login token into a permanent service credential. [O01](#) [O09](#) [O12](#)

8.4 Authorization code with PKCE: understand each step

The client generates a high-entropy verifier and sends its derived S256 challenge with the authorization request. The user authenticates at the authorization server, which applies its consent and policy requirements and redirects back with a short-lived code. The client exchanges that code with the original verifier at the token endpoint. Intercepting a code alone should not supply the verifier needed for a successful exchange. Use standard cryptographic/library operations for the challenge; do not design a new cryptographic scheme. [O05](#)

The client also binds the response to the login attempt, using state and relevant issuer checks. Exact redirect handling reduces the chance of delivering a code to an unintended destination. A public client does not become confidential because it uses PKCE, and PKCE does not encrypt the access token or replace TLS. A confidential code-flow client may still need PKCE as well as its client authentication, according to current security guidance and provider requirements. [O01](#) [O09](#)

In the lab, the native callback binds to 127.0.0.1:8787; the identity provider is on 127.0.0.1:8080; the application is on 127.0.0.1:3000. These three listeners belong to the local exercise. The authorization URL includes the intended resource, and the imported provider configuration includes an audience mapper for that single resource. This fixed mapping is a teaching shortcut, not a design for accepting arbitrary resource requests in production. [O08](#) [O09](#)

Useful addition — timeout discipline. It belongs here because an abandoned login or failed discovery should not leave a local callback server open indefinitely. The teaching client has a bounded callback wait and bounded metadata/token fetches. It refuses redirects in credential-bearing fetches and keeps the access token in memory. It does not implement refresh-token persistence, general client registration or production-grade recovery. Those omissions are declared rather than hidden behind the word OAuth.

8.5 Validate a token; then authorize an operation

For the selected JWT profile, verification needs a trusted issuer/key source, permitted algorithm, valid signature, intended audience, acceptable time claims and the required claim semantics. The exact required claims depend on the provider/profile; the lab explicitly requires expiration, issued-at and subject in addition to checking issuer and audience. Decoding base64url segments proves none of those properties. [O07](#) [O13](#)

The reference uses jose with a configured JWKS URL and jwtVerify. It does not trust a token-supplied arbitrary key URL. The public key can verify signatures without giving the resource server the private signing key. A key identifier helps select a candidate key, but is not itself proof of authenticity. Key rotation requires a library/provider-compatible cache and refresh strategy. A JWKS outage should fail closed without being confused with a new permission grant. [O07](#) [O13](#)

```
const { payload } = await jwtVerify(token, trustedKeySet, {
  issuer: configuredIssuer,
  audience: configuredResource,
  algorithms: ['RS256'],
  requiredClaims: ['exp', 'iat', 'sub'],
  clockTolerance: 5,
```

```
});
```

This is the selected lab policy, not a universal JWT verifier for every provider. Some providers issue other permitted algorithms or opaque tokens. Configure the verifier from a reviewed provider contract; do not “fix” an algorithm mismatch by accepting every algorithm or skipping the signature. Clock tolerance should address small expected skew, not extend expired credentials indefinitely.

After verification, turn the trusted claims into a request-specific principal and evaluate exact permissions. `scopes.includes('status:read')` on a tokenized scope array is different from substring matching a string that might contain `status:read-all`. A group called `admin` is not universally meaningful across issuers. User and application tokens can expose different claim structures and need explicit policy mappings. Deny unrecognized combinations instead of guessing. [O04 M06](#)

The reference’s JWT-error handling is deliberately coarse: key-fetch and verification failures reject access, with safe diagnostic categories. A production service should distinguish dependency outages from invalid credentials in its internal metrics and carefully designed external error policy. It should not leak token contents or turn a temporary verification outage into an authentication bypass.

8.6 Errors tell you which boundary failed

A missing or unacceptable bearer token normally leads to a 401 challenge. An accepted token that lacks the endpoint’s required scope can lead to 403 with insufficient-scope information. An accepted MCP interaction may still return a tool-level permission error because the principal can connect but cannot invoke a particular operation. The exact representation follows the operation and protocol; do not reduce every denial to “HTTP failed.” [O04 M03](#)

For this lab: missing token → 401; invalid/expired/wrong-audience token → 401; valid token missing `mcp:connect` → 403; valid token with `mcp:connect` but without `status:read` → an MCP tool error; correct token and known name → mock status. These are expected outcomes to test, not evidence of a completed provider integration.

A useful diagnostic record separates request ID, boundary, safe error category and relevant non-secret deployment version. The resource identifier and configured issuer are usually safe to compare; the token, password, authorization code and private key are not suitable routine log fields. Never paste real employer tokens into an online decoder. Local offline inspection, when permitted, still does not replace verification.

Caution about retries. A transient network timeout may justify a small bounded retry for an idempotent read. Repeating a forbidden operation with the same token is not a recovery strategy. Refreshing an expired token is a client/provider flow, not permission escalation. Authentication errors should not be hidden by returning fabricated operational status. [O04 M06](#)

8.7 Audience, forwarding and tool trust

An access token intended for the MCP resource should not be forwarded indiscriminately to another API. That API is a different resource with its own audience, permissions and credential relationship. Blind forwarding can leak credentials or create a confused-deputy path in which one component misuses another’s authority. Obtain appropriate downstream credentials or use a provider-supported delegation/token-exchange design that has actually been approved. This block does not invent such a production system. [M06 O08](#)

Read-only tools still require security review. A read can disclose sensitive data, accept attacker-controlled URLs, consume expensive resources or return content that tries to influence a model. The mock tool avoids these hazards by looking up a small fixed table; its read-only annotation does not enforce that behavior. If a future version accepts an arbitrary URL, that is a new outbound-network trust boundary and must be reviewed before implementation. [M03](#) [M06](#)

Keep authorization attached to the request, not a global variable populated by the most recent token. Concurrent requests must not borrow each other's identity. Adding replicas does not repair a within-process principal leak. Tests should exercise two principals with different scopes and prove that interleaved requests remain independent.

8.8 Packaging: inventory execution, not labels

"Plugin," "skill," "MCP server" and "client" describe different responsibilities and sometimes product-specific packaging conventions. There is no universal rule that every plugin contains a server binary. Inspect the selected host's actual manifest and packaging rules. This workbook supplies a host-independent native client, not an unverified manifest for a particular product.

For a local stdio configuration, the launched server executable/script, runtime, dependencies, configuration and supporting files must be available on the machine that launches it. Stdout carries protocol messages; diagnostic output must not corrupt it. For a remote HTTP service, the server's compiled code, runtime dependencies and data ship in the server image, while the local client holds the endpoint configuration and the required login/client implementation. A local helper script referenced by a client package still needs to ship locally even though the MCP endpoint is remote.

[M02](#) [M04](#)

In the reference project, `domain.ts` becomes JavaScript in `dist/`; TypeScript types do not ship as runtime validation. The data file remains a separate runtime asset under `data/`; compilation does not automatically copy it. `client.ts` and `oauth-client.ts` are local tools. The server image does not automatically install them into a user's host application. Endpoint/issuer identifiers belong in deployment/client configuration; passwords and tokens do not belong in source-controlled manifests.

For a work code review, make a small inventory with four columns: artifact, executing process, installation location and configuration source. Include scripts, templates and data files, not only the entry point. Trace one tool from host selection to actual handler and back. Then trace the same request's authorization separately. That exercise is more transferable than memorizing a particular plugin folder name.

8.9 Question block — attempt before reading the answers

Reliable published interview material specific to these evolving MCP authorization details was limited. The following bank therefore uses clearly labelled documentation checks and original scenarios, without invented company attribution. This is distinct from the published Docker/Node/Helm questions elsewhere.

AUTH-06 — Two discovery documents

Provenance: B · documentation-based practice. **Sources:** [O02](#) [O03](#) [M05](#). **Priority:** Core. **Type:** comparison.

What different questions do protected-resource metadata and authorization-server metadata answer?

Testing: Discovery responsibilities and issuer/resource binding.

AUTH-07 — Registration is a compatibility decision

Provenance: B · documentation-based practice. **Sources:** M05 M09 M11 O10. **Priority:** Core. **Type:** design.

How would you choose between pre-registration, DCR and client metadata documents?

Testing: Version, client and provider constraints.

AUTH-08 — Public clients and PKCE

Provenance: B · documentation-based practice. **Sources:** O01 O05 O09. **Priority:** Core. **Type:** explanation.

Why does the native client use PKCE without a bundled client secret?

Testing: Client confidentiality and code interception protection.

AUTH-09 — A well-formed token is rejected

Provenance: C · original token-diagnosis scenario. **Sources:** O07 O13 O04. **Priority:** Core. **Type:** troubleshooting.

A JWT decodes correctly but the resource server rejects it. What evidence would you inspect?

Testing: Verification requirements and safe diagnosis.

AUTH-10 — Signing keys and rotation

Provenance: B · documentation-based practice. **Sources:** O07 O13. **Priority:** Useful. **Type:** explanation.

How can a resource server verify JWTs without the signing private key, and what changes during key rotation?

Testing: Asymmetric verification and trusted key retrieval.

AUTH-11 — Authentication is not tool authorization

Provenance: C · original insufficient-permission scenario. **Sources:** M03 M05 O04. **Priority:** Core. **Type:** troubleshooting.

A user connects and lists tools but read_status returns permission_denied. Is login broken?

Testing: Endpoint admission versus operation policy.

AUTH-12 — User and application credentials

Provenance: B · documentation-based practice. **Sources:** O01 O12 M05. **Priority:** Useful. **Type:** design.

How do you avoid confusing a user-token policy with a machine-to-machine policy?

Testing: Principal types and provider-specific claim mapping.

AUTH-13 — Do not forward the caller token everywhere

Provenance: B · documentation-based practice. **Sources:** M06 O08. **Priority:** Core. **Type:** design.

Your MCP tool later calls another API. Should it forward its incoming access token unchanged?

Testing: Audience restriction and confused-deputy prevention.

AUTH-14 — Discovery can become an outbound attack

Provenance: C · original metadata trust scenario. **Sources:** M06 O02 O03. **Priority:** Useful. **Type:** design.

A server advertises an authorization endpoint at an internal network address. What should a generic client consider?

Testing: SSRF and metadata-fetch trust boundaries.

AUTH-15 — Choose useful errors without leaking credentials

Provenance: B · documentation-based practice. **Sources:** O04 M03 M05. **Priority:** Core. **Type:** comparison.

Distinguish missing token, invalid token, insufficient endpoint scope and tool-level permission denial.

Testing: Protocol-aware failure semantics and safe observability.

INT-01 — What actually ships locally?

Provenance: C · original packaging review scenario. **Sources:** M02 M04 D06 D11. **Priority:** Core. **Type:** configuration review.

A client package references helper.js while the MCP endpoint is remote. Where must the helper and server assets be installed?

Testing: Execution placement versus product packaging labels.

INT-02 — Trace three paths, not one arrow

Provenance: C · original integration explanation scenario. **Sources:** M03 K05 Y01 O07. **Priority:** Core. **Type:** explanation.

Explain application execution, network routing and authorization as separate paths for one tool call.

Testing: Layer separation and end-to-end clarity.

INT-03 — Two principals and a shared variable

Provenance: C · original concurrency-security scenario. **Sources:** T04 T06 M06. **Priority:** Core. **Type:** configuration review.

A middleware stores currentUser globally before awaiting a tool handler. Why is that dangerous?

Testing: Request isolation under async concurrency.

INT-04 — A browser client adds a boundary

Provenance: B · documentation-based practice. **Sources:** Y01 M02 M06. **Priority:** Useful. **Type:** comparison.

Why might a native client work while a browser client fails against the same HTTPS MCP endpoint?

Testing: Browser-origin policy versus server authentication and routing.

8.10 Answer key

AUTH-06 — Model answer

Interview-ready answer. Protected-resource metadata describes the service I am trying to access and points to its authorization servers. Authorization-server metadata describes a particular issuer and the endpoints and capabilities used to obtain tokens. I need both relationships: a syntactically valid token endpoint from an unrelated issuer does not establish trust for my resource. I would verify the resource identifier, issuer consistency and supported client flow before starting authorization, and use the chosen MCP version's discovery rules.

Deeper explanation. The documents are not credentials, but consuming their URLs crosses a trust boundary. A generic client needs controlled outbound fetching, redirect handling and validation, not unrestricted requests to every advertised address. [O02](#) [O03](#) [M05](#)

Example. The lab advertises a root resource-metadata URL in WWW-Authenticate, while the teaching OAuth client deliberately uses a preconfigured issuer and tests metadata separately.

Common weak answer. "The .well-known URL is always the same document" conflates two specifications and their distinct identifiers.

Follow-up with answer. Does successful metadata fetch prove token acceptance? No. Registration, redirect policy, requested resource, token validation and application permissions still need separate verification.

AUTH-07 — Model answer

Interview-ready answer. I would first check the actual client implementation and authorization-server policy rather than pick a mechanism in isolation. Pre-registration gives explicit control over client identity and redirects but needs onboarding. DCR is available only where the provider and client support and permit it. Newer MCP versions favor client metadata documents, while compatibility requirements may preserve older paths. I would document the selected mechanism and issuer binding, not replace an existing work configuration merely because a tutorial uses another one.

Deeper explanation. The July 2026 specification change matters here; the v1 lab remains pre-registered. A public registration endpoint is not a universal requirement of OAuth or evidence that an employer should enable one. [M05](#) [M09](#) [M11](#) [O10](#)

Example. The imported Keycloak realm defines workbook-native ahead of time, with an exact loopback callback and code flow.

Common weak answer. "Every MCP server must implement its own DCR endpoint" confuses resource-server and authorization-server responsibilities and ignores version policy.

Follow-up with answer. What if a client only supports DCR but the provider only allows pre-registration? That combination needs a supported client configuration/integration change or an approved alternative, not an invented bypass or a leaked shared secret.

AUTH-08 — Model answer

Interview-ready answer. A distributed native application cannot reliably protect a secret embedded in its package, so it is treated as a public client. PKCE binds the authorization code to a

verifier generated by that particular login attempt: the authorization request carries a derived challenge, and the token exchange proves possession of the verifier. This limits the usefulness of an intercepted code. It does not encrypt access tokens, replace TLS or authorize the tool by itself. Redirect and state checks remain important.

Deeper explanation. A confidential server-side client has a different credential-protection model, but that does not make PKCE irrelevant. Follow current security guidance and the provider's supported flow rather than treating the mechanisms as mutually exclusive. [001](#) [005](#) [009](#)

Example. The lab uses `randomBytes` and the standard SHA-256/base64url transformation for S256, then binds the callback to 127.0.0.1.

Common weak answer. "Put the client secret in the plugin and hide the file" does not make a distributed client confidential.

Follow-up with answer. Does PKCE prevent all malicious native applications from registering a callback? No. Redirect handling and platform-specific protections also matter; PKCE addresses a particular code interception/substitution risk, not every client threat.

AUTH-09 — Model answer

Interview-ready answer. I would identify the rejecting component and safe error category, then compare the configured issuer and resource audience with the token's intended profile using permitted local inspection. I would check signature/algorithm support, trusted signing-key discovery, expiration and other required claims, as well as clock skew. A well-formed payload only proves it can be parsed. I would not disable validation or upload the token to an external decoder. After a specific correction, I would repeat the original request and a negative case.

Deeper explanation. Wrong audience, expired credentials, stale keys and a key-fetch outage can present similar external symptoms but need different fixes. Separate internal dependency diagnostics from client policy failures while keeping errors non-sensitive. [007](#) [013](#) [004](#)

Example. Changing the lab `RESOURCE_URL` to a different resource makes a token issued for the original audience unacceptable; restore the correct identifier rather than accepting every audience.

Common weak answer. "The user name looks right, so the token must be valid" trusts unverified claims.

Follow-up with answer. Should you accept a token after exp because login just happened? Not by overriding expiry. Check clocks and token acquisition, use only a small documented skew tolerance, and obtain an appropriate fresh token.

AUTH-10 — Model answer

Interview-ready answer. For an asymmetric signature, the issuer signs with a private key and publishes corresponding public verification keys through its trusted key set. The resource server uses a maintained library to select an appropriate key and verify the signature and claims. Rotation introduces new keys and an overlap/cache strategy; I would follow the provider and library behavior rather than hard-code a single public key forever. A token's key identifier helps selection but does not make an arbitrary key URL trustworthy.

Deeper explanation. The trusted issuer/JWKS relationship comes from reviewed configuration or validated discovery, not an attacker-controlled token field. Verification failures must not become a reason to accept unsigned payloads. [007](#) [013](#)

Example. createRemoteJWKSet is configured with the approved JWKS URL; jwtVerify restricts the lab algorithm to RS256.

Common weak answer. “Download whichever key the token tells you to trust” lets the untrusted credential choose its own authority.

Follow-up with answer. What if the JWKS server is unavailable? Follow the library’s valid cached-key behavior and fail closed where verification cannot be completed. Measure the outage separately; do not silently remove signature checks.

AUTH-11 — Model answer

Interview-ready answer. Not necessarily. I would first inspect the actual MCP result and the accepted principal’s safe permission summary. In the lab, mcp:connect permits endpoint access while status:read permits the tool. A token with only the first can connect but should fail the tool check. I would confirm the intended policy and whether the authorization server granted the required scope, rather than bypass the check. Then I would test both an appropriately permitted token and the deliberately limited token.

Deeper explanation. Listing a tool is not itself permission to execute it. Some systems filter the visible catalog, but hiding a tool is not a substitute for an invocation-time policy check. [M03](#) [M05](#) [O04](#)

Example. Run the local OAuth client with SCOPE=mcp:connect to exercise the expected tool-level denial.

Common weak answer. “Any logged-in user is authorized for every tool” collapses two different security decisions.

Follow-up with answer. Should status:read-all satisfy status:read automatically? Not by substring matching. Use exact scope tokens and an explicitly defined implication policy, if one is required.

AUTH-12 — Model answer

Interview-ready answer. I would identify which flow issued the token, what principal it represents and which claims the provider guarantees. User delegation and application identity may use different permission structures. I would define explicit mappings for each supported principal type and deny unsupported combinations, rather than assume an application token contains the same groups as a user token. Client credentials or another workload flow must be supported by the provider and integration; a saved human token is not a durable service identity.

Deeper explanation. The names of claims and permissions are deployment contracts. The workbook’s two scopes are illustrative and are not assertions about SAP identity services or the user’s team. [O01](#) [O12](#) [M05](#)

Example. A policy table can separately state user principal + approved scope -> read permitted, and supported application principal + approved app permission -> read permitted, with real claim names supplied by the owner.

Common weak answer. “All tokens are JWTs with a groups field” invents a universal format and entitlement model.

Follow-up with answer. Does a valid Kubernetes service-account token automatically call this MCP service? No. Its issuer, audience and permission purpose are different unless an explicit supported trust integration has been configured.

AUTH-13 — Model answer

Interview-ready answer. Not indiscriminately. The incoming token is intended for a particular resource and permission context. A downstream API may require a different audience or credential relationship, and forwarding can leak authority across that boundary. I would use the approved downstream credential or provider-supported delegation design, validate the target and request only necessary permissions. If the architecture genuinely permits forwarding, that must be an explicit reviewed contract, not the default behavior of a generic HTTP helper.

Deeper explanation. A tool argument containing a URL must not decide where a bearer token is sent. Redirects can also change the destination after the original request was approved. [M06 O08](#)

Example. The teaching client refuses redirects in token-bearing requests and allows only its local endpoint; the mock tool makes no downstream request.

Common weak answer. “The same user owns both systems, so the token is interchangeable” ignores audiences, issuers and API policy.

Follow-up with answer. Would deleting the audience check solve interoperability? It would remove an important security boundary. Resolve issuance and trust configuration instead of widening acceptance without justification.

AUTH-14 — Model answer

Interview-ready answer. I would treat the advertised URL as untrusted input until policy and issuer validation establish an allowed destination. A generic client should not fetch arbitrary internal addresses or follow redirects that evade its restrictions, especially when it runs with privileged network access. It needs controlled schemes, destinations, resolution and redirect behavior appropriate to its environment. The local exercise is an explicit exception for known loopback services, not a policy to accept all private addresses in production.

Deeper explanation. A simple hostname string check alone may not cover DNS rebinding, redirects or alternative address representations. A production-capable generic client needs a maintained, reviewed implementation rather than a home-grown unrestricted fetch chain. [M06 O02 O03](#)

Example. The workbook OAuth client pins its known local issuer and refuses fetch redirects; it is not presented as a general internet discovery client.

Common weak answer. “It is a .well-known document, so every URL in it is trusted” mistakes a naming convention for authentication and authorization.

Follow-up with answer. Can a legitimate enterprise IdP be internal? Yes. The policy should allow explicitly approved internal destinations while preventing arbitrary ones; security is not identical to banning all private networks.

AUTH-15 — Model answer

Interview-ready answer. A missing or invalid access token belongs at the authentication boundary and normally produces a 401 challenge. An accepted token without the endpoint’s required scope can produce a 403 insufficient-scope response. Once an MCP interaction is accepted, an operation-

specific denial can be represented as a tool error according to the selected protocol. I would log a safe category and correlation identifier, not the bearer value. The client should inspect both HTTP and MCP-level outcomes rather than call every 200 a successful operation.

Deeper explanation. Precise status handling prevents useless retries and separates acquisition problems from entitlement policy. Internal dependency outages also need diagnostics even when external errors remain deliberately coarse. [O04](#) [M03](#) [M05](#)

Example. In the lab, `mcp:connect without status:read` yields `isError true` for `read_status`; a completely missing token is rejected before the tool runs.

Common weak answer. “Return 500 for every auth problem” hides meaningful client action and suggests a server defect where the credential is simply unacceptable.

Follow-up with answer. Should a read retry after `permission_denied`? Not with unchanged authority. Resolve the intended grant through the provider/policy path rather than repeatedly invoking a forbidden operation.

INT-01 — Model answer

Interview-ready answer. I would identify which process launches `helper.js`. If it is the local host or plugin, the helper, runtime and its own dependencies must be available locally. Remote HTTP moves server execution to the deployed service, but does not magically distribute client-side files. The server’s compiled code, runtime packages and status data belong in its image. I would inventory every referenced artifact, execution location and configuration source, then test from a clean client installation and the final server image.

Deeper explanation. A bundler may inline some imports, but arbitrary runtime file paths, dynamic imports or scripts can remain external. Inspect the actual build output rather than infer packaging from source imports alone. [M02](#) [M04](#) [D06](#) [D11](#)

Example. The reference server loads `../data/status.json` relative to `dist/server.js`; the Docker runtime stage explicitly copies `data/`.

Common weak answer. “Everything is in the plugin” is not an executable packaging specification.

Follow-up with answer. Does TypeScript compilation copy that JSON automatically? Not in the supplied build. The explicit image copy is part of the runtime contract and is tested separately from type checking.

INT-02 — Model answer

Interview-ready answer. The application path is SDK dispatch, runtime argument validation, permission policy, domain lookup and result conversion. The network path is client DNS/TLS and the approved gateway route to the Service and Pod listener. The authorization path is client registration and token acquisition, resource-server verification and the tool-specific check. They meet at the request, but a failure in one does not prove failure in the others. I would draw and test them separately, then connect them with a request identifier.

Deeper explanation. This separation makes evidence actionable: a missing Service endpoint is not repaired by adding a scope, while a tool denial after a valid response is not repaired by changing DNS. [M03](#) [K05](#) [Y01](#) [O07](#)

Example. Use a health probe for limited process evidence, an SDK client for protocol evidence and positive/negative token cases for policy evidence.

Common weak answer. “The request goes to Kubernetes and then OAuth happens” is too vague to locate the actual enforcement component.

Follow-up with answer. Where do secrets appear? Only in the components that require them, through approved runtime mechanisms; not as ordinary diagram labels copied into logs, source files or chart defaults.

INT-03 — Model answer

Interview-ready answer. Multiple requests can interleave around awaits in the same process. If one global variable represents the current principal, a second request can overwrite it before the first resumes, allowing the first handler to act with the wrong authority. I would pass an immutable request-specific principal through the handler context or a carefully implemented request-local mechanism. I would test interleaved callers with different permissions. Adding replicas or making the function async does not make a shared mutable identity safe.

Deeper explanation. Single-threaded JavaScript does not imply requests execute as indivisible transactions from arrival to completion. Async interleaving is enough to create this logical security bug. [T04](#) [T06](#) [M06](#)

Example. The reference creates `makeMcp(table, principal)` for the request rather than setting a module-level `currentUser`.

Common weak answer. “Node is single-threaded, so there is no concurrency issue” ignores interleaving and shared mutable state.

Follow-up with answer. Is a read-only global status table equally dangerous? An immutable shared table with no caller-specific authority is different. The risk comes from mutating shared security context, not from every use of module-level data.

INT-04 — Model answer

Interview-ready answer. A browser adds cross-origin enforcement and possibly preflight requests that a native client does not. I would compare the actual browser request, origin, allowed methods/headers and exposed response headers, while separately checking the same token and endpoint. An Origin allowlist alone does not generate a complete CORS response. I would configure the narrow approved browser policy at the intended layer and verify it, not solve the problem by disabling authentication or allowing every origin with credentials.

Deeper explanation. Proxy CORS behavior can differ from the application’s local response; inspect the externally observed headers. The workbook deliberately uses a native CLI client so browser integration is not an untested hidden requirement. [Y01](#) [M02](#) [M06](#)

Example. The Kyma reference notes that CORS configuration is explicit; the review-only `APIRule` does not claim a ready-made browser integration.

Common weak answer. “The browser failure proves the token is wrong” ignores an additional browser-enforced boundary.

Follow-up with answer. Does CORS secure the API from all clients? No. Non-browser clients are not controlled by browser same-origin policy; server-side authentication and authorization remain necessary.

8.11 Observable readiness checkpoint

Explain: draw the two metadata documents and a code/PKCE flow, then identify who issues, presents and validates the access token. **Use:** run the local flow and record a valid call plus a denied tool call without recording secrets. **Diagnose:** separate wrong audience, expired token, missing endpoint permission and missing tool permission. **Justify:** explain why the native client has no bundled secret, why token forwarding is constrained and what must be reviewed before remote deployment.

For integration, locate every runtime asset and identify what ships locally versus in the image. You may use official documentation for exact metadata field names. You should not need documentation to explain why a decoded payload is not trusted identity or why authentication alone does not authorize every tool.

9. Stage 3 practice, checkpoint and mock interview

Stage / track: 3, both tracks. **Course points:** Helm templates/values/release operations and the MCP authorization walkthrough. **Prerequisites:** Chapters 7–8, a built `status-lab:0.1.0` image, a working local SDK client and completed private Kubernetes checks. **Outcome:** package the same service and separately demonstrate a local standards-based login/token/permission flow. Kyma exposure remains a reviewed application of the knowledge unless an approved sandbox is actually available.

9.1 Exercise E5 — Package the working YAML as a chart

Tools. Helm 4.3.0, the existing compatible `kubectl/kind/Docker` setup. Check the Helm/Kubernetes support policy for the actual local server version. **Files.** `chart/status-lab/` and `chart/values-dev.yaml`. **Independent task.** Starting from the working Stage 2 YAML, create templates for the four application resources, parameterize image/replicas/configuration, and add consistent name/selector helpers. Do not use Helm to conceal a Service selector you cannot explain. [H08](#)

Ownership transition. Keep the local cluster, but remove the ordinary-manifest installation before the Helm exercise. In this disposable kind cluster only, `kubectl delete -f k8s/status-lab.yaml` deletes the workbook namespace and its contents. Check the context first and do not use this cleanup against an employer namespace. The image already loaded into kind remains available. Alternatively, use a separate disposable namespace and explicitly adapt every name/configuration reference; the default instructions use `workbook` consistently.

From the repository root:

```
helm lint chart/status-lab
helm template workbook chart/status-lab -n workbook > rendered-default.yaml
helm template workbook chart/status-lab -n workbook \
  -f chart/values-dev.yaml > rendered-dev.yaml
diff -u rendered-default.yaml rendered-dev.yaml
helm upgrade --install workbook chart/status-lab \
  -n workbook --create-namespace --wait=watcher --timeout=2m
helm status workbook -n workbook
helm get values workbook -n workbook --all
kubectl -n workbook get pods,svc
kubectl -n workbook port-forward service/workbook-status-lab 3000:80
```

Success criteria. You can point to the replica difference in the two renders, the Service selector matches the Pod labels, the application image/port/config are correct, the local release becomes ready and the same SDK client can list/call the tool. These are expected observations, not a claim that Helm was executed during authoring. The default render has two replicas; the development override has one. Do not accidentally claim the development override preserves the same redundancy.

Hints. First copy the proven plain YAML without substitution and render it. Then parameterize one field at a time. Finally factor helpers and add the values schema. Inspect the output after each step; debugging an entire unreviewed template tree at once produces unnecessary uncertainty.

Reference solution. The supplied chart has a `Chart.yaml`, defaults, a values schema and five template files. `_helpers.tpl` creates stable names and selector labels. The Deployment consumes the image and resource values, uses the same probes/security context as Stage 2 and includes a ConfigMap checksum. The Service maps its numeric port to the named Pod port. The schema reserves `H0ST` and `P0RT` from the free-form config map because those are generated from chart

fields; this prevents duplicate keys. Optional external secrets are references, not committed credentials.

Break it — ineffective value. Put replicas: 4 into a scratch override file even though the chart expects replicaCount. Render and observe whether replicas changed. Ask whether the file was supplied, whether a later override exists and which key the template reads. The reference repair is to use replicaCount: 4, then verify the render before applying. A stricter top-level schema could also reject unknown keys; that is an optional improvement, not behavior claimed for this schema.

Break it — failed release. In the local cluster only, upgrade to the intentionally unavailable tag does-not-exist, using an explicit short wait. Read status/history and inspect the new Pod events. Do not change registry credentials until the exact image reference and source of the failure are established.

```
helm upgrade workbook chart/status-lab -n workbook \
  --set-string image.tag=does-not-exist --wait=watcher --timeout=90s
helm history workbook -n workbook
kubectl -n workbook get pods
kubectl -n workbook describe pod NEW_POD_NAME
# Replace GOOD_REVISION with the known-good revision from history:
helm rollback workbook GOOD_REVISION -n workbook \
  --wait=watcher --timeout=2m
```

Diagnostic solution. Preserve the healthy old replicas, identify the new image/pull state and select the known-good revision. Verify rollout readiness and the original client request after recovery. A failure status is not the same as an empty deployment: inspect what actually remains available. Prevent recurrence by checking image publication/identity before upgrade and including a final-image test. Automatic rollback with Helm 4's --rollback-on-failure is a useful variation after you understand the manual evidence trail. [H06 H07 K02](#)

Useful variation — configuration rollout. Change a non-secret mock-lab setting that the application actually consumes, render and identify the ConfigMap and checksum changes, then observe replacement Pods. Explain why an external Secret changed outside Helm would need a separate mechanism. **Stretch:** review chart/values-auth.example.yaml and kyma/apirule-v2.example.yaml, annotate every placeholder/assumption and refuse to deploy them until those assumptions are confirmed. No public exposure is necessary to complete this workbook.

Cleanup. Stop port-forward and run `helm uninstall workbook -n workbook` for this release. Inspect what remains; Helm does not promise to delete every external dependency. Delete the disposable kind cluster when finished. **Interview evidence:** HELM-01-10 and KYMA-01-04.

9.2 Exercise E6 — Local authorization with a real development provider

Objective. Follow authorization code with PKCE, verify a resource-bound token and demonstrate separate endpoint/tool permissions. **Tools.** Docker, a browser, Node/npm and the built Stage 2 service. The local provider image is Keycloak **26.7.4**. **Files.** auth/workbook-realm.json, auth/run-keycloak.sh, auth/server-env.sh, and service/src/auth.ts, config.ts, oauth-client.ts, client.ts. Full source is in Appendix A. [O14 O15 O16](#)

Safety/setup assumptions. This exercise runs the MCP application and OAuth client on the host, not inside Kubernetes. The provider container is published only on host loopback. That choice keeps 127.0.0.1 consistent for the browser, callback, issuer and verifier. A Pod's localhost would point to the Pod, not the host provider. Do not transplant these URLs into Kyma values. The fixed audience

mapper and development mode are local teaching choices, not a deployable production identity system.

Stop any application/container/port-forward occupying port 3000, and check that 8080 and 8787 are available. From the project root, run:

```
bash auth/run-keycloak.sh
```

The script prompts for a new disposable administrator password without echoing it, passes it via an environment variable and does not commit it to a file. Keep the terminal running. Docker administrators can still inspect container configuration; this local convenience is not a full secrets-management solution. The container starts in development mode, imports the realm and has no durable volume. Stopping its --rm container discards the local accounts. [O15](#) [O16](#)

Open <http://127.0.0.1:8080> in your browser, sign in as workbook-admin with that disposable password, and select the imported workbook realm. Create a disposable user, fill any required profile fields and set a non-temporary password through the credentials interface. The realm file deliberately contains no user password. Inspect the workbook-native client: public, code flow enabled, exact callback <http://127.0.0.1:8787/callback>, PKCE S256, direct password grants disabled. Confirm both mcp:connect and status:read are optional client scopes and appear in the granted token scope when requested. UI labels may vary; verify the resulting configuration rather than inventing a paid lesson sequence.

Independent task. Draw the flow and identify the verifier, challenge, code, access token, resource and issuer before running the client. Explain why the resource server does not need the signing private key or a client secret. Do not inspect the reference handler until you have written the expected outcomes for each negative case below.

In a second terminal, from the project root:

```
source auth/server-env.sh
cd service
npm run build
npm start
```

In a third terminal, from service/:

```
curl -i -X POST http://127.0.0.1:3000/mcp
curl --fail http://127.0.0.1:3000/.well-known/oauth-protected-resource
npm run oauth-client
```

The first request should receive a 401 with a metadata challenge; the second should return non-secret resource metadata. The OAuth client prints a local authorization URL. Open it in the browser, use the disposable account, complete the provider flow and return to the terminal. The client exchanges the code with its verifier and invokes the SDK client with the access token held in memory. It does not print the token or save a refresh token. **Expected success:** read_status is listed and the known mock service is returned. This complete provider flow was not executed in the authoring environment.

9.3 Exercise E6 negative matrix and reference diagnosis

Run one variation at a time. Record the safe error category and which boundary rejected the call; restore the normal configuration after each. These expected outcomes are grounded in the selected validation/policy code and must be verified against the actual installed provider/SDK. [O04](#) [O07](#) [M03](#)

No token. Use the first curl above. **Expected:** 401 before tool dispatch. **Diagnosis:** verify that the request reaches the application or identify an earlier gateway rejection. **Small fix:** perform the correct client flow, not disable the server check.

Malformed token. Send the literal non-secret test string, not a real credential:

```
curl -i -X POST http://127.0.0.1:3000/mcp \
-H 'Authorization: Bearer not-a-token'
```

Expected: invalid-token rejection. **Diagnosis:** establish which verifier rejected it and that no lookup was performed. **Prevention:** a negative test asserting denied requests never reach sensitive domain work.

Valid connection scope but no tool scope. In service/ run:

```
SCOPE='mcp:connect' npm run oauth-client
```

Expected: the client can connect/list, but invocation produces the deliberate tool-level `permission_denied`. **Diagnosis:** inspect the intended scope grant locally without logging the token.

Small fix for the normal user flow: request and obtain the permitted `status:read` grant; retain the restricted case as a regression test. Tool visibility alone must not bypass invocation policy.

Valid token but no endpoint scope. Because both scopes are optional in the imported client, request only `status:read`:

```
SCOPE='status:read' npm run oauth-client
```

Expected: 403 at the endpoint because `mcp:connect` is absent. If your provider added it as a default scope, inspect the effective client-scope assignment and correct the disposable realm configuration. Do not claim the test covered missing scope when the token actually contained it.

Expired token without copying it into a file. The realm's selected access-token lifetime is 120 seconds. The local client offers a bounded delay after acquisition:

```
TOKEN_DELAY_MS=135000 npm run oauth-client
```

Expected: rejection after expiration, accounting for the verifier's small configured clock tolerance. The token remains in memory. Confirm the actual provider lifetime and clocks if the result differs. Do not extend the verifier tolerance until the expired token passes; that would erase the property under test.

Wrong audience. Stop only the application, retain the provider and source the normal environment again. Temporarily start the application with a different expected resource:

```
RESOURCE_URL='http://127.0.0.1:3000/different-resource' npm start
```

Run the normal OAuth client, which still obtains the configured token for `/mcp`. **Expected:** audience rejection. The teaching client is issuer-preconfigured rather than generic discovery, so this variation isolates the verifier. **Repair:** restore the normal `RESOURCE_URL`, restart and rerun both valid and wrong-audience cases. Never replace the audience check with "accept any token from this issuer."

Wrong issuer or signature — review and optional variation. For issuer, temporarily change only the verifier's expected issuer while retaining the approved key URL, then test a token from the original realm. For signature, a maintained JWT test library and a separately generated local test key can construct a token signed by an untrusted key; the verifier must reject it. This stretch does not require implementing cryptography or exposing private keys. The complete required negative matrix above can be completed without this additional token-construction task.

9.4 Hints, full solution boundaries and cleanup

Graduated hints. First confirm all three listeners and exact loopback addresses. Next verify discovery's issuer, the client's exact callback and the public-client/PKCE settings. Then inspect the audience mapper and optional scope assignments. Finally compare verifier and handler policy separately. Login success alone does not prove audience or scope compatibility, and a failed token exchange happens before the resource-server verifier is involved.

Complete reference solution. The imported realm pre-registers the public client and its scopes, with an audience mapper for this single resource. `oauth-client.ts` fetches metadata only from the known local issuer, checks its issuer, constructs a state/PKCE-bound code flow, runs a bounded callback listener, exchanges the code and passes the token in memory to `runClient.auth.ts` verifies through the trusted key set and requires endpoint scope; `mcp.ts` enforces tool scope. The normal environment file binds the resource/issuer/key URL consistently. Appendix A includes every file needed; no private employer component is assumed.

Known limits. This is not a complete generic MCP OAuth client: it does not implement arbitrary protected-resource discovery, registration negotiation, refresh persistence, browser CORS, multi-issuer credential storage or an approved remote proxy. The resource metadata is tested separately. Provider errors and key-fetch outages need richer operational handling for production. The lab's token profile, fixed audience mapper and loopback exceptions must not be copied into an unrelated environment as universal defaults.

Cleanup. Stop the Node server/client, clear shell variables with `unset AUTH_MODE OAUTH_ISSUER OAUTH_JWKS_URL RESOURCE_URL ALLOWED_HOSTS ALLOWED_ORIGINS`, and stop `workbook-keycloak` with `docker stop workbook-keycloak`. The `--rm` provider container is removed and its local accounts disappear. Close the browser's disposable session. Keep only non-secret source/configuration and safe notes. **Interview evidence:** AUTH-06-15, INT-01-04 and KYMA-03-04.

9.5 Stage 3 checkpoint and revision map

S3-CB1. Explain why a changed values file might not change a deployment. **Answer:** it may be the wrong path/key, not supplied, overridden later, unused by the template, not submitted to the intended release, or rejected downstream. Start by rendering with the exact command. See 7.3-7.4, HELM-04-07.

S3-CB2. State what rollback does not undo. **Answer:** external writes, incompatible database changes, independent secret rotation and arbitrary side effects require separate recovery. See 7.5, HELM-08.

S3-CB3. Explain resource metadata, issuer metadata and registration separately. **Answer:** they describe the protected API, the token issuer's endpoints/capabilities, and the client's recognized configuration respectively. None replaces the other. See 8.2-8.3, AUTH-06-07.

S3-CB4. Explain why a valid signature is insufficient. **Answer:** issuer, audience, time/profile claims and operation policy still matter. A token valid for another API or lacking a required permission must not be accepted for this tool. See 8.5-8.7, AUTH-09-15.

S3-T. The chart is healthy but the external client receives 401. **Model diagnosis:** record the exact public URL/status/challenge, identify the rejecting layer, confirm public metadata and issuer/resource identifiers, then inspect safe validation categories and token propagation. Do not

randomly change Pods or bypass the gateway. If the response is instead an MCP tool denial, move to operation policy rather than diagnosing TLS.

S3-D. Plan an authenticated two-replica rollout in an approved sandbox. **Model:** keep image identity and configuration explicit, render and review, verify capacity/readiness, define drain and rollback conditions, establish client/provider registration and resource identifiers, confirm gateway token handling and test positive/negative permissions through the public route. State unknown platform settings instead of filling them with fictional SAP details.

Use the same five-dimension rubric: explanation, evidence-led diagnosis, correct use/walkthrough, justification and honest communication, each 0–4. Score 14–20 with the security distinctions intact means ready for the final integration check. A lower score identifies revision, not failure of your career plan. Revise 7.3 for ineffective overrides, 7.5 for rollback overclaims, 7.7 for route/port confusion, 8.2–8.4 for OAuth sequence confusion and 8.5–8.7 for token/permission confusion. Stretch tasks are not required.

9.6 Stage 3 mock interview — 40 minutes

Deployment review, 8 minutes. “Show how a configuration change reaches a process.” **Model:** values → template → ConfigMap and checksum → changed Pod template → rollout → new process environment. **Follow-up:** “What about an external Secret?” **Answer:** this chart’s checksum does not detect independent rotation; use an approved explicit refresh/restart mechanism and test it.

Incident, 8 minutes. “The new image will not pull.” **Model:** inspect the exact image/tag, events, loaded local image or registry publication/access and preserve healthy old instances. Compare release history and choose a known-good revision if recovery requires it. **Follow-up:** “What if a database migration already ran?” **Answer:** verify backward compatibility and external recovery before rolling back; this mock app avoids that side effect but other applications may not.

Security flow, 10 minutes. “Walk through the native client’s login.” **Model:** pre-registered public client, state and verifier/challenge, user authorization at the provider, exact callback, code exchange with verifier, resource-bound access token, resource-server validation, endpoint scope, tool scope. **Follow-up:** “Where is the client secret?” **Answer:** none is bundled for this public native client; server-side confidential clients have a different model.

Denial branch, 8 minutes. “Login succeeded, but the tool is forbidden.” **Model:** determine whether endpoint or tool policy rejected it, compare granted scopes with intended policy, and request only the approved additional grant. **Follow-up:** “Could we forward the token to another API to check permissions?” **Answer:** not indiscriminately; that creates another audience and credential boundary requiring an approved mechanism.

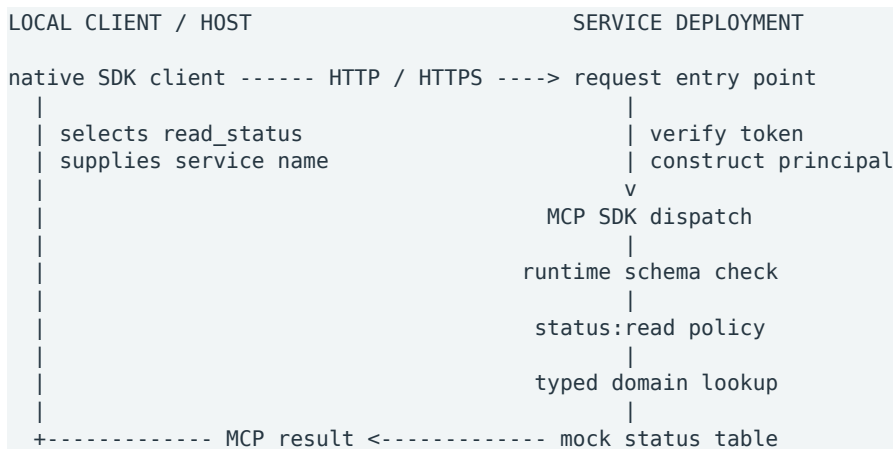
Integration reflection, 6 minutes. “What remains before Kyma exposure?” **Model:** installed API/module version, approved gateway/DNS/TLS, namespace/mesh policy, real issuer/client configuration, public resource identifier, token propagation, metadata challenge routing and full external positive/negative tests. State which tests you performed locally and which remain unverified. Evaluate the same rubric; clear boundaries are more valuable than a confident but invented production story.

10. Final integrated assessment

Scope: TypeScript → MCP → Docker → Kubernetes → Helm, with separate network and authorization explanations. This is an author-designed learning assessment, not a reported interview from a particular employer. Complete the Stage 1–3 Core material first. The reference answers describe what you could demonstrate or investigate; they do not claim that you personally operated any production system.

10.1 The architecture you should be able to explain

The project is a read-only mock service whose status domain is deliberately boring. That leaves room to learn execution, packaging, deployment and access boundaries. You can replace the mock domain later without treating the infrastructure lessons as disposable projects.



The arrows describe work and data movement, not automatic trust. The client supplies arguments; the server checks them. The caller presents a token; the server validates it. The model requests a tool; server policy determines whether it runs. The table is a packaged runtime asset, not information retrieved from the employer. The selected v1 SDK handles protocol mechanics, while application code owns the domain and policy. [M03](#) [M04](#) [O07](#)

Now draw the network route independently:

```

local Docker exercise:
  host 127.0.0.1:3000 -> published container port -> 0.0.0.0:3000

local Kubernetes exercise:
  host port-forward -> selected backend application
  separate in-cluster test -> Service DNS -> selected ready backend

approved Kyma design, not claimed deployed:
  public DNS -> TLS gateway -> APIRule -> Service port 80
  -> ready Pod endpoint / named http port -> process port 3000

```

Port-forward is intentionally listed separately from the complete Service/public-gateway route. A mesh can implement logical Service routing without a literal traversal of the Service's virtual IP. The architecture explanation should show what evidence each test supplies, not hide bypasses inside a single green arrow. [D07](#) [K05](#) [K06](#) [Y01](#)

Finally draw the authorization route without copying the network diagram:

```

known resource -> protected-resource metadata -> trusted issuer metadata
|
| compatible registration
|
| code flow + state + PKCE

```

```

      |
access token for that resource
      |
verified issuer/signature/audience/time/claim profile
      |
endpoint scope -> per-tool scope -> result

```

The native teaching client preconfigures the issuer and tests resource metadata separately; it is not a full implementation of the first generic-discovery arrow. Your explanation should preserve that limit. An approved real integration must match the actual client, provider and protocol version. M05 O02 O03 O05

10.2 Final practical checkpoint — the evidence portfolio

F-01 — Artifact identity. Starting with the TypeScript source and data file, identify the compiled entry point, runtime dependencies and final-image asset location. Demonstrate that a source-only change and a dependency change affect the build differently. **Complete answer:** compilation produces JavaScript, but does not erase runtime validation needs or automatically copy arbitrary data. Manifest-first dependency installation supports cache reuse; the final-stage copy list defines what actually exists at execution. Use DKR-06, DKR-09–10 and TS-10. Expected evidence is your reviewed Dockerfile and a final-image smoke test, not a claim that type-checking proves packaging.

F-02 — Request behavior. Use the compatible SDK client to list the tool, invoke a known name, invoke an unknown name and submit invalid input in a controlled test. **Complete answer:** distinguish discovery, accepted tool execution, domain error and schema/protocol failure. Do not return mock operational status for every case. Use MCP-08–10 and the reference adapter. Record actual observed status/result pairs.

F-03 — Deployment behavior. Show the plain Deployment and Service, then the rendered chart equivalents. Identify selectors, port mapping, image, resources, probes, service account and shutdown budget. **Complete answer:** the controller chain maintains desired Pods; the Service selects matching eligible backends; the chart renders those same contracts. Use K8S-02–11 and HELM-03–07. A rendered object is not evidence that the runtime accepted it.

F-04 — Safe rollout and recovery. Introduce the unavailable image only in the disposable local cluster, inspect events and recover to a known-good revision. **Complete answer:** preserve useful old capacity, distinguish scheduling from pull/start/readiness failures, verify recovery with a client, and acknowledge that this read-only app has no external write to undo. Use K8S-12–14 and HELM-08–10.

F-05 — Permission boundary. Demonstrate the normal local OAuth call, missing-token rejection, an endpoint denial and a tool-level denial. **Complete answer:** issuer/token verification, endpoint scope and operation policy are separate. The restricted token must remain restricted after the normal case succeeds. Use AUTH-06–15. Record categories and request IDs, not bearer values.

F-06 — Deployment versus authentication environment. Explain why the host-based IdP URLs do not work unchanged inside a Pod, and review the reserved-domain authenticated values/APIRule examples without deploying them blindly. **Complete answer:** localhost is relative to the process's network namespace; remote identifiers must be approved externally reachable resource/issuer URLs, with matching token issuance, validation and proxy behavior. Use KYMA-01–04 and INT-01–04.

For each item choose **Not attempted** / **Needed hints** / **Answered independently** / **Demonstrated practically**, then write the date, actual tool versions and evidence location. A walkthrough is legitimate learning evidence but is not the same as a tested deployment. The final authoring archive includes a validation record so you can start your own record without inheriting fictitious success claims.

10.3 Three complete incident walkthroughs

Application incident: the runtime data file is missing

Clarify. Does native execution work? Which image/tag/digest fails? Does the process exit before listening? Was the build or file layout changed? **Observe first.** Compare startup logs and exit state with the final Docker stage. Avoid assuming MCP authentication is involved when the server never started.

Inspect in order. Check the actual runtime working directory/module-relative path, then the final-stage copy instructions, build context and `.dockerignore`. Use the image itself, not only the source directory, to verify the file. A missing source file, an ignored file and a file copied only into a discarded build stage have different evidence.

Small safe repair. Restore the explicit data copy to the expected runtime location and rebuild a distinct image identity. Do not mount the entire home directory to make the error disappear. **Verify.** Start the final image, test health and a tool that reads the loaded table, then repeat the missing-file test in a disposable copy. **Prevent.** Add a final-artifact smoke test and review asset changes alongside code. These are author-designed diagnostic steps supported by the build/runtime contract. [D06](#) [D11](#)

Deployment incident: a Service has no useful backend

Clarify. Which namespace, Service, source location and exact URL? What changed? Is the error a connection failure or an application response? **Observe first.** Get the Service, its selector, Pod labels/readiness and EndpointSlices. An existing Service object alone proves little.

Distinguish causes. No matching labels points toward selection. Matching but unready Pods points toward readiness/startup. Matching ready endpoints with an incorrect target port points toward listener mapping. Healthy direct access but failed Service-path access narrows routing/DNS/policy. A permission-denied MCP result means traffic reached an application policy boundary and should not be “fixed” by changing selectors.

Small safe repair. Correct only the evidenced mismatch in reviewed configuration. **Verify.** Repeat the original route from the original source, not only a different port-forward. **Prevent.** Keep selectors in one reusable helper, render representative configurations and include a Service-path test in the lab’s evidence. [K04](#) [K05](#) [K06](#)

Permission incident: valid login, forbidden tool

Clarify. Which issuer/resource, principal type, endpoint and tool? Is the rejection HTTP 401, HTTP 403 or an MCP tool error? Who made the permission decision? **Observe first.** Record safe category/request ID and inspect the intended policy. Do not paste the token into a chat, screenshot or public decoder.

Distinguish causes. Invalid audience/issuer/time belongs to credential acceptance. Accepted token without endpoint scope belongs to endpoint admission. Accepted MCP interaction without tool scope belongs to operation policy. A provider that did not grant an optional scope differs from a server that parses an exact granted scope incorrectly.

Small safe repair. Request or assign only an approved permission through the provider/policy path, or correct a verified claim-mapping bug. Never replace the check with “allow everyone who logged in.” **Verify.** Run the allowed case and retain a genuinely denied principal case; success alone does not prove the boundary. **Prevent.** Keep explicit principal/permission mappings, request-local identity and negative tests. [O04](#) [O07](#) [M06](#)

10.4 Combined mock interview — 55 minutes

1. Architecture, 8 minutes. “Explain what you built and why the domain is small.” **Model answer:** “This is a learning service with a pure mock-status lookup, a version-matched MCP adapter and explicit validation/policy. I carried the same application through Docker, ordinary Kubernetes manifests and Helm so I could reason about each boundary rather than build unrelated demos. The mock domain avoids private data and lets me isolate infrastructure failures.” **Branch:** “Why not a database?” **Answer:** no mutable persistence requirement exists in this scope; adding one would obscure the current learning goal. **Strong evidence:** accurate boundary diagram and a justified small scope.

2. Code review, 7 minutes. “What does TypeScript prevent, and what does it not?” **Model answer:** “It catches selected inconsistencies in my checked code, but external JSON still needs runtime validation. Async work needs an awaited error boundary, and shared identity must be request-specific. I test the domain without networking and then test the adapter separately.” **Branch:** “Could async block the health check?” **Answer:** yes, synchronous CPU work in an async function still occupies the event loop. **Strong evidence:** TS-01, TS-08–09 and INT-03 reasoning, not language slogans.

3. Docker incident, 8 minutes. “The image builds but exits in deployment.” **Model answer:** “I would inspect the process exit and startup error, then compare runtime command, configuration and required assets with the final image. Build success does not prove that runtime files were copied or that configuration exists. I would repair the smallest evidenced packaging/configuration error and rerun a final-image test.” **Branch:** “It works when my source directory is mounted.” **Answer:** that may hide an incomplete image; remove the mount when validating the artifact. **Strong evidence:** DKR-08–10 and the first incident walkthrough.

4. Kubernetes/Helm, 10 minutes. “A chart upgrade has no ready replacement.” **Model answer:** “I would compare desired rollout state with the actual new Pods: Pending, image pull failure, crash or readiness failure imply different next checks. I would inspect rendered values and live events while preserving healthy old replicas. If the previous version is compatible, I would use a known-good release revision and verify the client path afterward.” **Branch:** “Did the rollback undo a database migration?” **Answer:** not automatically; external state requires a separate compatibility/recovery decision. **Strong evidence:** K8S-12–14 and HELM-08–10.

5. OAuth and policy, 10 minutes. “A token decodes and login succeeded, but the tool is rejected.” **Model answer:** “I would locate the rejection boundary, verify the intended issuer/audience/profile and then separate endpoint scope from tool scope. A readable payload is not verification, and a valid principal is not universal permission. I would correct issuance or policy mapping only after

checking actual grants, then retain a restricted-token regression test.” **Branch:** “Could the native client use a bundled secret?” **Answer:** not as a reliably confidential credential; it uses the approved public-client code/PKCE flow. **Strong evidence:** AUTH-08-15 and precise error classification.

6. Public integration, 7 minutes. “What differs when this moves behind Kyma HTTPS?” **Model answer:** “I must confirm the installed custom-resource/module version, approved gateway and certificate, Service/Pod port mapping, mesh restrictions and exact public resource/issuer configuration. I would test metadata challenges, token propagation, protocol compatibility and per-tool denials through the external route. Port-forward alone cannot verify that path.” **Branch:** “Can you deploy the supplied APIRule now?” **Answer:** only after replacing placeholders and confirming the approved environment; it is a review example, not evidence of an existing employer configuration. **Strong evidence:** KYMA-01-04.

7. Reflection, 5 minutes. “Which parts did you personally implement, and what remains unknown?” **Model answer:** use your actual record: “I wrote or modified these files, used starter code for these components, and verified these commands on these versions. I can explain the decisions and failures I investigated. This other boundary is still a reviewed design; I would verify it using this specific test in an approved environment.” **Strong evidence:** honesty, ownership and a concrete investigation approach, not invented scale.

10.5 Final scoring and the right next decision

Score each dimension 0–4: application explanation; network explanation; authentication/authorization explanation; deployment/recovery reasoning; evidence/communication. Four means independently correct with useful tradeoffs, three means correct with a minor prompt, two requires substantive hints, one mixes boundaries, zero has no workable account. Total 20.

Ready to continue learning: 14–20 with no unresolved unsafe shortcut such as treating decoded tokens as trusted or exposing the unauthenticated lab publicly. **Needs focused revision:** 9–13, or any major boundary confusion; select the small matching exercise instead of restarting every course. **Rebuild the weak foundation:** below 9; repeat the specific Stage 1/2 prerequisite that blocked the explanation. **Optional stretch mastery:** independently investigate a new failure and justify alternatives while maintaining the safety boundary. This is a study rubric, not a scientifically validated measure of employability or a hiring guarantee.

10.6 Discuss this project honestly in interviews

A useful architecture account has four sentences: what the service does, where the boundaries are, why you chose the deployment approach, and what you actually verified. For example: “This is a read-only mock-status service built to learn remote MCP deployment. The domain is separate from the protocol and authorization adapters. I carried it through an image, plain manifests and a Helm chart so I could compare each layer. My verification record distinguishes local tests from integration work not yet run.” Replace the final sentence with your own actual evidence as you progress.

For a debugging story, use **symptom** → **competing hypotheses** → **evidence** → **smallest fix** → **verification** → **prevention**. Do not inflate a local missing-file incident into a production outage. The learning value is your reasoning: you noticed that a bind mount concealed an incomplete final image, verified the missing asset in the actual runtime stage and added an artifact test. That is an honest, useful engineering discussion even without a fabricated latency metric.

For AI assistance, distinguish generated starter code from your decisions, modifications and tests. “I used an assisted starter, then implemented the chart and negative permission cases, reviewed each boundary and can explain the changes” is credible only when those actions happened. Merely reading the supplied reference solution does not justify “I designed the production architecture.” Keep commits and notes that show what you changed and why.

For an unfamiliar question, a strong response is not a bluff: “I have not configured that specific Kyma module version. I would check the installed API/schema, compare the versioned documentation and test the route in an approved sandbox. The generic Kubernetes behavior I can explain is...” Then explain that known boundary. Naming an uncertainty and a concrete investigation method demonstrates more judgment than claiming to remember an unverified setting.

11. Targeted revision, commands and glossary

11.1 Revision by symptom, not by restarting courses

Weak evidence	Return to workbook / questions	Matching course topic	Smallest useful repetition
Image builds but final runtime fails	1.2–1.3; DKR-06–10	Docker build context, COPY and multi-stage builds	E1/E3 missing-asset test
Host works, container does not	1.4; DKR-03, DKR-08	Docker ports/networking; selected systems lessons	E1 loopback variation
Types mistaken for input validation	2.1–2.3; TS-01–04, TS-07	Beginner's TypeScript object/union exercises	E2 malformed data
Discovery mistaken for execution	2.4–2.5, 5.3; MCP-01–06, MCP-08–10	HF MCP Unit 1; selected Unit 2 server/client	E3 list versus call
Pod state guessed from one log	4.5–4.6; K8S-08–12	Probes, resources, kubectl diagnosis	E4 wrong probe
Service selected as root cause without evidence	4.2–4.3; K8S-03–05, K8S-14	Labels, Services, target ports	E4 wrong selector
Async/install behavior unclear	5.1–5.2; TS-08–10	MDN async/promises; Node/npm reference	E3 build/test with lockfile
Values edit has no visible effect	7.2–7.4; HELM-03–07	Helm values/templates/helmers	E5 incorrect key
Rollback described as complete recovery	7.5; HELM-08–10	Helm release lifecycle; Kubernetes rollout	E5 failed-image recovery
OAuth terms collapse into "JWT login"	5.5–5.6, 8.1–8.5; AUTH-01–10	Official authorization walkthrough	E6 draw code/PKCE flow
Login success treated as full permission	8.5–8.7; AUTH-11–15	Token verification and operation policy	E6 limited-scope cases
Public integration claimed from port-forward	7.7, 8.8; KYMA-01–04, INT-01–04	Kyma reference; client packaging	Review three separate paths

These are diagnostic links, not another learning roadmap. Continue your existing course from the point you reached, using the smallest relevant repetition. Keep the main/parallel time split; a weakness in an advanced optional question need not interrupt the whole block.

11.2 Command reference with interpretation

Read commands before executing them. Commands containing uppercase placeholders require substitution. Logs can contain sensitive values in real systems; use approved access and redact

outputs. Commands here target the local workbook service unless stated otherwise. Read-only inspection should come before mutation.

Process, files and network

```
pwd; ls -la
printenv PORT                # inspect a specific non-secret variable
ps -ef
ss -lntp                     # Linux/WSL: listeners; visibility may need permission
curl -i http://127.0.0.1:3000/healthz
curl -v https://APPROVED_HOST/.well-known/oauth-protected-resource
```

Use the verbose HTTPS request only without real bearer credentials and review its output before sharing. On macOS, `lsof -nP -iTCP:3000 -sTCP:LISTEN` is a useful listener alternative. WSL commands inspect the WSL environment, not every Windows process. Windows-native PowerShell environment assignment differs from Bash; use the stated Bash default rather than copying `NAME=value` command unchanged into PowerShell. Refusal, timeout, TLS failure and an HTTP error are different observations. [T11](#) [T12](#)

Docker

```
docker ps -a
docker logs --tail 100 status-lab
docker inspect status-lab
docker port status-lab
docker exec status-lab node -e \
  'fetch("http://127.0.0.1:3000/healthz").then(r=>console.log(r.status))'
docker stop status-lab
```

`inspect` can expose environment secrets in real systems; restrict and redact it. `exec` needs a running container and the named executable must exist. A minimal image may not contain `curl`, `Bash` or a package manager. Do not install debugging tools into an immutable production container as your first step. `Stop` is a mutation; use it only for the named local exercise or an approved operational action. [D05](#) [D13](#) [D14](#)

Kubernetes

```
kubectl config current-context
kubectl -n workbook get deploy,rs,pods,svc,endpointslices
kubectl -n workbook describe pod POD_NAME
kubectl -n workbook logs POD_NAME --tail=100
kubectl -n workbook logs POD_NAME --previous
kubectl -n workbook get events --sort-by=.metadata.creationTimestamp
kubectl -n workbook rollout status deployment/status-lab --timeout=120s
kubectl -n workbook get endpointslices \
  -l kubernetes.io/service-name=status-lab -o wide
kubectl auth can-i get pods -n workbook
```

`--previous` asks for the previous terminated container instance where available, not all historical logs. Events and current status are transient evidence, not a durable incident database. In a multi-container Pod, select the intended container with `-c`; a sidecar's logs are not the application's logs. Use the Helm-generated Deployment/Service name `workbook-status-lab` after E5. The authorization check describes your Kubernetes API permission, not permission to invoke an MCP tool. [K06](#) [K12](#) [K14](#)

Helm 4.3

```
helm lint chart/status-lab
helm template workbook chart/status-lab -n workbook > rendered.yaml
helm get values workbook -n workbook --all
helm get manifest workbook -n workbook
```

```
helm status workbook -n workbook
helm history workbook -n workbook
helm rollback workbook GOOD_REVISION -n workbook --wait=watcher --timeout=2m
```

Render and review before install/upgrade. History and rollback are release operations, not a database recovery plan. `helm upgrade --rollback-on-failure` is the selected Helm 4 behavior; inspect your own version's help before translating a Helm 3 course example. Avoid printing secrets through dry-run or stored release values. [H05](#) [H06](#) [H07](#)

Auth diagnostics without secret spillage

```
curl -i -X POST http://127.0.0.1:3000/mcp
curl --fail http://127.0.0.1:3000/.well-known/oauth-protected-resource
# In service/, with the local provider and application running:
npm run oauth-client
SCOPE='mcp:connect' npm run oauth-client
SCOPE='status:read' npm run oauth-client
```

The first request tests a missing-token challenge, not a complete MCP interaction. The client obtains its own local token in memory. Do not create troubleshooting commands that paste employer bearer tokens into shell history. A metadata URL and a permission name are configuration, while a token is a credential. [O02](#) [O04](#) [M05](#)

11.3 Compact glossary

Image: packaged filesystem/runtime configuration used to create container instances. **Container:** a running or stopped instance with its own process/filesystem/network context, sharing the relevant host kernel rather than being a full separate guest kernel. **Build context:** files available to the builder for a particular build. **Layer/cache:** reusable build outputs whose validity depends on relevant inputs. **Digest:** content-addressed artifact identity; it does not freeze every external runtime dependency. **Bind mount:** a host path made available in the container. **Volume:** Docker-managed storage with a lifecycle separate from an individual container. [D01](#) [D02](#) [D03](#) [D06](#) [D09](#) [D10](#)

Listener: a process accepting connections on an address and port. **Loopback:** an address that refers back to the current network namespace, not universally to the physical laptop. **DNS:** name resolution, not proof that the resolved service is healthy. **TLS:** a secure transport protocol used by HTTPS; its trust/hostname checks are distinct from application permission checks. **Exit code:** a process result reported to its parent, useful evidence but not always a complete cause. [D07](#) [D08](#) [T09](#) [T11](#) [T12](#)

Union: a type representing multiple possible alternatives. **Narrowing:** establishing a more specific type from checks/control flow. **Runtime validation:** checking actual input values during execution. **Promise:** a representation of eventual completion or failure. **Event loop:** the mechanism scheduling JavaScript callbacks and related runtime work; long synchronous work can delay unrelated callbacks. **Lockfile:** a recorded dependency resolution used by reproducible installation procedures. [T01](#) [T02](#) [T04](#) [T06](#) [T07](#)

MCP host: the application coordinating model interactions and clients. **MCP client:** the protocol connection component interacting with a server. **MCP server:** the component exposing capabilities through the selected protocol. **Tool:** an invocable operation. **Resource:** exposed contextual data identified through the protocol. **Prompt:** a reusable interaction template. **Streamable HTTP:** the version-specific HTTP transport family; do not confuse it with the older separate HTTP+SSE transport. **Stateless application design:** no hidden caller-session state is required in one particular replica for the lab's next request. [M01](#) [M02](#) [M03](#) [M04](#)

Pod: Kubernetes' unit containing one or more closely associated containers with shared Pod-level context. **ReplicaSet:** controller maintaining a requested replica count for matching Pods.

Deployment: declarative management of replica sets and application updates. **Selector:** label-based criteria connecting resources. **Service:** stable discovery/access abstraction for selected backends. **EndpointSlice:** an API representation of network endpoints associated with a Service. **Namespace:** a resource-scoping mechanism, not an automatic complete security boundary. [K01](#) [K02](#) [K03](#) [K04](#) [K05](#)

Readiness: whether a container/Pod should be eligible for ordinary serving traffic. **Liveness:** whether a configured health failure should lead to a container restart. **Startup probe:** a startup-specific check that gates other probes. **Request:** scheduling/resource allocation input. **Limit:** a runtime resource constraint with resource-specific enforcement. **Service account:** workload identity for Kubernetes API interactions. **RBAC:** role-based authorization for specified API resources/actions. **Secret:** a Kubernetes object intended for sensitive data, requiring additional protection; base64 is not encryption. [K08](#) [K09](#) [K10](#) [K11](#) [K12](#)

Chart: a Helm package of templates and metadata. **Release:** a named installed instance with configuration/history. **Values:** template inputs. **Rendered manifest:** the Kubernetes YAML produced after template evaluation. **Rollback:** an update based on an earlier release revision, not reversal of all external side effects. **CRD:** a definition extending the Kubernetes API with a custom resource type. **APIRule:** Kyma API Gateway's custom resource for the version-appropriate exposure/access configuration. [H01](#) [H07](#) [H09](#) [Y01](#)

Authorization server: issuer of access tokens under its policy. **Resource server:** API accepting and validating access credentials. **Identity provider:** component authenticating identities, often integrated with an authorization server. **Scope:** a named access permission under an issuer/resource's contract. **Audience:** intended token recipient/resource identifier. **Issuer:** the authority that issued the token. **JWKS:** a JSON representation of keys used in a trusted verification relationship. **PKCE:** binding between a code-flow authorization request and token exchange through a verifier/challenge. **Public client:** a client unable to reliably protect a distributed secret. **Introspection:** a provider-supported token-status/metadata query, useful for appropriate token profiles. [O03](#) [O04](#) [O05](#) [O06](#) [O07](#) [O09](#)

11.4 Added topics and why they belong

Core — configuration validation and least privilege: malformed configuration can stop the service or weaken auth; Chapters 1, 4, 5 and 8 put these checks at the relevant boundaries. **Useful — automated domain/negative tests:** Chapters 2, 5 and E6 provide evidence independent of memorized answers. **Useful — structured logging/correlation:** Chapters 1, 5 and 8 support diagnosis without exposing credentials. **Useful — timeouts and bounded retries:** Chapter 5 and E3/E6 prevent indefinite waits and retrying forbidden operations. **Useful — code-review/rendered-diff practice:** Chapter 7 makes deployment changes reviewable. **Core — replica/stateless reasoning:** Chapters 4–5 prevents local-session assumptions from failing under two replicas. **Stretch — key-rotation experiments, independent diagnostic Pods and expanded browser integration:** introduced only as optional applications of the same boundaries.

These additions support this service and the first infrastructure block. They do not create a new required course on Terraform, distributed databases, GPU systems, model training or advanced networking.

11.5 Personal progress ledger

Use one line per question or exercise. A compact entry looks like: DKR-09 | Answered independently | 2026-10-__ | inspected final-stage asset path; repeat final-image test. The example date is a blank planning field, not a completion claim.

Before a session: choose two question IDs or one small exercise variation. **After a session:** record which distinction became clearer and what evidence you produced. **Before continuing to the next stage:** complete that stage's checkpoint and revise only the weak prerequisite. **Before an interview:** use the combined mock, then rehearse your actual project story with the tested/untested boundary intact.

12. Source bibliography and version notes

Research/access date for every entry: 22 September 2026. URLs are traceable source links, not requirements to read every page. Live documentation can change after access. Where a publication/update date was not displayed or verified, the entry says so rather than inferring one. The versions below describe the selected lab or observed documentation; they do not silently upgrade an employer project.

Technical answers use the primary documentation and standards. Published guides supply question provenance; their technical answers are not adopted blindly. Public candidate reports are anecdotal. The four employer descriptions are a small purposive sample used to assess topic relevance, not a representative market estimate. Public syllabuses support topic-level course mapping only.

Compatibility ledger. Reference app: Node 22.x, TypeScript 5.8.3, MCP SDK 1.30.0, Zod 3.25.76 and jose 6.1.3. Reference protocol family: 2025-11-25. Latest protocol observed: 2026-07-28, with incompatible lifecycle/SDK changes explained in 0.3. Helm command baseline: 4.3.0. Kubernetes manifests use stable APIs; record the local kind/node/kubectrl combination. Keycloak lab image: 26.7.4. Kyma examples: APIRule v2, subject to the actual installed module/schema.

The lack of a pre-generated lockfile is intentional because dependency resolution was not available in the authoring environment. Generate, review and retain a real one locally. A dependency pin and a selected documentation version do not constitute a complete security or compatibility certification.

Course syllabuses

C01 Docker Training Course for the Absolute Beginner — KodeKloud.

URL: <https://kodekloud.com/courses/docker-training-course-for-the-absolute-beginner>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Public syllabus only; purchased Udemy lecture sequence not verified.

C02 DevOps Pre-Requisite Course — KodeKloud.

URL: <https://kodekloud.com/courses/devops-pre-requisite-course>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Selected public syllabus topics only.

C03 Beginner's TypeScript — Total TypeScript / Matt Pocock.

URL: <https://www.totaltypescript.com/tutorials/beginners-typescript>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Public exercise-based course; 18 lessons advertised; no claim of watching videos.

C04 MCP Course, Unit 1: Fundamentals — Hugging Face.

URL: <https://huggingface.co/learn/mcp-course/unit1/introduction>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Course concepts; implementation details must be matched to protocol version.

C05 MCP Course, Unit 2: End-to-End Application — Hugging Face.

URL: <https://huggingface.co/learn/mcp-course/unit2/introduction>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Selected server/client topics; Gradio/Hugging Face deployment not imposed.

C06 Kubernetes for the Absolute Beginners, Hands-on Tutorial — KodeKloud.

URL: <https://kodekloud.com/courses/kubernetes-for-the-absolute-beginners-hands-on-tutorial>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Public syllabus; topic-level mapping rather than paid lecture numbers.

C07 Helm for Beginners — KodeKloud.

URL: <https://kodekloud.com/courses/helm-for-beginners>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Public syllabus; course may use an earlier Helm CLI.

Docker

D01 What is a container? — Docker.

URL: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D02 What is an image? — Docker.

URL: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D03 Building best practices — Docker.

URL: <https://docs.docker.com/build/building/best-practices/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D04 Optimize cache usage in builds — Docker.

URL: <https://docs.docker.com/build/cache/optimize/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D05 Dockerfile reference — Docker.

URL: <https://docs.docker.com/reference/dockerfile/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D06 Build context and .dockerignore — Docker.

URL: <https://docs.docker.com/build/concepts/context/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D07 Port publishing and mapping — Docker.

URL: <https://docs.docker.com/engine/network/port-publishing/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D08 Bridge network driver — Docker.

URL: <https://docs.docker.com/engine/network/drivers/bridge/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D09 Volumes — Docker.

URL: <https://docs.docker.com/engine/storage/volumes/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D10 Bind mounts — Docker.

URL: <https://docs.docker.com/engine/storage/bind-mounts/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D11 Multi-stage builds — Docker.

URL: <https://docs.docker.com/build/building/multi-stage/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D12 Build secrets — Docker.

URL: <https://docs.docker.com/build/building/secrets/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D13 Start containers automatically — Docker.

URL: <https://docs.docker.com/engine/containers/start-containers-automatically/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

D14 docker container stop — Docker.

URL: <https://docs.docker.com/reference/cli/docker/container/stop/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Engine/BuildKit current docs; basic Linux-container behavior. Limitation/use: Primary documentation.

Kubernetes and local cluster

K01 Pods — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/workloads/pods/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K02 Deployments — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K03 ReplicaSet — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K04 Labels and selectors — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K05 Service — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/services-networking/service/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K06 Debug Services — Kubernetes.

URL: <https://kubernetes.io/docs/tasks/debug/debug-application/debug-service/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K07 ConfigMaps — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/configuration/configmap/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K08 Secrets — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/configuration/secret/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K09 Configure liveness, readiness and startup probes — Kubernetes.

URL: <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K10 Resource management for Pods and containers — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K11 Service accounts — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/security/service-accounts/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K12 Using RBAC authorization — Kubernetes.

URL: <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K13 Volumes — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/storage/volumes/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K14 Pod lifecycle — Kubernetes.

URL: <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current v1.37 documentation; examples use stable v1/apps-v1 resources. Limitation/use: Primary documentation.

K15 kind Quick Start — Kubernetes SIGs / kind.

URL: <https://kind.sigs.k8s.io/docs/user/quick-start/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Local setup; choose the documented node image for the installed kind release.

TypeScript, JavaScript, Node and npm

T01 Everyday types — TypeScript.

URL: <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T02 Narrowing — TypeScript.

URL: <https://www.typescriptlang.org/docs/handbook/2/narrowing.html>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T03 Modules — TypeScript.

URL: <https://www.typescriptlang.org/docs/handbook/2/modules.html>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T04 Using promises — MDN Web Docs.

URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T05 async function — MDN Web Docs.

URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T06 Do not block the event loop — Node.js.

URL: <https://nodejs.org/learn/asynchronous-work/dont-block-the-event-loop>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T07 npm ci — npm.

URL: <https://docs.npmjs.com/cli/v10/commands/npm-ci/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T08 Lifecycle scripts — npm.

URL: <https://docs.npmjs.com/cli/v10/using-npm/scripts/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T09 Process API — Node.js.

URL: <https://nodejs.org/docs/latest-v22.x/api/process.html>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T10 HTTP API — Node.js.

URL: <https://nodejs.org/docs/latest-v22.x/api/http.html>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T11 An overview of HTTP — MDN Web Docs.

URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

T12 Transport Layer Security — MDN Web Docs.

URL: https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Transport_Layer_Security

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: TypeScript lab 5.8.3 / Node lab 22.x / npm lab 10.x; language docs live. Limitation/use: Primary documentation.

MCP specifications and SDK

M01 TypeScript SDK v1 documentation — Model Context Protocol.

URL: <https://ts.sdk.modelcontextprotocol.io/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Maintained v1; lab pins 1.30.0. Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M02 Transports — Model Context Protocol.

URL: <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 2025-11-25. Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M03 Tools — Model Context Protocol.

URL: <https://modelcontextprotocol.io/specification/2025-11-25/server/tools>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 2025-11-25.
Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M04 Lifecycle — Model Context Protocol.

URL: <https://modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 2025-11-25.
Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M05 Authorization — Model Context Protocol.

URL: <https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 2025-11-25.
Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M06 Security best practices — Model Context Protocol.

URL: https://modelcontextprotocol.io/docs/2025-11-25/tutorials/security/security_best_practices

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 2025-11-25.
Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M07 Stateless Streamable HTTP example — Model Context Protocol.

URL: <https://github.com/modelcontextprotocol/typescript-sdk/blob/v1.x/src/examples/server/simpleStatelessStreamableHttp.ts>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: v1.x moving branch, checked against lab v1 API. Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M08 Latest specification — Model Context Protocol.

URL: <https://modelcontextprotocol.io/specification/2026-07-28>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 2026-07-28.
Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M09 2026-07-28 changelog — Model Context Protocol.

URL: <https://modelcontextprotocol.io/specification/2026-07-28/changelog>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 2026-07-28.
Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M10 TypeScript SDK releases — Model Context Protocol.

URL: <https://github.com/modelcontextprotocol/typescript-sdk/releases>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: v1.30.0 and v2.0.0 releases checked, July 2026. Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M11 Current authorization specification — Model Context Protocol.

URL: <https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 2026-07-28.
Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

M12 Understanding authorization tutorial — Model Context Protocol.

URL: <https://modelcontextprotocol.io/docs/tutorials/security/authorization>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current URL; tutorial examples require SDK-version review. Limitation/use: Official specification, tutorial or SDK source; version boundary stated in workbook.

OAuth, OIDC, JWT and provider

O01 RFC 9700: Best Current Practice for OAuth 2.0 Security — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc9700.html>

Publication/update: January 2025. Accessed: 22 September 2026. Version/context: RFC 9700. Limitation/use: Primary documentation.

O02 RFC 9728: OAuth 2.0 Protected Resource Metadata — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc9728.html>

Publication/update: April 2025. Accessed: 22 September 2026. Version/context: RFC 9728. Limitation/use: Primary documentation.

O03 RFC 8414: OAuth 2.0 Authorization Server Metadata — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc8414.html>

Publication/update: June 2018. Accessed: 22 September 2026. Version/context: RFC 8414. Limitation/use: Primary documentation.

O04 RFC 6750: OAuth 2.0 Bearer Token Usage — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc6750.html>

Publication/update: October 2012. Accessed: 22 September 2026. Version/context: RFC 6750. Limitation/use: Primary documentation.

O05 RFC 7636: Proof Key for Code Exchange — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc7636.html>

Publication/update: September 2015. Accessed: 22 September 2026. Version/context: RFC 7636. Limitation/use: Primary documentation.

O06 RFC 7662: OAuth 2.0 Token Introspection — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc7662.html>

Publication/update: October 2015. Accessed: 22 September 2026. Version/context: RFC 7662. Limitation/use: Primary documentation.

O07 RFC 9068: JSON Web Token Profile for OAuth 2.0 Access Tokens — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc9068.html>

Publication/update: October 2021. Accessed: 22 September 2026. Version/context: RFC 9068. Limitation/use: Primary documentation.

O08 RFC 8707: Resource Indicators for OAuth 2.0 — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc8707.html>

Publication/update: February 2020. Accessed: 22 September 2026. Version/context: RFC 8707. Limitation/use: Primary documentation.

O09 RFC 8252: OAuth 2.0 for Native Apps — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc8252.html>

Publication/update: October 2017. Accessed: 22 September 2026. Version/context: RFC 8252. Limitation/use: Primary documentation.

O10 RFC 7591: OAuth 2.0 Dynamic Client Registration Protocol — IETF / RFC Editor.

URL: <https://www.rfc-editor.org/rfc/rfc7591.html>

Publication/update: July 2015. Accessed: 22 September 2026. Version/context: RFC 7591. Limitation/use: Primary documentation.

O11 OpenID Connect Core 1.0, errata set 2 — OpenID Foundation.

URL: https://openid.net/specs/openid-connect-core-1_0.html

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Core 1.0 with errata set 2. Limitation/use: Primary documentation.

O12 OpenID Connect layers and endpoints — Keycloak.

URL: <https://www.keycloak.org/securing-apps/oidc-layers>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Keycloak documentation; local lab 26.7.4. Limitation/use: Primary documentation.

O13 jose: JWT/JWK/JWKS implementation — panva / jose maintainers.

URL: <https://github.com/panva/jose>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab dependency 6.1.3; current upstream docs. Limitation/use: Primary documentation.

O14 Downloads — Keycloak.

URL: <https://www.keycloak.org/downloads>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 26.7.4 displayed on research date. Limitation/use: Version selection only; container not executed during authoring.

O15 Running Keycloak in Docker — Keycloak.

URL: <https://www.keycloak.org/getting-started/getting-started-docker>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: 26.7.4 selected local image. Limitation/use: Development mode is for a local disposable exercise, not production.

O16 Importing and exporting realms — Keycloak.

URL: <https://www.keycloak.org/server/importExport>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Current documentation; 26.7.4 lab. Limitation/use: Startup import skips existing realms; the lab uses disposable container storage.

Helm

H01 Values files — Helm.

URL: https://helm.sh/docs/chart_template_guide/values_files/

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H02 Template functions and pipelines — Helm.

URL: https://helm.sh/docs/chart_template_guide/functions_and_pipelines/

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H03 Flow control — Helm.

URL: https://helm.sh/docs/chart_template_guide/control_structures/

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H04 Named templates — Helm.

URL: https://helm.sh/docs/chart_template_guide/named_templates/

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H05 Debugging templates — Helm.

URL: https://helm.sh/docs/chart_template_guide/debugging/

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H06 helm upgrade — Helm.

URL: https://helm.sh/docs/helm/helm_upgrade/

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H07 helm rollback — Helm.

URL: https://helm.sh/docs/helm/helm_rollback/

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H08 Supported version skew — Helm.

URL: https://helm.sh/docs/topics/version_skew/

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H09 Charts — Helm.

URL: <https://helm.sh/docs/topics/charts/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H10 Helm releases — Helm.

URL: <https://github.com/helm/helm/releases>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

H11 Helm v3 end of life — Helm.

URL: <https://helm.sh/blog/helm-v3-end-of-life/>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Lab CLI reference Helm 4.3.0; chart API v2. Limitation/use: Official docs; template guide contains some older-version material, CLI flags checked separately.

Kyma

Y01 APIRule custom resource — Kyma project / SAP.

URL: <https://kyma-project.io/external-content/api-gateway/docs/user/custom-resources/apirule/04-10-apirule-custom-resource.html>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: gateway.kyma-project.io/v2; installed module/mesh must be checked. Limitation/use: No inference about SAP employer cluster, gateway name or access policies.

Y02 Changes in APIRule v2 — Kyma project / SAP.

URL: <https://kyma-project.io/external-content/api-gateway/docs/user/custom-resources/apirule/04-70-changes-in-apirule-v2.html>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: gateway.kyma-project.io/v2; installed module/mesh must be checked. Limitation/use: No inference about SAP employer cluster, gateway name or access policies.

Y03 Migrate JWT APIRule v1beta1 to v2 — Kyma project / SAP.

URL: <https://help.sap.com/docs/btp/sap-business-technology-platform/migrate-apirule-v1beta1-of-type-jwt-to-version-v2>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: gateway.kyma-project.io/v2; installed module/mesh must be checked. Limitation/use: No inference about SAP employer cluster, gateway name or access policies.

Interview-question provenance

I01 12 Essential Docker Interview Questions — Toptal.

URL: <https://www.toptal.com/developers/docker/interview-questions>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Published guide as accessible on research date. Limitation/use: Question provenance only, not proof of employer use or authority for technical answers; some answers dated.

I02 8 Essential Node.js Interview Questions — Toptal.

URL: <https://www.toptal.com/developers/nodejs/interview-questions>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Published guide as accessible on research date. Limitation/use: Question provenance only, not proof of employer use or authority for technical answers; some answers dated.

I03 11 Essential Linux Interview Questions — Toptal.

URL: <https://www.toptal.com/developers/linux/interview-questions>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Published guide as accessible on research date. Limitation/use: Question provenance only, not proof of employer use or authority for technical answers; some answers dated.

I04 TypeScript developer hiring guide — Toptal.

URL: <https://www.toptal.com/developers/typescript>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Published guide as accessible on research date. Limitation/use: Question provenance only, not proof of employer use or authority for technical answers; some answers dated.

I05 14 Essential DevOps Interview Questions — Toptal.

URL: <https://www.toptal.com/developers/devops/interview-questions>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Published guide as accessible on research date. Limitation/use: Question provenance only, not proof of employer use or authority for technical answers; some answers dated.

I06 Kubernetes developer hiring guide — Toptal.

URL: <https://www.toptal.com/developers/kubernetes>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Published guide as accessible on research date. Limitation/use: Question provenance only, not proof of employer use or authority for technical answers; some answers dated.

I07 Helm Interview Questions 2026: Real Answers and Commands — KodeKloud / Nimesha Jinarajadasa.

URL: <https://kodekloud.com/blog/helm-interview-questions/>

Publication/update: 25 July 2026. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: Published guide; marketing claims about frequency or real employer use not adopted.

I08 In an interview they asked me how K8s Service... — Reddit r/devops, original poster account deleted.

URL: https://www.reddit.com/r/devops/comments/198zj1m/in_an_interview_they_asked_me_how_k8s_service/

Publication/update: 17 January 2024. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: First-person candidate report; anecdotal, employer not independently verified; reported proposed answer not treated as correct.

I09 Kubernetes interview gone really bad — Reddit r/devops.

URL: https://www.reddit.com/r/devops/comments/1tlokfw/kubernetes_interview_gone_really_bad/

Publication/update: Exact date not verified; page displayed relative age. Accessed: 22 September 2026. Version/context: Live page accessed 22 September 2026. Limitation/use: First-person senior-role report; role-specific anecdotes, not a frequency survey.

Employer job descriptions

J01 Distributed Systems Testing Software Engineer, Python / Go — Canonical.

URL: <https://canonical.com/careers/2969042>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Live vacancy page, EMEA among eligible regions. Limitation/use: Requirements inform relevance, not evidence of interview questions; posting date not shown.

J02 Platform Engineer (m/f/d) — Polarise.

URL: <https://polarise.jobs.personio.com/job/2743985>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Germany; live employer-hosted job description. Limitation/use: Requirements reviewed; posting date not shown; experienced-role reference, not guaranteed learner fit.

J03 Junior Full Stack Developer, Python + React (m/f/d) — Preisenergie.

URL: <https://preisenergie.jobs.personio.de/job/598727?language=en>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Munich; live employer-hosted job description. Limitation/use: Requirements reviewed; frontend part extends beyond this workbook; posting date not shown.

J04 Senior Software Engineer (Python Backend & AI Infrastructure) — credium.

URL: <https://credium.jobs.personio.de/job/2734269?language=en>

Publication/update: Not stated; live documentation. Accessed: 22 September 2026. Version/context: Hybrid role; Europe-focused employer description, exact hiring geography not assumed. Limitation/use: Senior requirements used only as an adjacent direction, not a current application recommendation.

13. Coverage audit and evidence limits

13.1 Roadmap coverage

The supplied roadmap controls the required order: Docker/systems with TypeScript/MCP foundations; Kubernetes with implementation/auth overview; Helm/Kyma with complete authorization and integration. The table groups closely related items to keep the audit usable. Each listed exercise refers to the same status-lab service. “Review” means a written/configuration investigation is sufficient; it is not a claim of a running cluster or provider.

Roadmap topic or required adjacent prerequisite	Teaching / answer sections	Question evidence	Exercise or assessment
Images, containers, VMs and process boundaries	1.1	DKR-01-02	E1; S1-CB1
Dockerfile, build context, layers/cache; manifests before source	1.2-1.3	DKR-06-07, DKR-10	E1/E3; F-01
COPY, RUN, CMD, ENTRYPOINT; build versus runtime config	1.2	DKR-07-08	E1/E3
EXPOSE, publishing, localhost versus external listener	1.4	DKR-03, DKR-08	E1 loopback break
Environment variables, paths, permissions, shell quoting	1.1-1.4	DKR-08, DKR-11, SYS-01-02	E1/E2; command reference
Linux processes/listening ports, pipelines and exit codes	1.1, 1.4-1.5	DKR-02, SYS-01-02	E1; S1-T
Bind mounts, volumes and persistence	1.5	DKR-04	E1 mount variation
Container networking, DNS and host/container boundaries	1.4	DKR-03, DKR-08	E1; F-06
IP/TCP/HTTP/HTTPS/TLS; refusal, timeout and application errors	1.4, 7.7	SYS-02, KYMA-01	E1; 11.2
Logs, process exit, restart policy and diagnosis	1.5	DKR-02, DKR-12	E1/E3
Multi-stage builds and build/runtime dependencies	1.3, 5.2	DKR-10, TS-10	E3 final image
.dockerignore, runtime files and accidental omissions	1.3, 8.8	DKR-09, INT-01	E3 missing-file break
Tags, digests, reproducibility and image security	1.3	DKR-05, DKR-11	E1/E3; F-01
Non-root, secrets and graceful shutdown/signals	1.3, 1.5	DKR-11-12	E1; rollout discussion
JS prerequisites, TS types/interfaces, unions and narrowing	2.1-2.3	TS-01-05	E2
Optional values; compile-time versus runtime validation	2.1-2.3	TS-01, TS-03-04, TS-07	E2/E3
Modules/imports, structure and reading a handler	2.3, 5.2-5.3	TS-06-07, TS-10	E2/E3

Roadmap topic or required adjacent prerequisite	Teaching / answer sections	Question evidence	Exercise or assessment
MCP host/client/server and local/remote execution	2.4–2.6	MCP-01, MCP-04, MCP-07	E2 trace; F-06
Tools/resources/prompts, schemas and discovery	2.4–2.5	MCP-02–03	E2/E3
stdio, version-appropriate HTTP and protocol logging	2.5, 5.3	MCP-04–06	E2/E3
Kubernetes purpose, desired state and object ownership	4.1	K8S-01–02	E4 replacement Pod
Pods, Deployments, ReplicaSets, namespaces and YAML	4.1–4.2	K8S-02, K8S-04	E4
Labels/selectors, Services, DNS and port routing	4.2–4.3	K8S-03–05, K8S-14	E4 selector break
ConfigMaps, Secrets and their limitations	4.4	K8S-06–07	E4/E5
Readiness/liveness/startup probes	4.5	K8S-08	E4 wrong-probe break
CPU/memory requests, limits and observed failure	4.5	K8S-09	E4; S2-T
Relevant storage and service-account/access basics	4.4	K8S-10–11	E4 review
kubectl get/describe/logs/exec/events	4.6, 11.2	K8S-12, K8S-14	E4
Rollout prerequisites and unready replacement	4.5–4.6, 7.5	K8S-13, HELM-10	E4/E5
Pending, image pull, crash, Service and memory failures	4.5–4.6	K8S-09, K8S-12–14	E4; S2 mock
Promises, async/await, errors and Node execution	5.1, 5.4	TS-08–09	E3
npm scripts/dependencies/lockfiles; ci versus install	5.2	TS-10	E3 bootstrap/build
Disabled lifecycle scripts and consequences	5.2	TS-10	E3 review
Validated read-only tool, registration, execution/errors	5.3–5.4	MCP-08–09	E3
Compatible client, HTTP health versus MCP interaction	5.3–5.4	MCP-10, MCP-12	E3/E4
OAuth roles, authentication versus authorization	5.5–5.6	AUTH-01–02, AUTH-05	E3 trace; E6
Access tokens, metadata, decoding versus validation	5.6	AUTH-03–05	E6
Helm chart/values/templates/helpers and releases	7.1–7.2	HELM-01–03	E5
Overrides, conditions, loops, context, quoting/indentation	7.3	HELM-04–06	E5 render/break

Roadmap topic or required adjacent prerequisite	Teaching / answer sections	Question evidence	Exercise or assessment
Lint/template/install/upgrade/inspect/rollback	7.4–7.5	HELM-07–08	E5
Environment config, secret separation and rendered review	7.3–7.6	HELM-04, HELM-07, HELM-09	E5
Kyma exposure path, current API and generic K8s distinction	7.7	KYMA-01–02	E5 review; F-06
Readiness and limits of zero-downtime claims	7.5	HELM-10	E5; S3-D
Protected-resource and authorization-server metadata	8.2	AUTH-06	E6 metadata checks
Registration mechanisms; actual provider/client compatibility	8.3	AUTH-07	E6 pre-registration
Applicable flows, PKCE and public/confidential clients	8.3–8.4	AUTH-08, AUTH-12	E6
JWT or opaque validation; issuer/audience/time/signature	8.1, 8.5	AUTH-05, AUTH-09–10	E6 negative matrix
Signing-key discovery/rotation and trusted key source	8.5	AUTH-10	E6 review/stretch
Scopes, app permissions and per-tool checks	8.5–8.6	AUTH-11–12, AUTH-15	E6 limited scopes
Missing/expired/invalid/insufficient-permission cases	8.6	AUTH-09, AUTH-11, AUTH-15	E6; F-05
Audience restrictions, forwarding and trust boundaries	8.7	AUTH-13–14	E6 review; final mock
Client/plugin packaging and configuration placement	8.8	INT-01, MCP-07	E2/E3; F-01/F-06
Network path separately from auth/application execution	8.8, 10.1	INT-02, KYMA-03–04	Final integrated assessment
Request-local principal and native/browser integration	8.7–8.8	INT-03–04	E6 review
Honest interview story, debugging evidence and AI assistance	10.3–10.6	TS-06; stage/final mocks	Personal evidence portfolio

13.2 Quality checks and explicit limits

The assembly audit verifies that every bank question has a stable unique ID, a question, provenance/priority/type metadata, an interview-ready answer, deeper explanation, example, weak-answer warning and an answered follow-up. The six topic banks contain **84 distinct questions**, 28 in each stage. Stage checkpoints and mock-interview prompts are additional; they are not inflated into that question-bank count.

There are **six integrated exercise milestones**, each with independent tasks, expected observations, hints, a reference solution, verification and cleanup. The same service is used

throughout. A stage-level break/fix variation exists in every stage. Core scope stops at the end of this block. Adjacent additions are justified and labelled in 11.4.

The supplied brief was read in full. Public course pages informed topic-level mapping, but no paid video viewing, exact private lecture numbering, employer implementation, interview frequency or company usage has been invented. Published-guide questions establish provenance only. Candidate reports remain anecdotal. Employer job descriptions establish examples of skill demand, not a labor-market survey or proof of an interview question.

The native HTTP request/shutdown tests, four domain tests and seven configuration-boundary checks were actually executed. JSON/ordinary YAML and shell syntax were checked. The full SDK/dependency build, container/cluster/Helm/provider/Kyma integrations were not executed. Their commands and expected results are supplied for local verification, and the project includes `VALIDATION.md` with the exact boundary. A parsed YAML document is not a rendered Helm chart or a validated Kubernetes resource.

The default local Kubernetes/Helm configuration is private and explicitly unauthenticated with mock data; it creates no public ingress. The authenticated provider lab runs on the host with loopback endpoints. The reserved-domain authenticated values/APIRule files are **review-only**, not a claimed completed remote deployment. The native OAuth client uses a known issuer and is not a generic production MCP authorization library. No lockfile or successful deployment output is fabricated.

13.3 What remains outside the required block

No required topic from the supplied first block is intentionally omitted. The following are intentionally not claimed as completed implementations: a generic multi-provider OAuth client, a browser-host plugin manifest for a named product, production secret storage/rotation, a complete remote Kyma deployment, and operational availability under real load. Each would require the actual client/provider/platform contract and additional tests. The workbook teaches the relevant review boundaries and supplies the appropriate local exercises without inventing access or requirements.

Advanced infrastructure products, GPU serving, model training, distributed database internals and unrelated cloud certifications are outside this block. Finish the demonstrated TypeScript → MCP → Docker → Kubernetes → Helm chain before defining another required learning block.

Appendix A. Complete reference project

These are the full reference files for exercises E1–E6. Read the task briefs before using the solutions. The accompanying project archive preserves the same paths and contents for direct use. The initial real package-lock.json must be generated locally; it is deliberately not fabricated here.

Every file is a teaching example unless the validation record specifically identifies an executed check. The reserved-domain Kyma/authenticated-deployment examples are review-only. Default unauthenticated resources must stay in the private local lab.

A.01 .gitignore

```
node_modules/
dist/
.env
.env.*
*.log
artifacts/
```

A.02 README.md

```
# Status Lab – reference solution, 22 September 2026
```

```
One evolving mock service, not employer code. Read the companion workbook before opening solutions.
```

```
## Milestones
```

1. `milestones/01-http`: native Node HTTP starter and Docker solution.
2. `service`: TypeScript domain, stateless v1 MCP Streamable HTTP, client, tests and image.
3. `k8s`: ordinary manifests, local-cluster-only auth-disabled exercise.
4. `chart`: Helm packaging of that deployment; no public exposure resource.
5. `auth`: local Keycloak and authorization-code/PKCE exercise for the same service.

```
## Execution status
```

```
See `VALIDATION.md`. The HTTP starter and dependency-free domain tests were executed in the authoring container. MCP SDK integration, Docker, Kubernetes, Helm, Keycloak and Kyma were NOT executed there. Network access to the package registry and cluster tools was unavailable. No package-lock is fabricated.
```

```
## Bootstrap (Node 22.x, npm 10.x, Bash)
```

```
```sh
cd service
npm install # first resolution; inspect and commit package-lock.json
npm test
npm ci # now demonstrate installation from that actual lock
AUTH_MODE=disabled ALLOW_UNAUTHENTICATED_LOCAL_LAB=true npm start
another terminal, in service:
npm run client
```
```

```
This uses maintained SDK v1.30.0 and the 2025-11-25 protocol family, deliberately matching many course examples. It is NOT a sample of the 2026-07-28 protocol or SDK v2. Do not mix imports/handshakes from those versions. The local client does not persist tokens and is not a production OAuth implementation.
```

```
## Docker (after the initial install creates a real lockfile)
```

```
```sh
cd service
docker build -t status-lab:0.1.0 .
docker run --rm --name status-lab -p 127.0.0.1:3000:3000 \
```

```
-e AUTH_MODE=disabled -e ALLOW_UNAUTHENTICATED_LOCAL_LAB=true status-lab:0.1.0
```

```

Local Kubernetes

Use a disposable kind cluster called `workbook`, Linux containers, and a compatible kubectl. Use a current supported kind release and its documented default node image; record the resulting versions.

```
```sh
kind create cluster --name workbook
kind load docker-image status-lab:0.1.0 --name workbook
kubectl apply -f k8s/status-lab.yaml
kubectl -n workbook rollout status deployment/status-lab --timeout=120s
kubectl -n workbook port-forward service/status-lab 3000:80
```

```

Run the SDK client from the host. Port-forward proves the selected backend, not the entire in-cluster network path.

The manifests are intentionally private, unauthenticated mock-data lab resources. Do not add public ingress.

Helm 4.3 command baseline

Use `workbook` as both release and namespace here: the default allowed-host names assume that release.

Stop port-forward and remove only the previous lab resources before installing the chart.

WARNING: the delete command below also deletes the workbook namespace and all its contents.

Use it only in the disposable kind cluster after checking the current context.

```
```sh
kubectl delete -f k8s/status-lab.yaml
helm lint chart/status-lab
helm template workbook chart/status-lab -n workbook > rendered.yaml
helm upgrade --install workbook chart/status-lab -n workbook --create-namespace \
 --wait=watcher --timeout 2m
kubectl -n workbook port-forward service/workbook-status-lab 3000:80
```

```

Changing Helm release name or port requires matching `config.ALLOWED\_HOSTS`. Do not disable the check blindly.

`existingSecret` is an optional reference: this mock service does not require a client secret.

Local OAuth (application on host, not inside Kubernetes)

Run `bash auth/run-keycloak.sh`, then open <http://127.0.0.1:8080>.

Log in as workbook-admin using the disposable password you entered. In the workbook realm, create a disposable

user, enable it, set a non-temporary password, and remove required profile actions for this local user.

Never use an employer account or password. Check the client is public, standard flow on, PKCE S256, exact loopback redirect, both mcp:connect and status:read as optional scopes. Inspect the realm import if UI labels differ.

```
```sh
cd service
source ../auth/server-env.sh
npm start
another terminal in service:
npm run oauth-client
negative permission case:
SCOPE=mcp:connect npm run oauth-client
```

```

The reference realm issues a fixed audience for one resource. This is a local teaching shortcut, not a general

multi-resource OAuth authorization policy. The client sends the resource parameter; provider support and

production mapping must be reviewed separately. No DCR, refresh-token storage or browser CORS is implemented.

```
## Cleanup – destructive to these named LAB resources only
```

```
```sh
docker stop workbook-keycloak # removes this --rm container and its disposable users
helm uninstall workbook -n workbook
kind delete cluster --name workbook
```
```

Stop Node with Ctrl+C; remove only known temporary lab artifacts. Never use blanket system-prune commands.

A.03 VALIDATION.md

```
# Validation record – 22 September 2026
```

This record separates executed checks from reviewed reference examples.

```
## Executed in the authoring container
```

- Node 22.16.0: six native HTTP request checks and a clean SIGTERM exit.
- TypeScript 5.8.3: compiled the dependency-free domain, its tests and config module.
- Node test runner: four domain tests passed (valid data, unknown name, invalid data, exact scope matching).
- Seven process-level configuration checks passed: disabled-auth opt-in, valid opt-in, invalid port, missing JWT settings, non-loopback HTTP issuer, accepted loopback issuer, and rejection of non-loopback HTTP resource URLs.
- Parsed the shipped JSON and non-template YAML files.
- Bash syntax checks passed for the two auth shell files.

The isolated TypeScript checks used the authoring environment's installed Node type information. They are not a substitute for installing the project's real dependencies and running npm test locally.

```
## Not executed here
```

No full dependency installation/build, MCP SDK interaction, JWT/provider integration, Docker build/run, Kubernetes deployment, Helm render/install/rollback, Keycloak login, or Kyma exposure was executed. Those tools/services or registry access were unavailable. Template source review and plain YAML parsing are not claimed Helm/Kubernetes validation.

```
## Local verification
```

Follow E1–E6 in the workbook. First run npm install in service/, review and commit the real package-lock.json, then run npm test and npm ci. No fictitious lockfile is shipped. Record actual versions, commands, pass/fail categories and dates. Do not record passwords, authorization codes, access tokens or private keys. Keep negative permission cases after the normal case succeeds.

```
## Important safety and compatibility limits
```

The MCP reference is SDK v1.30.0 / 2025-11-25 protocol family, not SDK v2 / July 2026. The local auth-disabled chart must never be publicly exposed. The OAuth client is a single-known-issuer loopback teaching client, not a complete generic OAuth implementation. Keycloak's fixed single-resource audience mapper is a teaching shortcut. Authenticated remote values and APIRule are review-only and use reserved domains. The app must receive the bearer token through an explicitly reviewed gateway path if it verifies it itself. The key-fetch/verification external error policy is deliberately coarse; production handling should distinguish operational dependency failures in protected diagnostics.

A.04 auth/run-keycloak.sh

```
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")"
read -r -s -p "Disposable local Keycloak admin password: " KC_BOOTSTRAP_ADMIN_PASSWORD
printf '\n'
export KC_BOOTSTRAP_ADMIN_PASSWORD
# Local development only. No persistence: removing the container discards the realm/users.
docker run --rm --name workbook-keycloak \
```

```
-p 127.0.0.1:8080:8080 \
-e KC_BOOTSTRAP_ADMIN_USERNAME=workbook-admin \
-e KC_BOOTSTRAP_ADMIN_PASSWORD \
-v "$PWD/workbook-realm.json:/opt/keycloak/data/import/workbook-realm.json:ro" \
quay.io/keycloak/keycloak:26.7.4 start-dev --import-realm
```

A.05 auth/server-env.sh

```
# Source from the service directory: source ../auth/server-env.sh
export AUTH_MODE=jwt
export HOST=127.0.0.1 PORT=3000
export RESOURCE_URL=http://127.0.0.1:3000/mcp
export OAUTH_ISSUER=http://127.0.0.1:8080/realms/workbook
export OAUTH_JWKS_URL=http://127.0.0.1:8080/realms/workbook/protocol/openid-connect/certs
export ALLOWED_HOSTS=127.0.0.1:3000,localhost:3000
unset ALLOW_UNAUTHENTICATED_LOCAL_LAB
```

A.06 auth/workbook-realm.json

```
{
  "realm": "workbook",
  "enabled": true,
  "registrationAllowed": false,
  "accessTokenLifespan": 120,
  "sslRequired": "external",
  "clientScopes": [
    {
      "name": "mcp:connect",
      "protocol": "openid-connect",
      "attributes": {
        "include.in.token.scope": "true"
      }
    },
    {
      "name": "status:read",
      "protocol": "openid-connect",
      "attributes": {
        "include.in.token.scope": "true"
      }
    }
  ],
  "clients": [
    {
      "clientId": "workbook-native",
      "name": "Local workbook native client",
      "protocol": "openid-connect",
      "publicClient": true,
      "standardFlowEnabled": true,
      "directAccessGrantsEnabled": false,
      "serviceAccountsEnabled": false,
      "redirectUris": [
        "http://127.0.0.1:8787/callback"
      ],
      "webOrigins": [],
      "attributes": {
        "pkce.code.challenge.method": "S256"
      },
      "defaultClientScopes": [],
      "optionalClientScopes": [
        "mcp:connect",
        "status:read"
      ],
      "protocolMappers": [
        {
          "name": "workbook-resource-audience",
          "protocol": "openid-connect",
          "protocolMapper": "oidc-audience-mapper",
          "consentRequired": false,
          "config": {
```

```

        "included.custom.audience": "http://127.0.0.1:3000/mcp",
        "access.token.claim": "true",
        "id.token.claim": "false"
      }
    }
  ]
}
}
}

```

A.07 chart/status-lab/Chart.yaml

```

apiVersion: v2
name: status-lab
description: Local mock status MCP learning service
type: application
version: 0.1.0
appVersion: "0.1.0"

```

A.08 chart/status-lab/templates/_helpers.tpl

```

{{- define "status-lab.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
{{- end -}}
{{- define "status-lab.selectorLabels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end -}}

```

A.09 chart/status-lab/templates/configmap.yaml

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "status-lab.fullname" . }}-config
data:
  HOST: "0.0.0.0"
  PORT: {{ .Values.containerPort | quote }}
  {{- range $key, $value := .Values.config }}
  {{ $key }}: {{ $value | quote }}
  {{- end }}

```

A.10 chart/status-lab/templates/deployment.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "status-lab.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      {{- include "status-lab.selectorLabels" . | nindent 6 }}
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        {{- include "status-lab.selectorLabels" . | nindent 8 }}
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      serviceAccountName: {{ include "status-lab.fullname" . }}

```

```

automountServiceAccountToken: false
terminationGracePeriodSeconds: 30
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  runAsGroup: 1000
  seccompProfile:
    type: RuntimeDefault
containers:
- name: app
  image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
  imagePullPolicy: {{ .Values.image.pullPolicy }}
  ports:
    - name: http
      containerPort: {{ .Values.containerPort }}
  envFrom:
    - configMapRef:
        name: {{ include "status-lab.fullname" . }}-config
      {{- if .Values.existingSecret }}
    - secretRef:
        name: {{ .Values.existingSecret | quote }}
      {{- end }}
  resources:
    {{- toYaml .Values.resources | nindent 12 }}
  securityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities:
      drop: ["ALL"]
  startupProbe:
    httpGet:
      path: /healthz
      port: http
      periodSeconds: 2
      failureThreshold: 30
  readinessProbe:
    httpGet:
      path: /readyz
      port: http
      periodSeconds: 5
  livenessProbe:
    httpGet:
      path: /healthz
      port: http
      periodSeconds: 10

```

A.11 chart/status-lab/templates/service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: {{ include "status-lab.fullname" . }}
spec:
  type: ClusterIP
  selector:
    {{- include "status-lab.selectorLabels" . | nindent 4 }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: http

```

A.12 chart/status-lab/templates/serviceaccount.yaml

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: {{ include "status-lab.fullname" . }}
automountServiceAccountToken: false

```

A.13 chart/status-lab/values.schema.json

```

{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": [
    "replicaCount",
    "image",
    "containerPort",
    "service",
    "config"
  ],
  "properties": {
    "replicaCount": {
      "type": "integer",
      "minimum": 1
    },
    "containerPort": {
      "type": "integer",
      "minimum": 1,
      "maximum": 65535
    },
    "service": {
      "type": "object",
      "required": [
        "port"
      ],
      "properties": {
        "port": {
          "type": "integer",
          "minimum": 1,
          "maximum": 65535
        }
      }
    },
    "image": {
      "type": "object",
      "required": [
        "repository",
        "tag"
      ],
      "properties": {
        "repository": {
          "type": "string",
          "minLength": 1
        },
        "tag": {
          "type": "string",
          "minLength": 1
        }
      }
    },
    "config": {
      "type": "object",
      "additionalProperties": {
        "type": "string"
      },
      "propertyNames": {
        "not": {
          "enum": [
            "HOST",
            "PORT"
          ]
        }
      }
    },
    "existingSecret": {
      "type": "string"
    }
  }
}

```

A.14 chart/status-lab/values.yaml

```

replicaCount: 2
image:
  repository: status-lab
  tag: "0.1.0"
  pullPolicy: IfNotPresent
service:
  port: 80
containerPort: 3000
config:
  AUTH_MODE: "disabled"
  ALLOW_UNAUTHENTICATED_LOCAL_LAB: "true"
  ALLOWED_HOSTS: "127.0.0.1:3000,localhost:3000,workbook-status-lab,workbook-status-
lab.workbook.svc.cluster.local"
existingSecret: ""
resources:
  requests:
    cpu: 100m
    memory: 64Mi
  limits:
    cpu: 500m
    memory: 256Mi

```

A.15 chart/values-auth.example.yaml

```

# REVIEW ONLY: reserved domains must be replaced by approved real configuration.
# Do not deploy until issuer, keys, client registration and HTTPS path are reviewed.
replicaCount: 2
config:
  AUTH_MODE: "jwt"
  ALLOW_UNAUTHENTICATED_LOCAL_LAB: "false"
  RESOURCE_URL: "https://status.example.invalid/mcp"
  OAUTH_ISSUER: "https://idp.example.invalid/realms/workbook"
  OAUTH_JWKS_URL: "https://idp.example.invalid/realms/workbook/protocol/openid-connect/certs"
  ALLOWED_HOSTS: "status.example.invalid,workbook-status-lab,workbook-status-
lab.workbook.svc.cluster.local"
  ALLOWED_ORIGINS: ""
# Verification of asymmetric JWTs does not require an OAuth client secret.
existingSecret: ""

```

A.16 chart/values-dev.yaml

```

replicaCount: 1

```

A.17 k8s/status-lab.yaml

```

apiVersion: v1
kind: Namespace
metadata:
  name: workbook
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: status-lab
  namespace: workbook
automountServiceAccountToken: false
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: status-lab-config
  namespace: workbook
data:
  HOST: "0.0.0.0"
  PORT: "3000"
  AUTH_MODE: "disabled"

```

```
ALLOW_UNAUTHENTICATED_LOCAL_LAB: "true"
ALLOWED_HOSTS: "127.0.0.1:3000,localhost:3000,status-lab,status-lab.workbook.svc.cluster.local"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: status-lab
  namespace: workbook
spec:
  replicas: 2
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: status-lab
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: status-lab
    spec:
      serviceAccountName: status-lab
      automountServiceAccountToken: false
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        runAsGroup: 1000
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: app
          image: status-lab:0.1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 3000
          envFrom:
            - configMapRef:
                name: status-lab-config
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 256Mi
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          startupProbe:
            httpGet:
              path: /healthz
              port: http
              periodSeconds: 2
              failureThreshold: 30
          readinessProbe:
            httpGet:
              path: /readyz
              port: http
              periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /healthz
              port: http
              periodSeconds: 10
```

```

---
apiVersion: v1
kind: Service
metadata:
  name: status-lab
  namespace: workbook
spec:
  selector:
    app: status-lab
  ports:
    - name: http
      port: 80
      targetPort: http
  type: ClusterIP

```

A.18 kyma/apirule-v2.example.yaml

```

# REVIEW ONLY. This is not an approved or tested public deployment.
# Confirm installed API/module version, sidecar enrollment, internal access policy,
# DNS/TLS, gateway ownership, token forwarding and 401 metadata behavior first.
# The gateway below is a fictional placeholder, not a known cluster resource.
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: status-lab
  namespace: workbook
spec:
  gateway: approved-system/approved-gateway
  hosts:
    - status.example.invalid
  service:
    name: workbook-status-lab
    namespace: workbook
    port: 80
  rules:
    - path: /.well-known/oauth-protected-resource
      methods: [GET]
      noAuth: true
    - path: /mcp
      methods: [POST]
      jwt:
        authentications:
          - issuer: https://idp.example.invalid/realms/workbook
            jwksUri: https://idp.example.invalid/realms/workbook/protocol/openid-connect/certs
        authorizations:
          - audiences: [https://status.example.invalid/mcp]
            requiredScopes: [mcp:connect]
# The app independently validates bearer tokens and enforces status:read.
# Review how the gateway preserves the bearer token or use an approved identity
# propagation design. This example does not implement browser CORS or streaming.

```

A.19 milestones/01-http/.dockerignore

```

.git
node_modules
*.log
.env

```

A.20 milestones/01-http/Dockerfile

```

FROM node:22-bookworm-slim
WORKDIR /app
COPY --chown=node:node server.mjs ./
COPY --chown=node:node data ./data
USER node
ENV HOST=0.0.0.0 PORT=3000
EXPOSE 3000
CMD ["node", "server.mjs"]

```

A.21 milestones/01-http/data/status.json

```
{
  "api": "operational",
  "retrieval": "degraded",
  "worker": "operational"
}
```

A.22 milestones/01-http/server.mjs

```
import http from 'node:http';
import { readFile } from 'node:fs/promises';
const host = process.env.HOST ?? '127.0.0.1';
const port = Number(process.env.PORT ?? '3000');
if (!Number.isInteger(port) || port < 1 || port > 65535) {
  throw new Error('PORT must be an integer between 1 and 65535');
}
const file = new URL('./data/status.json', import.meta.url);
const data = JSON.parse(await readFile(file, 'utf8'));
let ready = true;
const server = http.createServer((req, res) => {
  const url = new URL(req.url ?? '/', 'http://localhost');
  res.setHeader('Content-Type', 'application/json');
  if (req.method !== 'GET') {
    res.writeHead(405, { Allow: 'GET' }).end('{}'); return;
  }
  if (url.pathname === '/healthz') {
    res.end(JSON.stringify({ alive: true })); return;
  }
  if (url.pathname === '/readyz') {
    res.writeHead(ready ? 200 : 503).end(JSON.stringify({ ready })); return;
  }
  if (url.pathname === '/status') {
    const name = url.searchParams.get('service');
    if (!name || !Object.hasOwn(data, name)) {
      res.writeHead(404).end(JSON.stringify({ error: 'unknown_service' })); return;
    }
    res.end(JSON.stringify({ service: name, status: data[name], source: 'mock' }));
    return;
  }
  res.writeHead(404).end('{}');
});
server.listen(port, host, () => {
  console.error(JSON.stringify({ event: 'listening', host, port }));
});
function stop(signal) {
  if (!ready) return;
  ready = false;
  console.error(JSON.stringify({ event: 'shutdown', signal }));
  const deadline = setTimeout(() => process.exit(1), 10000);
  deadline.unref();
  server.close(() => { clearTimeout(deadline); process.exit(0); });
}
process.on('SIGTERM', () => stop('SIGTERM'));
process.on('SIGINT', () => stop('SIGINT'));
```

A.23 service/.dockerignore

```
.git
node_modules
dist
.env
.env.*
*.log
coverage
```

A.24 service/Dockerfile

```
FROM node:22-bookworm-slim AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build
RUN npm prune --omit=dev

FROM node:22-bookworm-slim AS runtime
WORKDIR /app
ENV NODE_ENV=production HOST=0.0.0.0 PORT=3000
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/dist ./dist
COPY --chown=node:node package.json ./
COPY --chown=node:node data ./data
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

A.25 service/data/status.json

```
{
  "api": "operational",
  "retrieval": "degraded",
  "worker": "operational"
}
```

A.26 service/package.json

```
{
  "name": "status-lab",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "engines": {
    "node": ">=22.16.0 <23"
  },
  "scripts": {
    "build": "tsc",
    "start": "node dist/server.js",
    "client": "node dist/client.js",
    "oauth-client": "node dist/oauth-client.js",
    "test": "npm run build && node --test dist/*.test.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "1.30.0",
    "zod": "3.25.76",
    "jose": "6.1.3"
  },
  "devDependencies": {
    "typescript": "5.8.3",
    "@types/node": "22.15.30"
  }
}
```

A.27 service/src/auth.ts

```
import { createRemoteJWKSet, jwtVerify } from 'jose';
import type { IncomingMessage } from 'node:http';
import { config } from './config.js';
export interface Principal { subject: string; scopes: string[]; }
export class AccessFailure extends Error {
  constructor(public status: 401 | 403, public code: string) { super(code); }
}
const jwks = config.mode === 'jwt'
```

```

    ? createRemoteJWKSet(new URL(config.jwks), { timeoutDuration: 5000 })
    : undefined;
export async function authenticate(req: IncomingMessage): Promise<Principal> {
  if (config.mode === 'disabled') {
    return { subject: 'explicit-local-lab', scopes: ['mcp:connect', 'status:read'] };
  }
  const match = /^Bearer ([^\s+]$)/i.exec(req.headers.authorization ?? '');
  if (!match?.[1]) throw new AccessFailure(401, 'missing_token');
  try {
    const { payload } = await jwtVerify(match[1], jwks!, {
      issuer: config.issuer, audience: config.resource,
      algorithms: ['RS256'], requiredClaims: ['exp', 'iat', 'sub'],
      clockTolerance: 5,
    });
    if (typeof payload.sub !== 'string') throw new Error('Invalid subject');
    const scopes = typeof payload.scope === 'string'
      ? payload.scope.split(/\s+/).filter(Boolean) : [];
    if (!scopes.includes('mcp:connect')) throw new AccessFailure(403, 'insufficient_scope');
    return { subject: payload.sub, scopes };
  } catch (error) {
    if (error instanceof AccessFailure) throw error;
    // Log only a bounded error category: not the token, claims, or request headers.
    console.error(JSON.stringify({ event: 'token_rejected',
      category: error instanceof Error ? error.name : 'unknown' }));
    throw new AccessFailure(401, 'invalid_token');
  }
}

```

A.28 service/src/client.ts

```

import { Client } from '@modelcontextprotocol/sdk/client/index.js';
import { StreamableHTTPClientTransport } from '@modelcontextprotocol/sdk/client/streamableHttp.js';
export async function runClient(token?: string) {
  const url = new URL(process.env.MCP_URL ?? 'http://127.0.0.1:3000/mcp');
  // This teaching client is loopback-only; no redirect-based credential forwarding.
  if (!['http:', 'https:'].includes(url.protocol) || url.username || url.password ||
    url.hash || !['localhost', '127.0.0.1'].includes(url.hostname)) {
    throw new Error('Use a reviewed OAuth-capable client for remote endpoints');
  }
  const transport = new StreamableHTTPClientTransport(url, {
    requestInit: { redirect: 'error', headers: token ? { Authorization: `Bearer ${token}` } : {} },
  });
  const client = new Client({ name: 'status-lab-client', version: '0.1.0' });
  try {
    await client.connect(transport);
    console.log(JSON.stringify(await client.listTools(), null, 2));
    console.log(JSON.stringify(await client.callTool({ name: 'read_status',
      arguments: { service: process.env.SERVICE_NAME ?? 'api' } }), null, 2));
  } finally { await client.close(); }
}
if (process.argv[1]?.endsWith('/client.js') || process.argv[1]?.endsWith('\\client.js')) {
  await runClient();
}

```

A.29 service/src/config.ts

```

function required(name: string): string {
  const value = process.env[name];
  if (!value) throw new Error(`Missing configuration: ${name}`);
  return value;
}
function trustedUrl(name: string): string {
  const value = required(name);
  const url = new URL(value);
  const loopback = ['localhost', '127.0.0.1', ':::1'].includes(url.hostname);
  if (url.username || url.password || url.hash ||
    (url.protocol !== 'https:' && !(url.protocol === 'http:' && loopback))) {
    throw new Error(`${name} must be HTTPS, or HTTP loopback for this local lab`);
  }
}

```

```

    return value;
  }
  const port = Number(process.env.PORT ?? '3000');
  if (!Number.isInteger(port) || port < 1 || port > 65535) throw new Error('Invalid PORT');
  const mode = process.env.AUTH_MODE ?? 'jwt';
  if (mode !== 'jwt' && mode !== 'disabled') throw new Error('Invalid AUTH_MODE');
  if (mode === 'disabled' && process.env.ALLOW_UNAUTHENTICATED_LOCAL_LAB !== 'true') {
    throw new Error('Disabled auth needs explicit local-lab opt-in');
  }
  export const config = {
    host: process.env.HOST ?? '127.0.0.1', port, mode,
    resource: process.env.RESOURCE_URL ?? 'http://127.0.0.1:3000/mcp',
    allowedOrigins: (process.env.ALLOWED_ORIGINS ?? '').split(',').filter(Boolean),
    allowedHosts: (process.env.ALLOWED_HOSTS ?? `127.0.0.1:${port},localhost:${port}`).split(',').filter(Boolean),
    issuer: mode === 'jwt' ? trustedUrl('OAUTH_ISSUER') : '',
    jwks: mode === 'jwt' ? trustedUrl('OAUTH_JWKS_URL') : '',
  };
  // resource is a deployment identifier, never taken from an untrusted Host header.
  const resourceUrl = new URL(config.resource);
  if (resourceUrl.hash || resourceUrl.username || resourceUrl.password ||
    ![ 'http:', 'https:' ].includes(resourceUrl.protocol) ||
    (resourceUrl.protocol === 'http:' &&
      ![ 'localhost', '127.0.0.1', '::1' ].includes(resourceUrl.hostname))) {
    throw new Error('Invalid RESOURCE_URL');
  }
}

```

A.30 service/src/domain.test.ts

```

import test from 'node:test';
import assert from 'node:assert/strict';
import { parseTable, readStatus, hasScope } from './domain.js';
test('valid data yields a read-only mock status', () => {
  const table = parseTable({ api: 'operational' });
  assert.deepEqual(readStatus(table, 'api'), {
    service: 'api', status: 'operational', source: 'mock'
  });
  assert.equal(Object.isFrozen(table), true);
});
test('unknown names are not invented', () => {
  assert.equal(readStatus(parseTable({}), 'missing'), undefined);
});
test('runtime shape validation rejects malformed data', () => {
  for (const value of [null, [], 'wrong', { api: 3 }, { 'Bad Name': 'degraded' }]) {
    assert.throws(() => parseTable(value));
  }
});
test('scope matching is exact, not substring matching', () => {
  assert.equal(hasScope(['status:read'], 'status:read'), true);
  assert.equal(hasScope(['status:read-all'], 'status:read'), false);
  assert.equal(hasScope([], 'status:read'), false);
});

```

A.31 service/src/domain.ts

```

export type Status = 'operational' | 'degraded';
export type StatusTable = Readonly<Record<string, Status>>;
export function parseTable(input: unknown): StatusTable {
  if (typeof input !== 'object' || input === null || Array.isArray(input)) {
    throw new Error('Status data must be an object');
  }
  const result: Record<string, Status> = Object.create(null);
  for (const [name, status] of Object.entries(input)) {
    if (!/^[a-z][a-z0-9-]{0,31}$/.test(name) ||
      (status !== 'operational' && status !== 'degraded')) {
      throw new Error('Invalid status data entry');
    }
    result[name] = status;
  }
}

```

```

    return Object.freeze(result);
}
export function readStatus(table: StatusTable, name: string) {
    const status = table[name];
    return status === undefined ? undefined : { service: name, status, source: 'mock' as const };
}
export function hasScope(scopes: readonly string[], required: string): boolean {
    return scopes.includes(required);
}

```

A.32 service/src/mcp.ts

```

import { McpServer } from '@modelcontextprotocol/sdk/server/mcp.js';
import { z } from 'zod/v3';
import { hasScope, readStatus, type StatusTable } from './domain.js';
import type { Principal } from './auth.js';
export function makeMcp(table: StatusTable, principal: Principal): McpServer {
    const server = new McpServer({ name: 'status-lab', version: '0.1.0' });
    server.registerTool('read_status', {
        title: 'Read mock service status',
        description: 'Returns mock status, not employer or production data.',
        inputSchema: { service: z.string().regex(/^[a-z][a-z0-9_-]{0,31}$/) },
        annotations: { readOnlyHint: true, destructiveHint: false, idempotentHint: true },
    }, async ({ service }) => {
        if (!hasScope(principal.scopes, 'status:read')) {
            return { isError: true, content: [{ type: 'text', text: 'permission_denied' }] };
        }
        const result = readStatus(table, service);
        if (!result) return {
            isError: true, content: [{ type: 'text', text: 'unknown_service' }]
        };
        return { content: [{ type: 'text', text: JSON.stringify(result) }] };
    });
    return server;
}

```

A.33 service/src/oauth-client.ts

```

// LOCAL TEACHING CLIENT ONLY: one trusted issuer, no refresh/token persistence/DCR.
import http from 'node:http';
import { randomBytes, createHash } from 'node:crypto';
import { runClient } from './client.js';
const issuer = 'http://127.0.0.1:8080/realms/workbook';
const redirect = 'http://127.0.0.1:8787/callback';
const resource = 'http://127.0.0.1:3000/mcp';
const clientId = 'workbook-native';
const scope = process.env.SCOPE ?? 'mcp:connect status:read';
// Local negative-test option: retain the token only in memory, then test expiry.
const tokenDelay = Number(process.env.TOKEN_DELAY_MS ?? '0');
if (!Number.isInteger(tokenDelay) || tokenDelay < 0 || tokenDelay > 180000) {
    throw new Error('TOKEN_DELAY_MS must be an integer from 0 through 180000');
}
const state = randomBytes(32).toString('base64url');
const verifier = randomBytes(32).toString('base64url');
const challenge = createHash('sha256').update(verifier).digest('base64url');
const response = await fetch(`${issuer}/.well-known/openid-configuration`, {
    signal: AbortSignal.timeout(5000), redirect: 'error',
});
if (!response.ok) throw new Error(`Discovery failed: ${response.status}`);
const metadata: unknown = await response.json();
if (!metadata || typeof metadata !== 'object') throw new Error('Invalid discovery');
const m = metadata as Record<string, unknown>;
if (m.issuer !== issuer) throw new Error('Issuer mismatch');
function endpoint(name: string): string {
    const value = m[name];
    if (typeof value !== 'string' || !value.startsWith(`${issuer}/`)) {
        throw new Error(`Untrusted ${name}`);
    }
    return value;
}

```

```

}
const authorization = new URL(endpoint('authorization_endpoint'));
const tokenEndpoint = endpoint('token_endpoint');
for (const [key, value] of Object.entries({ response_type: 'code', client_id: clientId,
  redirect_uri: redirect, scope, resource, state, code_challenge: challenge,
  code_challenge_method: 'S256' })) authorization.searchParams.set(key, value);
const code = await new Promise<string>((resolve, reject) => {
  const callback = http.createServer((req, res) => {
    const url = new URL(req.url ?? '/', redirect);
    if (url.pathname !== '/callback') { res.writeHead(404).end(); return; }
    const incomingIssuer = url.searchParams.get('iss');
    const issuerRequired = m.authorization_response_iss_parameter_supported === true;
    if (req.headers.host !== '127.0.0.1:8787' ||
      url.searchParams.get('state') !== state ||
      (issuerRequired && incomingIssuer !== issuer) ||
      (incomingIssuer !== null && incomingIssuer !== issuer)) {
      res.writeHead(400).end('Invalid callback'); return;
    }
    const value = url.searchParams.get('code');
    clearTimeout(timer); callback.close();
    if (!value || url.searchParams.has('error')) {
      res.writeHead(400).end('Authorization failed'); reject(new Error('Authorization failed'));
      return;
    }
    res.end('Authorization complete. Return to the terminal.');
```

A.34 service/src/server.ts

```

import http, { type IncomingMessage, type ServerResponse } from 'node:http';
import { readFile } from 'node:fs/promises';
import { randomUUID } from 'node:crypto';
import { StreamableHTTPServerTransport } from '@modelcontextprotocol/sdk/server/streamableHttp.js';
import { parseTable } from './domain.js';
import { config } from './config.js';
import { authenticate, AccessFailure } from './auth.js';
import { makeMcp } from './mcp.js';
const table = parseTable(JSON.parse(await readFile(
  new URL('../data/status.json', import.meta.url), 'utf8')));
let ready = true;
function json(res: ServerResponse, code: number, value: unknown) {
  res.writeHead(code, { 'Content-Type': 'application/json', 'Cache-Control': 'no-store' });
  res.end(JSON.stringify(value));
}
async function route(req: IncomingMessage, res: ServerResponse): Promise<void> {
  const requestId = randomUUID();
  res.setHeader('X-Request-ID', requestId);
  const path = new URL(req.url ?? '/', 'http://localhost').pathname;
  // No sensitive data in probes. They are not published by the Kyma example.
```

```

    if (path === '/healthz' && req.method === 'GET') { json(res, 200, { alive: true }); return; }
    if (path === '/readyz' && req.method === 'GET') { json(res, ready ? 200 : 503, { ready });
return; }
    if (!config.allowedHosts.includes(req.headers.host ?? '')) {
        json(res, 403, { error: 'untrusted_host' }); return;
    }
    if (req.headers.origin && !config.allowedOrigins.includes(req.headers.origin)) {
        json(res, 403, { error: 'untrusted_origin' }); return;
    }
    if (path === '/.well-known/oauth-protected-resource' && req.method === 'GET') {
        if (config.mode !== 'jwt') { json(res, 404, {}); return; }
        json(res, 200, { resource: config.resource, authorization_servers: [config.issuer],
            scopes_supported: ['mcp:connect', 'status:read'], bearer_methods_supported: ['header'] });
        return;
    }
    if (path !== '/mcp') { json(res, 404, {}); return; }
    if (!ready) { json(res, 503, { error: 'draining' }); return; }
    if (req.method !== 'POST') {
        res.setHeader('Allow', 'POST'); json(res, 405, { error: 'method_not_allowed' }); return;
    }
    try {
        const principal = await authenticate(req);
        const mcp = makeMcp(table, principal); // request-scoped identity; no global current user
        const transport = new StreamableHTTPServerTransport({
            sessionIdGenerator: undefined, enableJsonResponse: true,
        });
        res.on('close', () => { void mcp.close().catch(() => undefined); });
        await mcp.connect(transport);
        await transport.handleRequest(req, res);
    } catch (error) {
        if (res.headersSent) { res.end(); return; }
        if (error instanceof AccessFailure) {
            const metadata = new URL('/.well-known/oauth-protected-resource', config.resource).href;
            const detail = error.status === 403 ? ', error="insufficient_scope", scope="mcp:connect"'
                : error.code === 'invalid_token' ? ', error="invalid_token"' : '';
            res.setHeader('WWW-Authenticate', `Bearer resource_metadata="${metadata}"${detail}`);
            json(res, error.status, { error: error.code, requestId });
        } else {
            console.error(JSON.stringify({ event: 'request_failed', requestId }));
            json(res, 500, { error: 'internal_error', requestId });
        }
    }
}
const server = http.createServer((req, res) => {
    // Catch failures outside the narrower authentication/MCP boundary too.
    void route(req, res).catch(() => {
        console.error(JSON.stringify({ event: 'unhandled_request_failure' }));
        if (!res.headersSent) json(res, 500, { error: 'internal_error' });
        else res.end();
    });
});
server.requestTimeout = 15000;
server.headersTimeout = 10000;
server.listen(config.port, config.host, () => {
    console.error(JSON.stringify({ event: 'listening', host: config.host,
        port: config.port, authMode: config.mode }));
});
function stop(signal: string) {
    if (!ready) return;
    ready = false;
    console.error(JSON.stringify({ event: 'shutdown', signal }));
    const timer = setTimeout(() => process.exit(1), 10000); timer.unref();
    server.close(() => { clearTimeout(timer); process.exit(0); });
}
process.on('SIGTERM', () => stop('SIGTERM'));
process.on('SIGINT', () => stop('SIGINT'));

```

A.35 service/tsconfig.json

```
{
```

```
"compilerOptions": {
  "target": "ES2022",
  "module": "NodeNext",
  "moduleResolution": "NodeNext",
  "strict": true,
  "noUncheckedIndexedAccess": true,
  "esModuleInterop": true,
  "skipLibCheck": true,
  "rootDir": "src",
  "outDir": "dist"
},
"include": [
  "src/**/*.ts"
]
}
```